

BANGLADESH UNIVERSITY OF ENGINEERING AND TECHNOLOGY

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

CSE 402: Numerical Analysis, Simulation and Modeling Sessional (Level 4, Term 1)

Kinetic-KAN: Solver-Aware Neural Dynamics

Project Final Report

Kolmogorov-Arnold Network Neural ODEs — cross-domain stability fixes, solver-order effects, basis-function design, and a damping–solver stability map, verified against the project’s own training runs, checkpoints and figures.

Submitted By

- 2105032** Nawriz Ahmed Turjo – Adjoint vs. Autograd Profiling
- 2105033** Abhishek Roy – Gradient Norm Dynamics
- 2105048** Shams Hossain Simanto – Learnable Basis Blending
- 2105055** Md. Abrar Jahin – Stiffness–Solver Stability Analysis
- 2105043** Monjur Hossain Khan (Shovon) – Sparse Regression & Epidemic Fit

September 24, 2026

Code, saved results and checkpoints: <https://github.com/NawrizTurjo/kinetic-kan>
(training results as of commit 75acec5; post-hoc analysis and figure scripts added with this revision)

Executive Summary

This report documents a Kolmogorov-Arnold Network Neural ODE (KAN-ODE) [11, 14, 3] project that reproduces the base paper’s Lotka-Volterra benchmark, extends it to two additional dynamical systems (a damped pendulum and an SIR epidemic model), diagnoses and fixes two genuine cross-domain training failures, and runs five independent follow-on studies probing solver order, differentiation method, basis-function design, damping (originally intended as a stiffness probe), and comparison against sparse symbolic regression.

Headline results

- At the 10,000-epoch budget, KAN-ODE’s extrapolation error is essentially flat out to $t = 28$ ($1.1\times$ degradation), while a parameter-matched MLP degrades $8.3\times$ — a $40.6\times$ KAN-ODE advantage at the longest horizon. **This advantage depends on the budget:** retrained to 50,000 epochs (with gradient clipping, on a different platform, Section 12), the same MLP extrapolates $6.8\times$ (near) to $10.8\times$ (far) *better* than KAN-ODE.
- The paper’s claim that KAN-ODE needs $10\times$ fewer epochs than an MLP is not reproduced: the parameter-matched MLP reaches 10^{-3} training loss $4.1\times$ sooner. The paper’s literal $[2, 50, 2]$ tanh MLP fails to train under this recipe at any budget tested, and an audit of the paper’s code identifies a learning-rate and initialization difference as the likely cause.
- The pendulum failure (extrapolation $R^2 = -4.64$ on the common test window, -1.318 on its own longer one) was fixed by extending the training window, to $R^2 = +0.647$. SIR’s divergence traced to a roughly $18\text{--}19\times$ unit-scale mismatch at initialization: a change of time units and a conservation projection remove the divergence (R^2 from -20.97 to -0.145), and a vanishing-field gate on the infected fraction, a physical prior, brings extrapolation R^2 to $+0.9981$.
- Adjoint sensitivity already uses less memory than direct autograd at every trajectory length tested (17.1 MB against 18.6–21.5 MB for the two training runs, and up to ~ 160 MB at 3,201 points), but costs $1.9\text{--}2.1\times$ wall-clock — direct autograd remains the correct default at this project’s scale.
- Gradient-norm roughness does not explain Euler’s convergence floor (Spearman $\rho = -0.088$, $p = 0.87$); shared, order-independent spike bursts dominate all six solvers’ gradient traces.
- A learnable spline/RBF basis blend never beats the better pure basis on Lotka-Volterra (it is $1.7\text{--}2.1\times$ worse there), or pure RBF on the pendulum, where no pure B-spline run was made. On the pendulum it overfits badly past $\sim 3,000$ epochs, although measured directly the B-spline supplies only $\sim 1\%$ of its basis output.
- The damping sweep never reaches a stiff regime (timescale ratio at most 4.3), and no run diverged, as expected from the small step size ($|h\lambda| \leq 0.30$), which keeps every solver’s discrete map close to the learned field’s own flow; the only predicted instability is a mild numerical anti-damping for Euler at $\mu = 0.1$. Its failures at $\mu \in \{0.5, 1.0, 2.0\}$ trace to the random seed rather than the solver, apart from one Euler-specific failure at $\mu = 1.0$. At $\mu = 2.0$, Newton’s method confirms a genuine, attracting spurious equilibrium of the learned field, shared by all four solvers.
- Sparse regression with finite-difference derivatives degrades far less under noise than KAN-ODE ($9\times$ vs. $1540\times$, $\sigma = 0 \rightarrow 0.1$, from a much higher noise-free error) but is less accurate at every noise level. On noise-free data, given exact derivatives, it recovers the true Lotka-Volterra equation outright, so there the accuracy gap comes from the derivative estimate; whether it does under noise was not tested.
- On a synthetic, non-mechanistic epidemic curve, two of SIR’s three stability fixes transfer, including the one predicted not to.

All seven basis functions, six integrators and the training loop were implemented for this project and verified before use. Every integrator reaches its theoretical order of convergence on a problem with a known solution, and 249 of 250 automated tests pass (the remaining one, which needs a GPU, is skipped). Every number and figure in this report is traceable to a saved training record, a regenerable figure, or a post-hoc analysis script of a saved checkpoint (Section 4). The main limitations are that almost every result is single-seed, that the budget reversal coincides with a change in gradient clipping and in platform (operating system and PyTorch version), and that three differences from the paper’s recipe (learning rate, initial-weight scale, RBF width) have been identified but not yet measured.

Contents

Executive Summary	1
1 Introduction	4
1.1 Background: Kolmogorov-Arnold Networks as vector fields	4
1.2 Architecture, and what was built rather than imported	4
1.3 What the base paper fixed, and what this project varies	4
1.4 Why training itself needs to be treated as an object of study	5
1.5 Structure of this report	6
2 Datasets and Experimental Setup	6
2.1 The four dynamical systems	6
2.2 Ground truth, sampling and train/test splits	7
2.3 Models and training settings	7
3 Numerical Implementation	8
3.1 Where the implementation comes from	8
3.2 One KAN layer	9
3.3 The integrator	11
3.4 Differentiating through the solver	12
3.5 Structural modifications of the field	12
4 Verification and Testing Framework	13
4.1 Solver verification against a known solution	13
4.2 The automated test suite	14
4.3 From a configuration to a table entry	14
4.4 Metrics	15
5 Baseline Recipe and Evaluation Protocol	15
5.1 Evaluation protocol	16
6 Benchmark Ablation and Cross-Domain Stability Analysis	16
6.1 Solver, basis and architecture ablation	16
6.2 Testing the paper’s KAN-versus-MLP claims	19
6.3 Robustness to observation noise	23
6.4 Diagnosing and fixing two cross-domain failures	23
6.5 Extended extrapolation and its budget dependence	26
6.6 Pendulum energy dissipation: an independent physical check	28
7 Adjoint Sensitivity vs. Direct Autograd: Memory and Wall-Clock Cost	29
8 Gradient-Norm Dynamics and Solver Order	30
9 Learnable Basis-Function Blending	32
10 Damping–Solver Stability and Loss-Landscape Analysis	34
10.1 The stability question resolves quickly	35
10.2 What actually drives the failures: initialization	35
10.3 The mechanism: a genuine spurious equilibrium	36
10.4 Weight-space loss landscape: an independent measurement	36
11 Sparse Regression Comparison and Non-Mechanistic Epidemic Validation	41
11.1 Noise robustness: KAN-ODE vs. sparse symbolic regression	41
11.2 Edge pruning: testing whether a term-by-term comparison is even possible	43

11.3	Testing whether SIR’s fixes transfer to a non-mechanistic epidemic curve	44
12	Validation: Epoch Budget, Implementation Audit and Paper Claims	45
12.1	Epoch-budget sensitivity	46
12.2	Implementation audit against the base paper	48
12.3	The paper’s claims against this project’s measurements	48
13	Discussion	49
14	Conclusion	51
14.1	Limitations	52
A	Butcher Tableaus of the Fifth-Order Solvers	54
B	Damping-Study Trajectories	55

1 Introduction

1.1 Background: Kolmogorov-Arnold Networks as vector fields

A standard Neural ODE [3] approximates an unknown dynamical system $\dot{\mathbf{u}}(t) = \mathbf{F}(\mathbf{u}(t))$ by a learned vector field $\mathbf{f}_\theta(\mathbf{u}(t))$, parameterized by a multilayer perceptron: fixed nonlinear activations on each node, with all learnable structure in the linear weights, $\mathbf{y} = \sigma(\mathbf{W}\mathbf{x} + \mathbf{b})$. A Kolmogorov-Arnold Network (KAN) [14] inverts this: every *edge* carries its own learnable univariate function $\phi(x)$, and each node simply sums its incoming edges, $y_j = \sum_i \phi_{i,j}(x_i)$, following the Kolmogorov-Arnold representation theorem – any continuous multivariate function on a bounded domain can be written as a finite composition of continuous univariate functions and addition,

$$f(\mathbf{x}) = \sum_{q=1}^{2n+1} \Phi_q \left(\sum_{p=1}^n \phi_{q,p}(x_p) \right), \quad (1)$$

so that no fixed nonlinearity needs to be chosen in advance – the network learns what shape each univariate function should take. In practice each edge function is represented as a fixed smooth residual plus a learnable expansion in a chosen basis,

$$\phi(x) = w_b b(x) + \sum_{i=1}^G c_i B_i(u), \quad u = \tanh x, \quad b(x) = \text{silu}(x) = \frac{x}{1 + e^{-x}}, \quad (2)$$

where B_i are the G basis functions centred on a grid of G points in $[-1, 1]$, evaluated at the normalized input u (Section 3), c_i are their trainable coefficients, and the trainable weight w_b sets how much of the fixed residual $b(x)$ each edge keeps. Each edge therefore has $G + 1$ parameters. (The original KAN implementation uses $G + k$ B-spline functions on G intervals and a separate scale on the expansion; in the formulation used here that scale is absorbed into the c_i .) The choice of B_i is a design decision, not a mathematical necessity: a Gaussian radial basis function (RBF) grid, $B_i(u) = \exp(-(u - z_i)^2 / 2\delta^2)$ with grid spacing δ , gives each edge smooth global support, while a degree- k B-spline built by the Cox-de Boor recursion gives each edge compact local support, zero outside its own knot span. KAN-ODEs [11] put this construction inside a Neural ODE’s vector field: $\dot{\mathbf{u}} = \mathbf{f}_\theta(\mathbf{u})$ with every edge of $\mathbf{f}_\theta$ built from Equation 2, and the resulting trajectory obtained by numerically integrating that field forward in time.

1.2 Architecture, and what was built rather than imported

The reference KAN-ODE of this report, the base paper’s Lotka-Volterra model, is a two-layer network with widths $[2, 10, 2]$, each edge carrying the expansion of Equation 2 over a grid of $G = 5$ basis functions, wrapped inside an explicit Runge-Kutta integrator that advances the state and hands the resulting trajectory to the loss. The other systems use the same structure at a different size (Table 2). Figure 1 shows the structure schematically, and marks the three places where this report changes it.

With $G = 5$ RBF centers per edge, layer one ($2 \rightarrow 10$) contributes $10 \times (2 \times 5 + 2) = 120$ parameters and layer two ($10 \rightarrow 2$) contributes $2 \times (10 \times 5 + 10) = 120$ parameters, for the 240 total the base paper reports. The reference Python KAN library, `pykan` [14], represents every edge with its own B-spline object and refits its grid during training. The base paper does not use it: its Julia code [12] builds the network from the `KDense` layer of `KolmogorovArnold.jl` [15], which evaluates every edge’s basis functions in one array operation followed by a single matrix product. This project ports that layer to PyTorch, which makes every basis substituted into Equation 2 a one-function change evaluated by an identical code path (Section 3, Algorithm 1). The explicit Runge-Kutta integrators, the training loop and all diagnostics were written for this project rather than imported, so every basis and solver could be instrumented identically. The base paper runs its integration loop in Julia’s SciML ecosystem, while this project runs it in PyTorch, so wall-clock figures reported later are comparable across this project’s own runs but not directly against the paper’s reported times.

1.3 What the base paper fixed, and what this project varies

The base paper reports two central claims on the Lotka-Volterra predator-prey system: a KAN-ODE with 240 parameters extrapolates Lotka-Volterra’s closed orbit accurately from a short training window, and it reaches a target training loss in roughly 10^4 epochs against a parameter-matched multilayer-perceptron Neural ODE’s 10^5 – a claimed tenfold training-speed advantage at 252 parameters. To obtain these results, the paper commits to three specific design decisions that this project treats as free variables rather than fixed choices: the

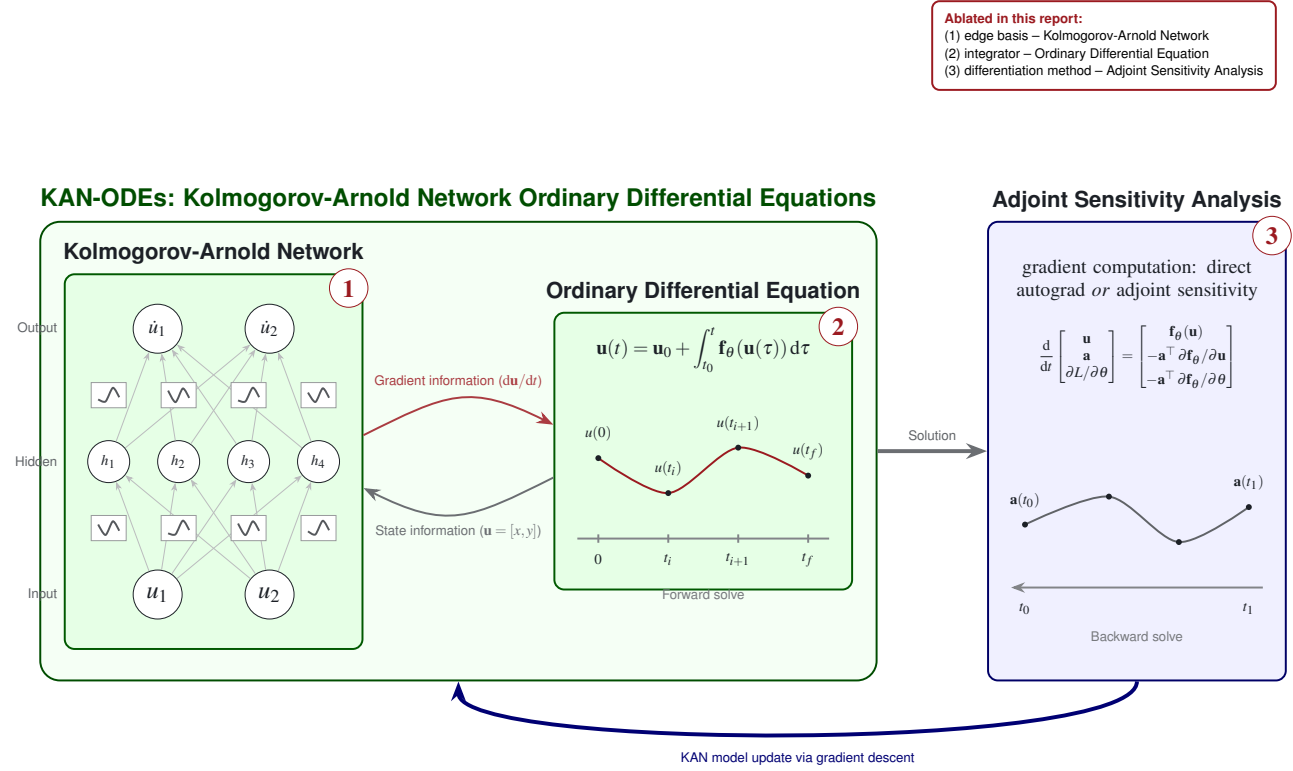


Figure 1. The KAN-ODE training loop, redrawn in the style of the base paper’s own architecture figure [11]. The legend at top right lists this report’s three ablated components and which block of the diagram each corresponds to.

integrator (an adaptive fifth-order Runge-Kutta method, Tsit5), the **basis** (Gaussian RBF), and the **differentiation method** used to compute gradients through the integrator (continuous adjoint sensitivity, motivated by its constant memory cost regardless of trajectory length).

The basis choice is not a detail invented for this project to vary: the original Kolmogorov-Arnold Network paper [14] used the cubic B-spline as its default edge basis, and the base paper replaces it with a Gaussian RBF grid for the ODE setting. The two differ in ways that matter once the edge function sits inside a differentiated integrator: the Gaussian is infinitely differentiable, nonzero everywhere and a single closed-form expression, while the cubic B-spline is only twice continuously differentiable, exactly zero outside a local knot span, and built by a recursion; the B-spline run took $3.9\times$ the RBF run’s wall-clock time (Section 3). The switch is a trade-off, not an obvious improvement, so Section 6 restores the B-spline and adds five further bases (Chebyshev, Lagrange, Newton, and two further bell-shaped kernels, RSWAF and IQF) to measure it directly rather than assume it.

Each of those three fixed choices maps directly onto one of this report’s studies: Section 7 tests whether the adjoint method’s memory motivation actually holds at this project’s scale; Sections 6, 8, and 10 test the integrator choice, sweeping solver order and, in the damping study (originally intended as a stiffness probe), solver order jointly with the pendulum’s damping coefficient; and Section 9 tests the basis choice directly, by making the basis itself a learnable, trained quantity rather than a fixed decision made in advance. Section 11 extends the comparison outside the base paper’s own scope entirely, benchmarking KAN-ODE against a sparse-regression alternative built for the same symbolic-discovery goal the RBF basis was partly chosen to support, on both synthetic Lotka-Volterra data and a synthetic, non-mechanistic epidemic curve.

1.4 Why training itself needs to be treated as an object of study

This project uses two testbeds beyond Lotka-Volterra – the SIR epidemic model and a damped pendulum – chosen for simple, well-understood governing equations, so any training failure is attributable to the architecture or optimization rather than to ambiguous target dynamics. The integrator’s interaction with training is not predictable from classical numerical-analysis theory alone, because it sits inside a backpropagation graph rather than a standalone forward solve: a solver that is stable at the step size used can still train badly, train to a wrong answer, or fail to generalize. Diagnosing failures like these means treating training itself as an object of study,

not just the final trajectory error.

1.5 Structure of this report

Three method sections come first: the datasets, splits and training settings (Section 2); the numerical implementation of the KAN layer, the integrators, the training loop and the structural fixes, with their algorithms and cost (Section 3); and the verification of that implementation against known solutions and an automated test suite (Section 4). The results then follow the order the work was carried out. A baseline recipe is reproduced and used to establish the evaluation protocol (Section 5), then stress-tested with a solver and basis-function ablation that surfaces and fixes two training failures (Section 6). Five further studies build on that fixed baseline: gradient-method cost (Section 7), whether gradient noise explains a solver-order effect (Section 8), a learnable basis blend (Section 9), solver-damping interaction (planned as a stiffness test) (Section 10), and KAN-ODE against sparse regression on synthetic data and a non-mechanistic epidemic curve (Section 11). A validation stage (Section 12) retrains key configurations at up to five times the epoch budget and audits this project's recipe, and the base paper's claims, against the paper itself. Section 13 draws together cross-cutting patterns, and Section 14 closes the report.

2 Datasets and Experimental Setup

Every study in this report trains a model on a short observed window of one trajectory and then scores it on a later window it never saw. This section fixes, once, what those trajectories are, how they are generated and split, and which model and optimizer settings each system is trained with, so that later sections only need to state what they change.

2.1 The four dynamical systems

Three of the four systems are defined by a known ordinary differential equation; the fourth is deliberately *not*, so that it can test whether fixes derived from a known equation still work when no such equation exists.

Lotka-Volterra predator-prey. The base paper's benchmark [11]: prey u_1 and predators u_2 follow

$$\dot{u}_1 = \alpha u_1 - \beta u_1 u_2, \quad \dot{u}_2 = \delta u_1 u_2 - \gamma u_2, \quad (\alpha, \beta, \gamma, \delta) = (1.5, 1.0, 3.0, 1.0), \quad (3)$$

from $\mathbf{u}_0 = (1, 1)$. The system has a conserved quantity, $V(u_1, u_2) = \delta u_1 - \gamma \ln u_1 + \beta u_2 - \alpha \ln u_2$, so every trajectory is a closed orbit. From this initial state the period is 3.32 time units, so the training window $[0, 3.5]$ shows the model almost exactly one cycle. An accurate model must therefore keep returning to the same orbit, and any error in the learned field shows up as a slow drift in amplitude or phase that grows with the extrapolation horizon. (The base paper writes the same system with the names of γ and δ swapped; the dynamics are identical.)

Damped pendulum. Angle θ and angular velocity ω ,

$$\dot{\theta} = \omega, \quad \dot{\omega} = -\frac{g}{L} \sin \theta - \mu \omega, \quad g/L = 9.81, \mu = 0.5, \quad (4)$$

from $(\theta_0, \omega_0) = (2.0, 0)$ rad, a large initial swing where $\sin \theta \approx \theta$ is a poor approximation, so the model has to learn a genuinely nonlinear restoring force. Friction makes the trajectory spiral into the equilibrium $(0, 0)$: the mechanical energy $E = \frac{1}{2}mL^2\omega^2 + mgL(1 - \cos \theta)$ satisfies $dE/dt = -mL^2\mu\omega^2 \leq 0$, so it can never increase. A model can fit the trajectory closely and still break this law, which makes it an independent physical check (Section 6). The damping study (Section 10) varies $\mu \in \{0.1, 0.5, 1, 2, 5, 8\}$.

SIR epidemic. Susceptible, infected and recovered fractions of a fixed population,

$$\dot{S} = -\beta SI, \quad \dot{I} = \beta SI - \gamma I, \quad \dot{R} = \gamma I, \quad \beta = 0.35, \gamma = 0.10 \quad (R_0 = \beta/\gamma = 3.5), \quad (5)$$

from $(S(0), I(0), R(0)) = (0.99, 0.01, 0)$, with time measured in days. The right-hand sides sum to zero, so $S + I + R = 1$ holds exactly for all time. The infected curve rises to a single peak and then decays. The training window already contains the rise, the peak (day 22.5) and most of the decline, so the extrapolation window tests the *tail* of the outbreak: a low- S , low- I corner of state space that the training data visits only once.

Synthetic non-mechanistic epidemic curve. To test whether the SIR fixes transfer to data that no compartmental ODE generated (Section 11), an infection curve is built directly from a closed-form skewed bell shape rather than from any differential equation,

$$\tilde{I}(t) = \exp\left(-\frac{1}{2}\left(\frac{t-38}{14}\right)^2\right)\left(1 + \tanh\left(1.4\frac{t-38}{14}\right)\right) + \varepsilon_t, \quad \varepsilon_t \sim \mathcal{N}(0, 0.01^2), \quad (6)$$

sampled daily for $t \in \{0, \dots, 120\}$, clipped at zero and smoothed with a centred 7-day moving average, the way public case counts are usually reported. The second state is a cumulative-recovered proxy, $\tilde{R}(t) = 0.08 \sum_{s \leq t} \tilde{I}(s)$. Both columns are min-max normalized to $[0, 1]$. The tanh factor makes the rise steeper than the decay, which no symmetric SIR peak reproduces exactly. This curve is *synthetic*: it imitates the shape of a real outbreak but is not recorded case data.

2.2 Ground truth, sampling and train/test splits

The three mechanistic systems are integrated to near machine precision (SciPy’s `solve_ivp`, eighth-order Dormand-Prince, relative and absolute tolerance 10^{-12}), far tighter than any error the learned models reach, so the reference trajectory contributes no measurable error of its own. Each trajectory is sampled on a uniform grid of spacing Δt and cut at t_{train} : points up to and including t_{train} form the training set, and every later point forms the test (extrapolation) set. There is one trajectory per system, trained full-batch; there is no random shuffling and no validation split, because the test window is defined by time, not sampled from the same distribution as training.

Table 1. Dataset definitions. “Train” is $[0, t_{\text{train}}]$; “Test” is $(t_{\text{train}}, t_{\text{end}}]$. Point counts include both endpoints of the training window.

System	State dim.	Spacing Δt	t_{end}	Train window (points)	Test window (points)	Test window covers
Lotka-Volterra	2	0.1	14	$[0, 3.5]$ (36)	$(3.5, 14]$ (105)	≈ 3.2 unseen cycles; $(14, 28]$ adds ≈ 4.2 more (Table 14)
Damped pendulum	2	0.05	10	$[0, 5]$ (101) ^a	$(5, 10]$ (100)	late decay toward rest
SIR	3	0.5	80	$[0, 50]$ (101)	$(50, 80]$ (60)	end of the outbreak
Synthetic epidemic	2	1 day	120	days 0–44 (45) ^b	days 45–120 (76)	decay after the peak

^aThe original recipe used $[0, 3]$ (61 points); Section 6 explains why it was extended. ^bSplit controls in Section 11 move the cut to day 60 and day 70.

Observation noise. Where a study adds noise (the Lotka-Volterra noise sweep, Table 10, whose models are reused in Section 11), independent Gaussian noise $\mathcal{N}(0, \sigma^2)$ with $\sigma \in \{0, 0.01, 0.05, 0.1\}$ is added to the *training* observations only. The test window is always scored against the clean reference trajectory, so the reported error measures how well the model recovered the true dynamics, not how well it reproduced the noise.

2.3 Models and training settings

All KAN-ODEs use the edge parameterization of Equation 2 with a tanh input normalizer and a SiLU residual path (Section 3 gives the exact computation). A KAN layer from n_{in} to n_{out} nodes with G basis functions per edge has

$$P_{\text{layer}} = n_{\text{out}} n_{\text{in}} (G + 1) \quad (7)$$

parameters: G basis coefficients plus one residual weight on each of its $n_{\text{in}} n_{\text{out}}$ edges. The size of the network differs by system: SIR, with its three-dimensional state, and the pendulum, with its sharper nonlinearity, were given more capacity than Lotka-Volterra.

MLP baselines. Two multilayer-perceptron Neural ODEs are trained on Lotka-Volterra under exactly the same protocol for comparison: the base paper’s literal architecture, $[2, 50, 2]$ with tanh (252 parameters), and a parameter-matched deeper network, $[2, 14, 8, 8, 2]$ with SiLU (252 parameters), introduced because the literal one fails to train under this protocol (Table 9).

Table 2. Model and training settings for each system’s main runs. Studies that deviate (solver or basis ablations, the damping sweep’s 2,000-epoch budget and its range of damping coefficients) say so where they are reported.

Setting	Lotka-Volterra	Pendulum	SIR	Synthetic epidemic
KAN widths	[2, 10, 2]	[2, 10, 2]	[3, 16, 3]	[2, 10, 2]
Basis, grid size G	RBF, 5	RBF, 8	RBF, 8	RBF, 5
Parameters (Eq. 7)	240	360	864	240
Solver, substeps m	Tsit5, 2	Tsit5, 2	Tsit5, 2	Tsit5, 2
Learning rate (Adam)	2×10^{-3}	3×10^{-3}	3×10^{-3}	2×10^{-3}
Gradient-norm clipping	none (1.0 in the extended-budget reruns, Section 12)	1.0	1.0	1.0
Epochs	10,000	10,000	10,000	2,000 (probes), 5,000 (full)
Structural fixes	none	none	time scale $s = 10$, projection, gate on I	time scale $s = 24$; projection and gate tested as arms
Seed	42	42	42	42

Why 10,000 epochs. The base paper trains for 10^5 epochs. This project fixed its budget at 10^4 for three reasons. First, cost: the full ablation is 24 Lotka-Volterra configurations plus runs on the two other mechanistic systems, and at the measured 0.03–0.68 s per epoch on a CPU a 10^5 -epoch sweep of the Lotka-Volterra ablation alone would take on the order of a week of serial CPU time, before any other system or follow-up study. Second, the paper itself reports that its KAN-ODE reaches 2.6×10^{-5} training loss within 10^4 epochs, already below its MLP’s 10^5 -epoch result, so 10^4 is where the paper’s own speed claim is made. Third, the ablations compare configurations against each other at a common budget, so what matters is whether the *ranking* is stable, not whether each run has fully converged. That last assumption was tested directly by retraining five configurations to 25,000–50,000 epochs; Section 12 reports the result, which largely confirms the ranking among the KAN variants (Euler stays last; RBF and B-spline stay within a factor of two but swap places) but reverses the KAN-versus-MLP comparison.

Training objective and model selection. Each epoch integrates the learned field once over the whole training window from the known initial state and minimizes the mean squared error against the observed points,

$$\mathcal{L}(\theta) = \frac{1}{N_{\text{train}} d} \sum_{n=0}^{N_{\text{train}}-1} \|\hat{\mathbf{u}}_{\theta}(t_n) - \mathbf{u}(t_n)\|_2^2, \quad (8)$$

where d is the state dimension and $\hat{\mathbf{u}}_{\theta}(t_n)$ is the integrator’s prediction. No sparsity or smoothness regularization is added in any run. Adam uses a constant learning rate with no schedule, matching the base paper. The checkpoint that is reported is the one with the lowest *training* loss seen during the run; the test window is monitored but never used to select a model, so no reported number is tuned on the data it is scored on. Algorithm 3 in Section 3 gives the complete loop.

3 Numerical Implementation

A KAN-ODE is two numerical objects stacked on top of each other: a function approximator that evaluates the vector field $\mathbf{f}_{\theta}$, and an explicit integrator that turns that field into a trajectory. Training differentiates through both. This section gives the exact computation for each, the cost of one training epoch in terms of them, and the three structural modifications to the field that later sections use as fixes.

3.1 Where the implementation comes from

The paper’s own code [12] is written in Julia and builds its network from the KDense layer of Kolmogorov-Arnold.jl [15], a KAN library that evaluates every edge’s basis functions as one dense array operation instead of one spline object per edge. This project ports that layer to PyTorch rather than using pykan [14], the reference Python KAN implementation. pykan stores a separate B-spline object on each edge and periodically refits its grid during training, which is flexible but slow and makes the basis hard to swap. The KDense formulation

reduces a whole layer to one basis tensor and one matrix product (Algorithm 1), so replacing the basis is a change to a single function, and every basis in this report is evaluated by exactly the same code path. The integrators, the training loop and every diagnostic were written for this project and not taken from an ODE library, so that each solver could be instrumented identically. Two library components are used only as independent baselines: the continuous-adjoint gradient in Section 7 comes from `torchdiffeq` [2], and the sparse regression of Section 11 from `pysindy` [6, 10].

The port matches `KDense` in every structural detail: a fixed grid of G evenly spaced centres on $[-1, 1]$ with spacing $\delta = 2/(G - 1)$, a tanh normalizer on the basis path, a SiLU residual path applied to the *unnormalized* input, and Glorot-uniform initialization. It differs in two numerical details, both recorded in the implementation audit (Table 30): the Gaussian is $\exp(-\frac{1}{2}((u - z_g)/\delta)^2)$, as written in the paper’s Equation 5, whereas the paper’s code uses $\exp(-((u - z_g)/\delta)^2)$, a kernel narrower by a factor of $\sqrt{2}$; and the initial weights are not divided by 10^5 as the paper’s code does.

3.2 One KAN layer

Figure 2 expands a single edge of the network in Figure 1. For an input vector $\mathbf{x} \in \mathbb{R}^{n_{\text{in}}}$, the layer computes

$$y_j = \sum_{i=1}^{n_{\text{in}}} \phi_{j,i}(x_i) = \sum_{i=1}^{n_{\text{in}}} \left(\underbrace{\sum_{g=1}^G C_{j,i,g} B_g(\tanh x_i)}_{\text{learned basis expansion}} + \underbrace{W_{j,i} \text{silu}(x_i)}_{\text{residual path}} \right), \quad j = 1, \dots, n_{\text{out}}, \quad (9)$$

with trainable coefficient tensor $C \in \mathbb{R}^{n_{\text{out}} \times n_{\text{in}} \times G}$ and residual weights $W \in \mathbb{R}^{n_{\text{out}} \times n_{\text{in}}}$. Every edge ($i \rightarrow j$) therefore owns its own univariate function $\phi_{j,i}$, shaped by its own G coefficients. The tanh is needed because the grid covers only $[-1, 1]$; it maps any real input into that range, where the basis functions are placed. The residual path keeps a direct, unsquashed route from input to output, which stops the layer from saturating when an input is large.

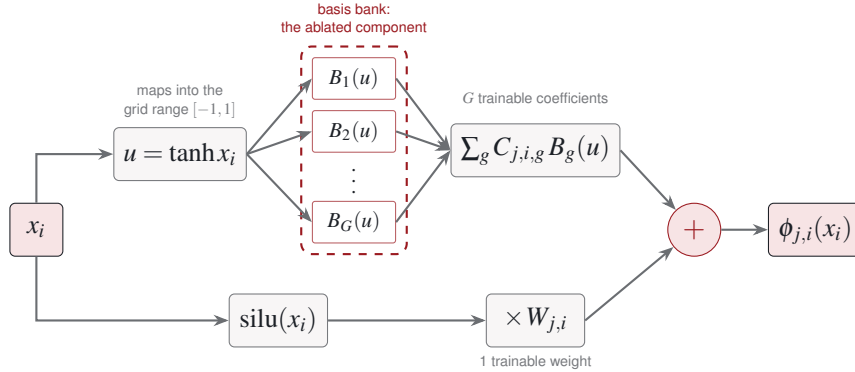


Figure 2. One edge $\phi_{j,i}$ of a KAN layer (Equation 9). The input is squashed into $[-1, 1]$, expanded in G fixed basis functions, and recombined with G learned coefficients; a separate residual path adds a learned multiple of $\text{silu}(x_i)$. The basis ablation of Section 6 replaces only the dashed box. In the original KAN paper that box holds cubic B-splines; the base paper replaces them with Gaussian RBFs.

Summing Equation 9 over edges collapses the whole layer into two matrix products. Stacking the basis values of all inputs into one vector $\beta(\mathbf{x}) \in \mathbb{R}^{n_{\text{in}}G}$ and reshaping C into an $n_{\text{out}} \times n_{\text{in}}G$ matrix gives

$$\mathbf{y} = C\beta(\mathbf{x}) + W \text{silu}(\mathbf{x}), \quad (10)$$

which is what Algorithm 1 evaluates, for a whole batch of states at once.

The full vector field is two such layers, $\mathbf{f}_\theta = \text{KDense}_2 \circ \text{KDense}_1$. Because line 2 is the only place the basis appears, all seven bases compared in this report (Figure 3) share lines 1 and 3–6 exactly.

Cost of one field evaluation. Lines 4–5 of Algorithm 1 perform $n_{\text{out}}n_{\text{in}}(G + 1)$ multiply-adds per state, which is exactly the layer’s parameter count (Equation 7), so the $[2, 10, 2]$, $G = 5$ network costs 240 multiply-adds per evaluation of $\mathbf{f}_\theta$. Line 2 costs $O(n_{\text{in}}G)$ elementwise operations for the RBF, RSWAF and IQF kernels, which are

Algorithm 1 Forward pass of one KAN layer (tensor shapes in brackets; b is the batch size).

Input: states $X [b \times n_{\text{in}}]$; grid $\mathbf{z} [G]$, spacing δ ; coefficients $C [n_{\text{out}} \times n_{\text{in}} G]$; residual weights $W [n_{\text{out}} \times n_{\text{in}}]$

Output: layer output $Y [b \times n_{\text{out}}]$

- 1: $U \leftarrow \tanh(X)$ $\triangleright [b \times n_{\text{in}}]$, now inside $[-1, 1]$
 - 2: $\mathcal{B}[\cdot, i, g] \leftarrow B_g(U[\cdot, i]; \mathbf{z}, \delta)$ for all i, g $\triangleright$ one broadcast call, $[b \times n_{\text{in}} \times G]$
 - 3: $\mathcal{B} \leftarrow \text{reshape}(\mathcal{B}, b \times n_{\text{in}} G)$
 - 4: $Y \leftarrow \mathcal{B} C^\top$ $\triangleright$ one matrix product for every edge's expansion
 - 5: $Y \leftarrow Y + \text{silu}(X) W^\top$ $\triangleright$ residual path on the unnormalized input
 - 6: **return** Y
-

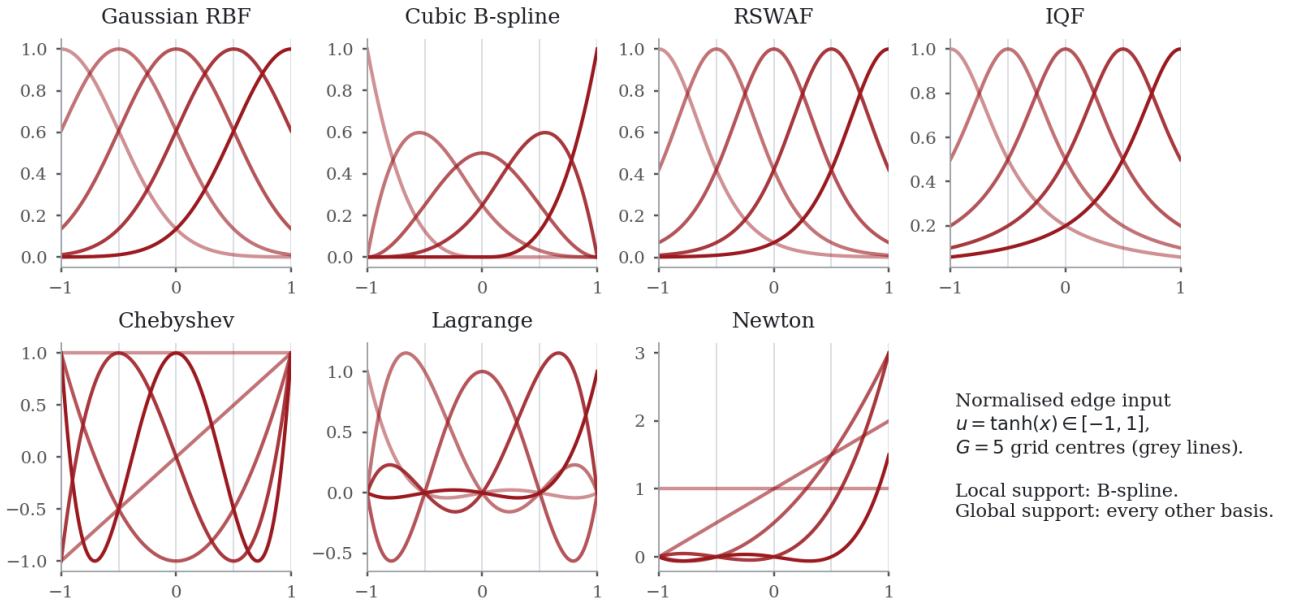


Figure 3. The seven basis families $B_g(u)$ available to line 2 of Algorithm 1, evaluated by this project's own implementation at $G = 5$. Only the cubic B-spline has compact support, although at $G = 5$ the clamped knot vector has a single interior knot, so only B_1 and B_5 vanish on half of $[-1, 1]$ while B_2 – B_4 span all of it. Every other basis is nonzero almost everywhere, so each coefficient affects the edge function over the whole input range. Equations 26–28 give their formulas.

single closed-form expressions. The cubic B-spline uses a clamped (open-uniform) knot vector: each boundary knot is repeated $k + 1 = 4$ times, with $G - k - 1$ evenly spaced interior knots, so its G basis functions sum to exactly one everywhere on $[-1, 1]$, including at the edges. It then runs the Cox–de Boor recursion, which builds the basis from degree-0 indicator functions through three recursion levels, with two divisions per knot interval at each level. Since the matrix product is identical for every basis, this extra basis work is the most likely reason the B-spline run takes $3.9\times$ the RBF run’s wall-clock time at the same parameter count (Table 7). The time was measured; the breakdown of where it goes was not profiled.

3.3 The integrator

Every solver in this report is an explicit Runge-Kutta method. An s -stage method advances the state by a step h as

$$\mathbf{k}_i = \mathbf{f}_\theta \left(\mathbf{u}_n + h \sum_{j=1}^{i-1} a_{ij} \mathbf{k}_j \right), \quad i = 1, \dots, s, \quad \mathbf{u}_{n+1} = \mathbf{u}_n + h \sum_{i=1}^s b_i \mathbf{k}_i, \quad (11)$$

where the coefficients (a_{ij}, b_i, c_i) form the method’s Butcher tableau. Euler, Midpoint, Heun and RK4 (Equations 22–25) are special cases with $s = 1, 2, 2, 4$; DOPRI5 [7] and Tsit5 [16] use $s = 7$ tableaus, listed in full in Appendix A. Both are “first same as last” methods whose seventh stage has zero solution weight ($b_7 = 0$). It exists only to feed the embedded error estimate used for adaptive step control. Since this project integrates with a fixed step, that stage is skipped, and both methods cost six field evaluations per step.

Fixed steps and substeps. The base paper integrates with *adaptive* Tsit5, which chooses its own step from an error estimate. This project uses a *fixed* step instead, so that the number of function evaluations, and hence the backpropagation graph, is identical every epoch and can be compared across solvers. Between two consecutive observation times the solver takes m equal substeps (Algorithm 2); every run in this report uses $m = 2$, so the integration step is $h = \Delta t / 2$ (0.05 for Lotka-Volterra, not the data spacing of 0.1).

Algorithm 2 Fixed-step trajectory integration with substeps.

Input: field $\mathbf{f}_\theta$; initial state $\mathbf{u}_0$; observation times $t_0 < \dots < t_N$; one-step map Φ_h (Eq. 11); substeps m

Output: predicted states $\hat{\mathbf{u}}(t_0), \dots, \hat{\mathbf{u}}(t_N)$

```

1:  $\hat{\mathbf{u}}(t_0) \leftarrow \mathbf{u}_0$ ;  $\mathbf{u} \leftarrow \mathbf{u}_0$ 
2: for  $n = 0, \dots, N - 1$  do
3:    $h \leftarrow (t_{n+1} - t_n) / m$ 
4:   for  $r = 0, \dots, m - 1$  do
5:      $\mathbf{u} \leftarrow \Phi_h(\mathbf{f}_\theta, t_n + rh, \mathbf{u})$  ▷  $s$  evaluations of  $\mathbf{f}_\theta$ 
6:    $\hat{\mathbf{u}}(t_{n+1}) \leftarrow \mathbf{u}$ 
7: return  $\hat{\mathbf{u}}(t_0), \dots, \hat{\mathbf{u}}(t_N)$ 
```

A training epoch therefore evaluates the field

$$\text{NFE}_{\text{epoch}} = (N_{\text{train}} - 1) m s \quad (12)$$

times. For Lotka-Volterra, $N_{\text{train}} = 36$ and $m = 2$, giving 70 for Euler, 140 for the second-order methods, 280 for RK4 and 420 for DOPRI5 and Tsit5, which are the values in Table 5. Because backpropagation keeps every intermediate stage (Section 7), $\text{NFE}_{\text{epoch}}$ also sets the memory and time of the backward pass.

Accuracy. A method of order p commits a local truncation error of $O(h^{p+1})$ per step, which accumulates to a global error of $O(h^p)$ over a fixed horizon [9]:

$$\|\mathbf{u}_n - \mathbf{u}(t_n)\| \leq K h^p \quad \text{for } t_n \leq T, \quad (13)$$

with K depending on T and on the smoothness of the field. Halving h should therefore reduce the error by 2^p : by 2 for Euler, 4 for Midpoint and Heun, 16 for RK4, and 32 for the fifth-order methods. The solver test suite (Section 4) checks exactly this on ODEs with known solutions.

Stability. Applied to the linear test equation $\dot{u} = \lambda u$, an explicit method reduces to $u_{n+1} = R(z) u_n$ with $z = h\lambda$ and a polynomial stability function

$$R_{\text{Euler}}(z) = 1 + z, \quad R_{\text{RK2}}(z) = 1 + z + \frac{z^2}{2}, \quad R_{\text{RK4}}(z) = \sum_{k=0}^4 \frac{z^k}{k!}, \quad (14)$$

and the numerical solution decays, as the true solution does for $\text{Re } \lambda < 0$, only while $|R(z)| \leq 1$ [8]. On the negative real axis this holds for $h|\lambda| \leq 2$ (Euler and both second-order methods) and $h|\lambda| \lesssim 2.79$ (RK4). A polynomial R always grows without bound for large $|z|$, so every explicit method has a bounded stability region; this is the classical reason explicit solvers fail on stiff problems, and it is the prediction the damping study (Section 10) tests against the trained models.

3.4 Differentiating through the solver

The loss (Equation 8) depends on θ only through the integrated trajectory. This project computes its gradient by *direct automatic differentiation*: every operation of Algorithms 1–2 is recorded, and the reverse pass applies the chain rule back through all $\text{NFE}_{\text{epoch}}$ field evaluations. The result is the exact gradient of the discrete computation, at a memory cost linear in the number of steps. The base paper instead uses the continuous adjoint method, which solves a second ODE backward in time; Section 7 states it and measures which of the two is cheaper at this project’s scale.

3.5 Structural modifications of the field

Three optional wrappers change *what* is integrated without changing the network itself. Each is used as a fix in Section 6 and tested for transfer in Section 11.

Time rescaling. With a scale factor s , the model is integrated in rescaled time $\tau = t/s$, so the network learns $\mathbf{g}_\theta = s\mathbf{f}$ instead of $\mathbf{f}$:

$$\frac{d\mathbf{u}}{d\tau} = \mathbf{g}_\theta(\mathbf{u}), \quad \hat{\mathbf{u}}(t) = \hat{\mathbf{u}}(\tau = t/s). \quad (15)$$

This is a change of units only. It lets a system whose true rates are small (SIR’s rates are $\sim 10^{-2}$ per day) be learned by a network whose initial output is $\sim 10^{-1}$, without changing the dynamics being fit.

Zero-sum projection. For a system whose state components must sum to a constant, the field is projected onto the zero-sum subspace,

$$\mathbf{f}_{\text{proj}}(\mathbf{u}) = \mathbf{f}_\theta(\mathbf{u}) - \frac{1}{d} \sum_{k=1}^d f_{\theta,k}(\mathbf{u}) \mathbf{1}, \quad \text{so} \quad \frac{d}{dt} \sum_k u_k = \sum_k f_{\text{proj},k} = 0. \quad (16)$$

The invariant then holds for every state and every horizon, up to rounding error, rather than only where a penalty term was fitted. Every Runge-Kutta step is a linear combination of field values, so a zero-sum field gives a zero-sum update at any order and any step size.

Vanishing-field gate. For compartmental epidemic models every transition is proportional to the infected fraction, so $I = 0$ is a line of equilibria: a population with no infection stays that way. The gate builds this in by multiplying the field by one state component u_d ,

$$\mathbf{f}_{\text{gate}}(\mathbf{u}) = u_d \mathbf{f}_\theta(\mathbf{u}), \quad (17)$$

so that $\mathbf{f}_{\text{gate}} = \mathbf{0}$ whenever $u_d = 0$. For SIR ($d = I$) the true field is I times a cofactor that is affine in S , which leaves the network a much simpler function to learn. When the gate and the projection are used together (SIR), the gate is applied to the projected field, $\mathbf{f} = u_d \mathbf{f}_{\text{proj}}$, which is still zero-sum. Unlike the other two wrappers, this one makes a physical assumption about the system.

The training loop. Algorithm 3 assembles the pieces. Its two safety mechanisms were added one after the other, in response to the cross-domain failures of Section 6. Gradient clipping, which bounds the size of one optimizer step, was added after the first, unclipped pendulum and SIR runs showed gradient-norm explosions. The non-finite guard, which skips any step whose gradient has overflowed, was added after the clipped SIR run of Section 6 still ended in NaN. Clipping alone cannot handle an overflow: rescaling an infinite gradient produces $\infty \times 0 = \text{NaN}$, which corrupts every parameter. The 10,000-epoch Lotka-Volterra benchmarks of Section 6 predate both mechanisms and ran unclipped.

Algorithm 3 Training loop (`train.py`) used for the benchmark runs; the study-specific loops follow the same checkpoint-selection rule.

Input: training data $\{(t_n, \mathbf{u}_n)\}_{n < N_{\text{train}}}$; model $\mathbf{f}_\theta$ (optionally wrapped by Eqs. 15–17); learning rate η ; clip threshold c ; epochs E

Output: checkpoint θ^* with the lowest training loss

- 1: $\mathcal{L}^* \leftarrow \infty$; initialize θ (Glorot uniform), Adam state
- 2: **for** $e = 1, \dots, E$ **do**
- 3: $\hat{\mathbf{u}} \leftarrow \text{Algorithm 2}(\mathbf{f}_\theta, \mathbf{u}_0, t_{0:N_{\text{train}}-1})$
- 4: $\mathcal{L} \leftarrow \text{Equation 8}$; $\mathbf{g} \leftarrow \nabla_\theta \mathcal{L}$ ▷ direct autograd through the solver
- 5: **if** $\mathcal{L} < \mathcal{L}^*$ **then**
- 6: $\mathcal{L}^* \leftarrow \mathcal{L}$; $\theta^* \leftarrow \text{copy}(\theta)$ ▷ weights that produced $\mathcal{L}$
- 7: **if** $\|\mathbf{g}\|$ and $\mathcal{L}$ are finite **then**
- 8: **if** clipping is enabled **then** $\mathbf{g} \leftarrow \mathbf{g} \cdot \min(1, c/\|\mathbf{g}\|)$
- 9: $\theta \leftarrow \text{Adam}(\theta, \mathbf{g}, \eta)$
- 10: **else**
- 11: skip the update; abort if 100 consecutive steps are non-finite
- 12: every 10 epochs: record loss on the full horizon ▷ monitoring only, never used for selection
- 13: **return** θ^*

4 Verification and Testing Framework

Every result in this report depends on code written for this project: seven basis functions, six integrators, the training loop and every diagnostic. Before any of that code is trusted to produce a benchmark number, it is checked against cases whose correct answer is known independently. This section describes those checks and the pipeline that turns one training run into the numbers in the tables.

4.1 Solver verification against a known solution

The integrators are checked on $\dot{y} = -y$, $y(0) = 1$, whose exact solution $y(t) = e^{-t}$ is known. Integrating to $t = 1$ with n equal steps gives an error $e(h) = |y_n - e^{-1}|$ at step $h = 1/n$, and by Equation 13 the observed order between two step sizes is

$$\hat{p} = \frac{\log e(h_1) - \log e(h_2)}{\log h_1 - \log h_2}. \quad (18)$$

Table 3 shows that every solver reaches its theoretical order. Halving the step divides the error by 2^p , as predicted. It also shows a difference that order alone does not capture: at the same fifth order, Tsit5’s error is about $5\times$ smaller than DOPRI5’s, because Tsitouras chose its coefficients to minimize the leading error constant K in Equation 13 [16].

Midpoint and Heun agree to every digit here because, for a linear field, both second-order methods reduce to the same update $y_{n+1} = (1 - h + h^2/2)y_n$. They differ only on nonlinear fields such as the learned ones. Further solver tests check that the Tsit5 and DOPRI5 tableaus are internally consistent (for DOPRI5, row sums equal to the nodes and the quadrature order conditions up to fifth order; for Tsit5, weights summing to one and a consistent embedded fourth-order pair), that energy on a harmonic oscillator drifts orders of magnitude less under RK4 and Tsit5 than under Euler, that time-dependent and backward-in-time integration are correct, and that gradients flow through every solver.

Table 3. Observed convergence order of this project’s integrators on $\dot{y} = -y$ over $[0, 1]$ (double precision, one substep). The automated test requires $|\hat{p} - p|$ below 0.20 (Euler) to 0.45 (fifth order).

Solver	Order p	$e(h=0.1)$	$e(h=0.05)$	$e(h=0.025)$	$\hat{p}$ (0.1→0.05)	$\hat{p}$ (0.05→0.025)
Forward Euler	1	1.92×10^{-2}	9.39×10^{-3}	4.65×10^{-3}	1.03	1.02
Midpoint	2	6.62×10^{-4}	1.59×10^{-4}	3.91×10^{-5}	2.06	2.03
Heun	2	6.62×10^{-4}	1.59×10^{-4}	3.91×10^{-5}	2.06	2.03
Classical RK4	4	3.33×10^{-7}	2.00×10^{-8}	1.22×10^{-9}	4.06	4.03
DOPRI5	5	1.21×10^{-9}	3.48×10^{-11}	1.04×10^{-12}	5.12	5.06
Tsit5	5	2.56×10^{-10}	6.43×10^{-12}	1.77×10^{-13}	5.31	5.18

4.2 The automated test suite

The checks run as an automated `pytest` suite of 250 tests. On the final code, 249 pass and 1 is skipped: that test measures peak GPU memory and runs only when a CUDA device is present. Table 4 groups the tests by what they protect.

Table 4. Automated tests, grouped by the component they verify.

Component	Tests	What is checked
Integrators	32	observed order (Table 3); tableau consistency; energy drift; substep monotonicity; non-autonomous and backward integration; batching; gradient flow
Basis functions and KAN layer	57	shape and validity of all seven bases; exact known values (Chebyshev values and recurrence, Newton values, Lagrange cardinal property); partition of unity (B-spline, Lagrange); gradient flow through every basis; layer forward and backward passes
MLP baseline	22	parameter count and KAN/MLP parameter matching; supported activations; initialization statistics; gradient flow through the ODE; seed reproducibility
Datasets	10	shapes, splits and reproducibility; pendulum energy conserved at $\mu = 0$ and dissipated at $\mu > 0$; the SIR invariant $S + I + R = 1$; epidemic-curve normalization
Metrics	11	MSE, MAE, R^2 , relative L_2 , gradient norm and function-evaluation counts on inputs with known answers; Lipschitz estimate on a linear map and on $\sin$
Training pipeline	23	end-to-end training; the selected checkpoint is the best one, not the last; a reloaded checkpoint rebuilds the same field, including every fix; rescaling time equals rescaling the field, and $s = 1$ changes nothing bit for bit; the projected field sums to zero far outside the training window; the gated field vanishes on $u_d = 0$ (Eqs. 15–17)
Plotting	7	every figure function runs on real outputs
Study-specific checks	88	adjoint vs. direct gradient agreement (8); gradient-noise statistic (18); hybrid-basis gate (10); sparse-regression and pruning procedures (33); damping-study verdict classifier and non-finite guard (19)
Total	250	249 passed, 1 skipped (requires CUDA)

4.3 From a configuration to a table entry

Figure 4 shows the path every benchmark number takes. One function trains any configuration (system, model, basis, solver, epochs, seed and fixes) and writes a self-describing results record: the full configuration, the software version and code revision, and every metric at both the selected and the final checkpoint. A benchmark driver runs lists of configurations through that function, and the values in this report’s tables are taken from the saved records (collated to CSV by a script) rather than typed in from memory, so any number can be traced back to the run that produced it.

Post-hoc analyses. A few numbers were computed afterwards from saved checkpoints and training records, without training anything: the pendulum models rescored on a common test window, the hybrid basis’s measured contributions, the RBF/B-spline comparison at a matched budget, the far-horizon scores of the 50,000-epoch models, the Newton root-finding and Jacobian spectra of the damping study, the damping study’s trajectories redrawn for its figures, and the exact-derivative sparse regression. Each is produced by a short script in the project repository (`implementation/experiments/posthoc_checks`), and every redrawn report figure

by a script in `report/figure_scripts`. These scripts were added with this revision of the report, after the results commit named on the title page, and are not covered by the automated tests of Table 4. Their outputs were checked against the saved records they build on instead: each rebuilt checkpoint reproduces its own recorded metrics (for example, the 10,000-epoch far-horizon scores to three significant figures, the pendulum and SIR R^2 exactly, and every damping-study trajectory its recorded best training MSE), and the finite-difference sparse regression reproduces the saved noise-free result.

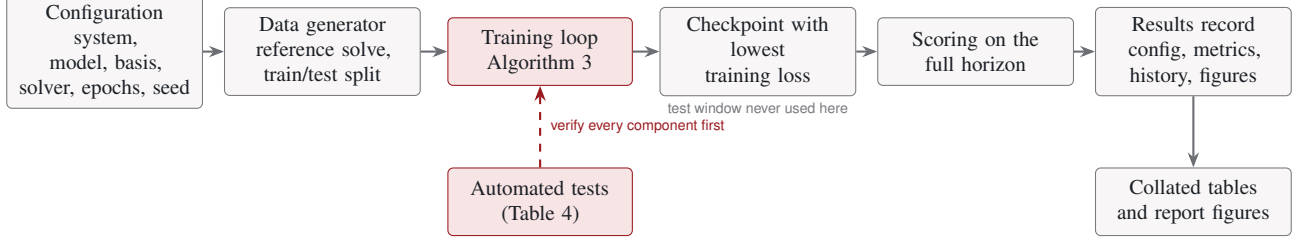


Figure 4. The benchmark pipeline. Model selection uses only the training window; the test window is used once, when the selected checkpoint is scored. The same training and scoring code produces the benchmark tables; the post-hoc analyses are separate scripts.

4.4 Metrics

A model’s predicted trajectory $\hat{\mathbf{u}}$ is compared with the reference $\mathbf{u}$ on a set $\mathcal{T}$ of time points (the test window unless stated otherwise), with d state components:

$$\text{MSE} = \frac{1}{|\mathcal{T}|d} \sum_{t \in \mathcal{T}} \|\hat{\mathbf{u}}(t) - \mathbf{u}(t)\|_2^2, \quad \text{RMSE} = \sqrt{\text{MSE}}, \quad (19)$$

$$R^2 = 1 - \frac{\sum_{t \in \mathcal{T}} \|\hat{\mathbf{u}}(t) - \mathbf{u}(t)\|_2^2}{\sum_{t \in \mathcal{T}} \|\mathbf{u}(t) - \bar{\mathbf{u}}\|_2^2}, \quad \text{rel. } L_2 = \frac{\|\hat{\mathbf{u}} - \mathbf{u}\|_2}{\|\mathbf{u}\|_2}, \quad (20)$$

where $\bar{\mathbf{u}}$ is the per-component mean over $\mathcal{T}$. $R^2 = 1$ is a perfect match, $R^2 = 0$ is no better than predicting that mean, and a negative R^2 is worse than it. The smoothness of a learned field is summarized by an empirical Lipschitz estimate,

$$\hat{L} = \max_k \frac{\|\mathbf{f}_\theta(\mathbf{u}_k + \boldsymbol{\varepsilon}_k) - \mathbf{f}_\theta(\mathbf{u}_k)\|_2}{\|\boldsymbol{\varepsilon}_k\|_2}, \quad \|\boldsymbol{\varepsilon}_k\|_2 = 10^{-3}, \quad (21)$$

taken over the training states $\mathbf{u}_k$, each with a random perturbation direction. This is the largest local slope of the field observed along the training trajectory. It is a *lower* bound on the field’s true Lipschitz constant, which would require a maximum over all pairs of points, and it is used only to compare how sharp different trained fields are. Training cost is reported as function evaluations per epoch (Equation 12) and as wall-clock time. All timings are CPU timings (a local development machine, or cloud CPUs for the extended-budget runs of Section 12), except the adjoint-versus-autograd comparison of Section 7, which measures GPU memory and runs on an RTX 3060. Timings are compared only within one machine, except in the cost note of Section 12, where the difference between machines is itself the point.

5 Baseline Recipe and Evaluation Protocol

The baseline stage adapts the base paper’s KAN-ODE recipe [11] (differences are listed in Table 30) on the Lotka-Volterra predator-prey system (Equation 3), using a two-layer Kolmogorov-Arnold Network, $[2, 10, 2]$, with $G = 5$ Gaussian radial-basis-function grid points per edge (240 parameters), trained on $t \in [0, 3.5]$ (36 points, almost exactly one predator-prey cycle) and scored strictly on $t \in (3.5, 14.0]$ (105 unseen points, about 3.2 further cycles). Extrapolation is scored this way, on points the training loss never touched, because it is the only meaningful test of whether the network learned the underlying dynamics rather than memorized the shape of the training window. A model that only reproduces the 36 training points could still be badly wrong about the system it is meant to describe.

Two additional systems were carried forward alongside Lotka-Volterra: the SIR epidemic model (Equation 5) and the damped pendulum (Equation 4), with the splits of Table 1. Both were chosen because their governing

equations are textbook-simple and well understood, so any training failure can be attributed to the architecture or the optimization process rather than to ambiguous target dynamics. Each also carries a physical law that is separate from trajectory error. SIR conserves $S + I + R = 1$ exactly, so any model that violates it has learned a vector field whose structure is wrong, not merely inaccurate. The pendulum’s energy must decrease monotonically, which a model can violate while still fitting the trajectory closely.

5.1 Evaluation protocol

This stage established the evaluation protocol used throughout the rest of this report. Model selection uses the minimum *training*-window loss only (Algorithm 3). The headline error is the mean squared error (MSE, Equation 19) between the model’s predicted trajectory and the true trajectory, computed strictly outside the training horizon, so that a number cannot improve simply by widening what counts as fit. The coefficient of determination, R^2 (Equation 20), reports the same comparison on a scale where 1 is a perfect match and 0 is no better than predicting the mean of the true trajectory; a negative value means the model performs worse than that trivial baseline. Every later study keeps this selection rule, and says so where it reports a different quantity (for example the full-horizon MSE on the pendulum).

6 Benchmark Ablation and Cross-Domain Stability Analysis

Before extending KAN-ODE to new systems, two questions needed settling: which solver and basis choices this architecture actually prefers, and whether the baseline recipe (Section 5) transfers unchanged to systems beyond Lotka-Volterra. Both were answered empirically, and the second uncovered two genuine training failures that required a mechanistic diagnosis before they could be fixed.

6.1 Solver, basis and architecture ablation

Every integrator in Table 5 advances the state by one step h using only evaluations of the learned field $\mathbf{f}_\theta$, and all were implemented directly for this project rather than called from an external ODE library. Writing $\mathbf{u}_n$ for the state at step n :

$$\text{Forward Euler (order 1): } \mathbf{u}_{n+1} = \mathbf{u}_n + h \mathbf{f}_\theta(\mathbf{u}_n) \quad (22)$$

$$\text{Midpoint (order 2): } \mathbf{k}_1 = \mathbf{f}_\theta(\mathbf{u}_n), \quad \mathbf{u}_{n+1} = \mathbf{u}_n + h \mathbf{f}_\theta\left(\mathbf{u}_n + \frac{h}{2} \mathbf{k}_1\right) \quad (23)$$

$$\text{Heun’s method (order 2): } \mathbf{k}_1 = \mathbf{f}_\theta(\mathbf{u}_n), \quad \mathbf{k}_2 = \mathbf{f}_\theta(\mathbf{u}_n + h \mathbf{k}_1), \quad \mathbf{u}_{n+1} = \mathbf{u}_n + \frac{h}{2}(\mathbf{k}_1 + \mathbf{k}_2) \quad (24)$$

$$\text{Classical RK4 (order 4): } \mathbf{u}_{n+1} = \mathbf{u}_n + \frac{h}{6}(\mathbf{k}_1 + 2\mathbf{k}_2 + 2\mathbf{k}_3 + \mathbf{k}_4) \quad (25)$$

with the classical RK4 stages $\mathbf{k}_1 = \mathbf{f}_\theta(\mathbf{u}_n)$, $\mathbf{k}_2 = \mathbf{f}_\theta(\mathbf{u}_n + \frac{h}{2}\mathbf{k}_1)$, $\mathbf{k}_3 = \mathbf{f}_\theta(\mathbf{u}_n + \frac{h}{2}\mathbf{k}_2)$, $\mathbf{k}_4 = \mathbf{f}_\theta(\mathbf{u}_n + h\mathbf{k}_3)$. Here h is the integration step, which in every run is half the data spacing Δt (two substeps, Algorithm 2). DOPRI5 and Tsit5 follow the same stage-and-combine pattern at order 5 (Equation 11), each with a seven-stage Butcher tableau (Appendix A) whose last stage has zero weight, so both cost six field evaluations per step at a fixed step size. Their coefficients are chosen so the error terms of orders one through four cancel exactly; the two differ in which remaining error constant they minimize, and Tsit5’s is about $5\times$ smaller on the test problem of Table 3. Every extra stage in Equations 23–25 costs one additional evaluation of $\mathbf{f}_\theta$, which is why function evaluations per epoch, not solver order alone, is the operative cost measure in Table 5.

Table 5 sweeps these six explicit Runge-Kutta integrators at fixed radial-basis-function (RBF) basis; Table 7 sweeps seven basis representations at fixed Tsit5 solver. Both use the protocol of Section 5. The number of function evaluations per training epoch (Equation 12) is reported because it is the dominant cost driver for training time. The Lipschitz estimate L of the trained field (Equation 21) – the largest rate at which the output of $\mathbf{f}_\theta$ changes per unit change of its input, measured along the training trajectory – is reported alongside accuracy because it is a direct measure of how smooth or jagged the basis expansion in Equation 2 ended up, independent of the trajectory error.

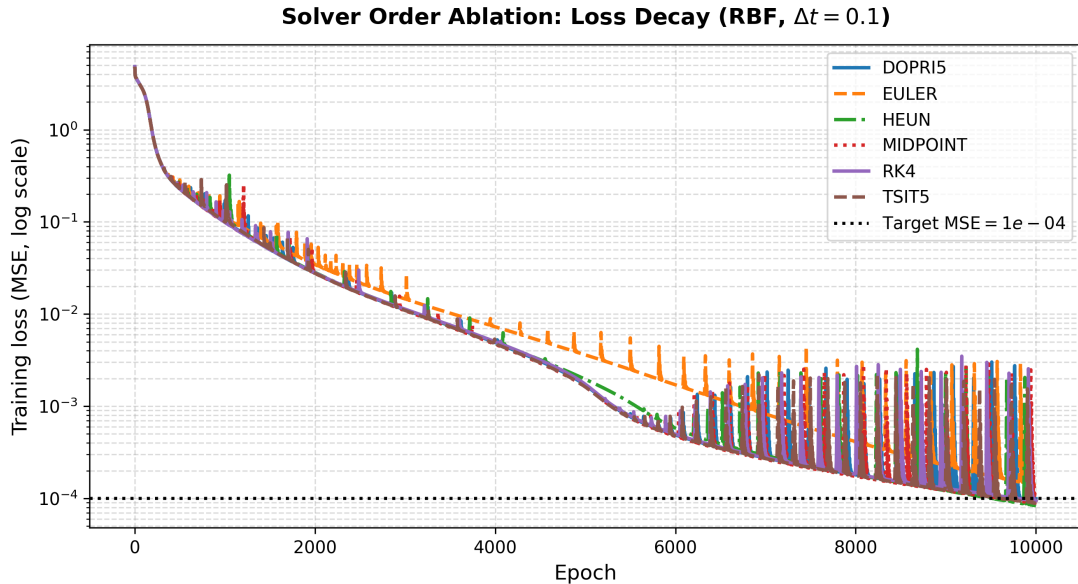
Truncation error dominates only at order $p = 1$: Euler’s extrapolation MSE is $28\times$ Tsit5’s, and it is the only solver that fails to reach 10^{-4} training loss within budget. Its trained field is also the least smooth of the six ($L = 8.55$, the highest in the table), consistent with a systematic bias forcing the network to fit a sharper correction than the smoother integrators require. Midpoint, RK4, DOPRI5 and Tsit5 then collapse into a 4.1% spread in extrapolation error with no monotonic order dependence, which suggests that at $h = 0.05$ integrator

Table 5. Solver ablation (fixed RBF basis, data spacing $\Delta t = 0.1$ with two substeps, so integration step $h = 0.05$; 240 parameters; 10,000 epochs).

Solver	Order	Function evaluations / epoch	Training MSE	Extrapolation MSE	Extrapolation R^2	Lipschitz L	Wall-clock
Forward Euler	1	70	1.38×10^{-4}	2.51×10^{-3}	0.99914	8.55	254 s
Heun RK2	2	140	8.32×10^{-5}	3.10×10^{-4}	0.99989	7.07	499 s
Midpoint	2	140	8.86×10^{-5}	8.95×10^{-5}	0.99997	6.96	498 s
Classical RK4	4	280	9.41×10^{-5}	9.29×10^{-5}	0.99997	6.88	1019 s
DOPRI5	5	420	9.28×10^{-5}	9.21×10^{-5}	0.99997	6.89	1699 s
Tsit5	5	420	8.83×10^{-5}	8.92×10^{-5}	0.99997	6.91	1712 s

truncation error has fallen below the optimization noise floor. Heun is the exception: it has the same order and the same cost as Midpoint, yet extrapolates $3.5\times$ worse (3.10×10^{-4} against 8.95×10^{-5}). Two methods of the same order and cost should not differ systematically by that much, so this gap most likely reflects single-seed variance in where training settles. It also means the single-seed spread among second-order methods is at least $3.5\times$, which limits how much any one solver ranking in this table can be trusted. On this seed Midpoint matches Tsit5’s accuracy within 0.3% at one third of the function evaluations, but given Heun’s result, calling it the cost-normalized optimum would need more seeds to confirm.

The wall-clock column confirms that function evaluations are the operative cost. Training times of 254 : 499 : 498 : 1019 : 1699 : 1712 s, in ratios of 1 : 2.0 : 2.0 : 4.0 : 6.7 : 6.7, follow the evaluation counts’ ratios of 1 : 2 : 2 : 4 : 6 : 6 closely (the fifth-order methods carry about 12% extra overhead), because backpropagation unrolls every stage of every step, so each extra stage adds forward and backward work in proportion. This close match also indicates that the timings are not contaminated by scheduling noise. Figure 5 shows the training-loss histories: the five higher-order solvers track each other closely throughout (Heun lags briefly around epoch 5000), while Euler separates after about 2,000 epochs and descends more slowly for the rest of the run.

**Figure 5.** Training loss over 10,000 epochs for the six solvers (RBF basis, data spacing $\Delta t = 0.1$, two substeps). The recurring spikes are the optimizer bursts analysed in Section 8.

Step size. A separate sweep holds Tsit5 and the RBF basis fixed and varies the data spacing $\Delta t \in \{0.05, 0.1, 0.2\}$ over the same window $[0, 3.5]$, so the integration step $h = \Delta t/2$ and the number of training points change together (Table 6). Tsit5’s truncation error at these step sizes is tiny: on the test problem of Table 3 it is $\sim 10^{-10}$ at $h = 0.1$, while the extrapolation errors here correspond to root-mean-square errors of $\sim 10^{-2}$. The differences between rows are therefore very unlikely to come from the integrator’s accuracy, and the data agree: the finest step is *not* the most accurate. Extrapolation error is lowest at $\Delta t = 0.1$, $1.5\times$ higher

at 0.05 and $5.1 \times$ higher at 0.2, where only 19 training points remain. Halving the step doubles the function evaluations and the wall-clock time without buying accuracy. For a learned field, the step size matters through training (how many points the loss sees and how deep the backpropagation graph is), not through classical truncation error. Figure 6 shows the same data on log-log axes next to first- and second-order reference slopes: the extrapolation error follows neither, and the training error is almost flat across the three step sizes.

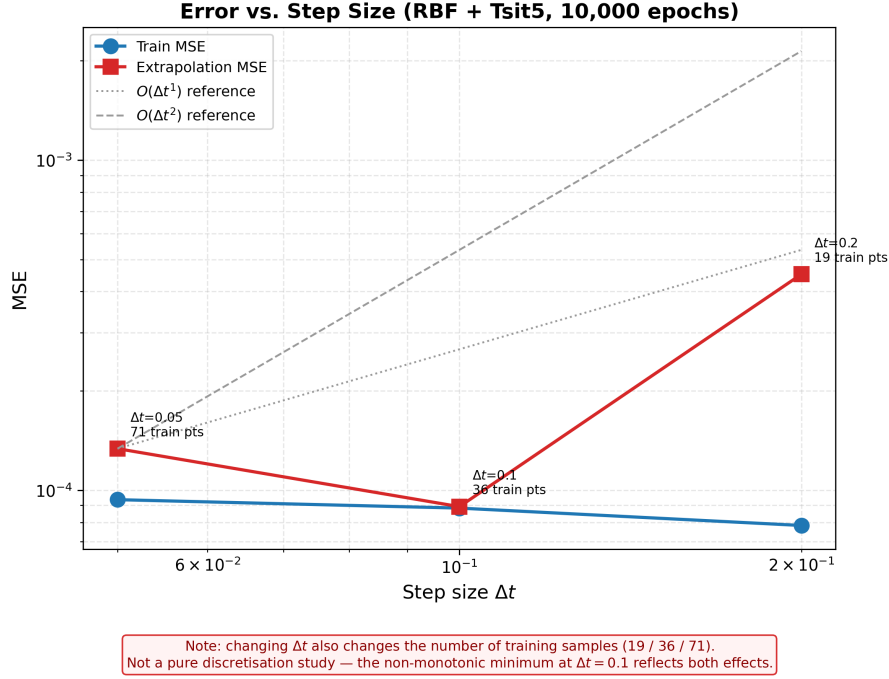


Figure 6. Training and extrapolation MSE against the data spacing Δt (Table 6), with $O(\Delta t)$ and $O(\Delta t^2)$ reference lines. The minimum at $\Delta t = 0.1$ is not a truncation-error effect, since the number of training points changes with Δt as well.

Table 6. Step-size sweep (Tsit5, RBF, two substeps, 10,000 epochs, training window $[0, 3.5]$). The $\Delta t = 0.1$ row is the reference configuration run again as part of this sweep: its errors match Tables 5 and 7 exactly, and its wall-clock differs from Table 7’s 1712 s by one second.

Δt	Step h	Training points	Function evaluations / epoch	Training MSE	Extrapolation MSE	L	Wall-clock
0.05	0.025	71	840	9.37×10^{-5}	1.34×10^{-4}	7.46	3425 s
0.1	0.05	36	420	8.83×10^{-5}	8.92×10^{-5}	6.91	1713 s
0.2	0.1	19	216	7.83×10^{-5}	4.51×10^{-4}	6.64	882 s

Writing z_g for the g -th of G evenly spaced grid centers with spacing δ (not the integration step h), and $u = \tanh(x) \in [-1, 1]$ the normalized edge input, the seven bases compared here (Figure 3) are

$$B_g^{\text{RBF}}(u) = \exp\left(-\frac{1}{2} \left(\frac{u - z_g}{\delta}\right)^2\right), \quad (26)$$

$B_g^{\text{spline}}(u)$ = Cox–de Boor recursion of degree k on the clamped knot vector of Section 3,

$$B_g^{\text{Cheb}}(u) = T_g(u) = \cos(g \arccos u), \quad B_g^{\text{Lag}}(u) = \prod_{j \neq g} \frac{u - z_j}{z_g - z_j}, \quad B_g^{\text{Newt}}(u) = \prod_{j < g} (u - z_j), \quad (27)$$

$$B_g^{\text{RSWAF}}(u) = \text{sech}^2\left(\frac{u - z_g}{\delta}\right) = 1 - \tanh^2\left(\frac{u - z_g}{\delta}\right), \quad B_g^{\text{IQF}}(u) = \frac{1}{1 + \left(\frac{u - z_g}{\delta}\right)^2}, \quad (28)$$

each substituted for B_i in Equation 2 and evaluated at $u = \tanh x$. RSWAF (reflectional switch activation function) and IQF (inverse quadratic) are, like the Gaussian, bell-shaped kernels centred on each grid point. The Gaussian and RSWAF decay exponentially away from the centre, while IQF decays only as $1/r^2$ and

so has much heavier tails. The Cox–de Boor recursion builds a degree- k B-spline from degree-0 indicator functions by repeated linear blending over the knots t_g (the clamped knot vector, not the grid centres z_g), $B_{g,0}(u) = \mathbf{1}[t_g \leq u < t_{g+1}]$ and $B_{g,p}(u) = \frac{u-t_g}{t_{g+p}-t_g}B_{g,p-1}(u) + \frac{t_{g+p+1}-u}{t_{g+p+1}-t_{g+1}}B_{g+1,p-1}(u)$, with a term dropped whenever its denominator vanishes at a repeated knot. This is what gives it compact support: $B_{g,k}(u) = 0$ once u leaves the $k+2$ knots the recursion references. None of the other six bases has such a cutoff: every coefficient c_g influences $\phi(x)$ at every x . For the polynomial bases (Chebyshev, Lagrange, Newton) that influence does not decay at all away from z_g , which is one plausible reason they extrapolate 4–400 $\times$ worse than the Gaussian once the trajectory reaches states their coefficients were not fitted on (Table 7); support alone does not set the ranking, as the bell-shaped kernels below show. The residual term $w_b \text{silu}(x)$ in Equation 2 is the same for every basis choice, since it is a fixed part of the edge parameterization, not one of the substituted terms above.

Table 7. Basis ablation (fixed Tsit5 solver, $G = 5$ grid points, 240 parameters).

Basis function	Training MSE	Extrapolation MSE	R^2	Lipschitz L	Wall-clock time
Cubic B-spline	7.20×10^{-5}	8.38×10^{-5}	0.99997	8.44	6680 seconds
Gaussian RBF	8.83×10^{-5}	8.92×10^{-5}	0.99997	6.91	1712 seconds
Chebyshev	9.80×10^{-5}	4.37×10^{-4}	0.99985	6.48	2317 seconds
Lagrange	1.07×10^{-4}	3.93×10^{-4}	0.99987	6.70	6829 seconds
IQF	2.08×10^{-4}	9.61×10^{-4}	0.99967	7.16	1824 seconds
RSWAF	1.00×10^{-4}	1.46×10^{-3}	0.99950	8.72	1715 seconds
Newton	1.47×10^{-3}	3.65×10^{-2}	0.98746	5.65	2187 seconds

B-spline edges out RBF on accuracy, at $3.9\times$ RBF’s wall-clock time and a higher Lipschitz estimate (8.44 vs. 6.91). This result is for the B-spline with clamped boundary knots (Section 3), which keeps its basis functions summing to one right up to the edges of $[-1, 1]$. An earlier version used an unclamped knot vector, which left only about 17% coverage at the domain edges and produced a diverging model. A tempting explanation of the higher Lipschitz estimate is that compact local support lets the B-spline fit sharper local features. The longer-budget runs do not support it: at 25,000 epochs the B-spline’s estimate (7.05) is *lower* than RBF’s at 50,000 (7.75, Table 28). The two budgets differ, so this is not a matched comparison, but it is enough to show that the ordering of L between the two bases is not stable, and it is not attributed to support here. The two bases are statistically tied on error (6% apart, inside single-seed noise), which makes RBF the better accuracy-per-second choice. Lagrange is the slowest basis (6829 s) even though its formula is closed form, because this implementation builds each cardinal polynomial with an unvectorized double loop over grid points: $G(G-1)$ small tensor operations per basis call, on every layer of every field evaluation, so per-operation overhead dominates. Its cost is an implementation artifact, not a property of the basis. Polynomial global-support bases (Chebyshev, Lagrange) fit the training window well and extrapolate 4–5 $\times$ worse – the expected signature of a basis whose every coefficient influences the whole domain rather than a local region of it. Support is not the whole story, however. RSWAF and IQF are bell-shaped kernels just like the Gaussian, cost the same to evaluate, and fit the training window to within about $2.4\times$ of it, yet they extrapolate $16\times$ (RSWAF) and $11\times$ (IQF) worse than it. The shape of each kernel, not only whether it is local, decides how the fitted field behaves outside the training window; with one seed per basis, this gap is a measured ranking, not an explained one. Newton’s basis is the outlier on smoothness in the opposite direction: its lowest Lipschitz constant in the table (5.65) reflects underfitting, not desirable flatness, since it is also the only basis that never reaches 10^{-3} training loss.

The training histories (Figure 7) show the same ordering developing over time. The B-spline has the lowest best-so-far loss for most of the first 2,000 epochs and again from about epoch 6300 onward. Chebyshev is lowest in between (RSWAF briefly ties it near epoch 5400) and reaches 10^{-3} first (at epoch 3892, against RBF’s 5282), but it does not keep that lead. Newton stays about an order of magnitude above the best basis throughout ($10\text{--}20\times$).

6.2 Testing the paper’s KAN-versus-MLP claims

The base paper’s two headline claims on Lotka–Volterra are that its 240-parameter KAN-ODE is more accurate than a similarly sized MLP Neural ODE, and that it gets there with about $10\times$ fewer epochs: 2.6×10^{-5} training

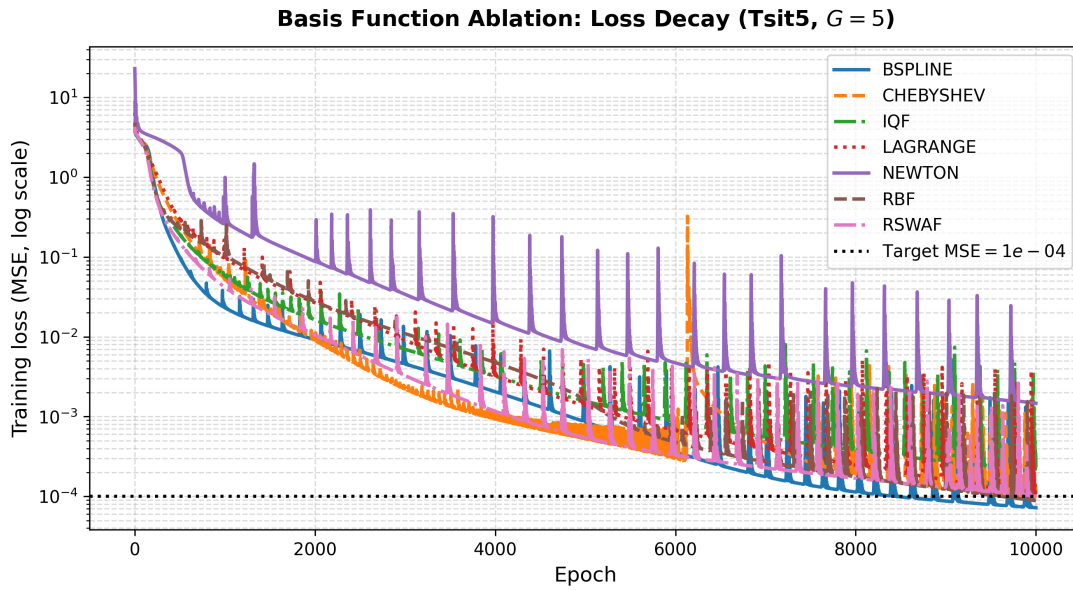


Figure 7. Training loss over 10,000 epochs for the seven bases (Tsit5, $G = 5$).

loss after 10^4 epochs, already below the MLP’s 3×10^{-5} after 10^5 [11]. Table 8 tests both under this project’s protocol, against the paper’s literal MLP and against a parameter-matched MLP that trains.

Table 8. KAN-ODE vs. MLP Neural ODEs on Lotka-Volterra, identical protocol (Tsit5, two substeps, Adam at 2×10^{-3} , seed 42, 10,000 epochs). “Epochs to 10^{-k} ” is the first epoch whose training loss falls below that level. The 50,000-epoch row comes from the budget check of Section 12, whose runs also used gradient clipping at 1.0 and a different platform.

Quantity	KAN-ODE	MLP-ODE (SiLU)	MLP-ODE (paper: tanh)
Architecture	$[2, 10, 2]$, $G = 5$, RBF	$[2, 14, 8, 8, 2]$, SiLU	$[2, 50, 2]$, tanh
Parameters	240	252	252
Training MSE	8.83×10^{-5}	9.50×10^{-5}	1.08
Extrapolation MSE	8.92×10^{-5}	4.77×10^{-4}	1.69
Extrapolation R^2	0.99997	0.99984	0.42
Relative L_2 error	0.33%	0.76%	45.2%
Lipschitz estimate L	6.91	5.07	38.2
Epochs to 10^{-3}	5282	1297	never
Epochs to 10^{-4}	9617	9145	never
Seconds per epoch (CPU)	0.1705	0.2856	0.0736
Extrapolation MSE at 50,000 epochs, with gradient clipping	4.02×10^{-5}	5.90×10^{-6}	4.32

At the 10,000-epoch budget the two claims split. On accuracy, KAN-ODE extrapolates $5.3 \times$ better than the parameter-matched MLP from statistically identical training loss (8% apart), which agrees with the paper. On speed, the MLP reaches 10^{-3} training loss $4.1 \times$ sooner (epoch 1297 against 5282) and also reaches 10^{-4} first, which contradicts the paper’s tenfold claim. Per epoch, KAN-ODE costs $2.3 \times$ the paper-spec MLP (0.1705 vs. 0.0736 s), close to the paper’s own ratio of 19 against 7 minutes per 10^4 epochs [11]. The SiLU MLP’s per-epoch time was measured in a different batch of runs, on a different PyTorch build; in the 50,000-epoch reruns, on one platform, it was faster than the KAN-ODE (0.27 against 0.40 s), so per-epoch differences involving it are not interpreted.

The accuracy advantage does not survive a longer run. Retrained to 50,000 epochs (last row; Section 12), the parameter-matched MLP’s extrapolation error falls to 5.90×10^{-6} , $6.8 \times$ lower than KAN-ODE’s. Those reruns also used gradient clipping, which the 10,000-epoch runs did not, and ran on a different platform, and these changes were not separated from the budget. The KAN-ODE lead measured at 10,000 epochs is therefore a statement about that setting, not a stable property of the two architectures on this problem. Both runs use a

single seed.

Figure 8 shows where the speed difference comes from. The SiLU MLP’s training loss falls much faster over the first 1,000 epochs and stays below the KAN-ODE’s for most of the run, but it is interrupted by large recurring spikes. The KAN-ODE descends more slowly, with smaller spikes that appear mostly late in the run, and the two meet near 10^{-4} only at the end of the budget. The paper’s tanh MLP levels off near a training loss of 1 by about epoch 2000, apart from a brief burst of spikes near epoch 2700, and improves only from 1.21 to 1.08 after that.

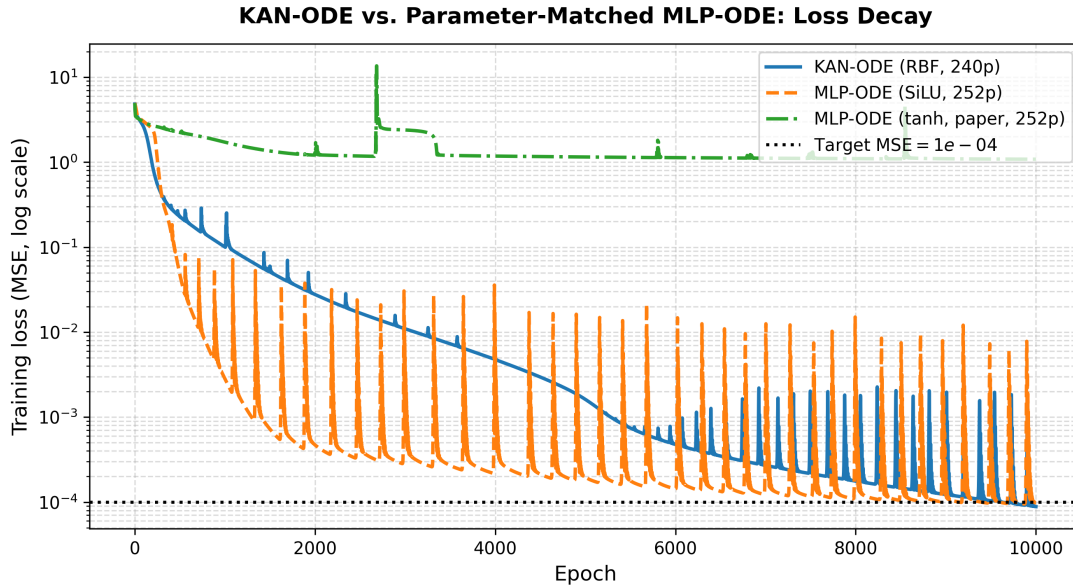


Figure 8. Training loss over 10,000 epochs for the KAN-ODE and the two MLP baselines of Table 8.

Phase-space geometry. Trajectory error alone does not show how each model fails. Figure 9 plots each model’s predicted orbit against the true closed orbit. The KAN-ODE and the SiLU MLP both close the orbit correctly and are hard to tell apart by eye. Both show slight jaggedness at the right-hand turning point, where the true orbit bends most sharply, and the MLP’s extrapolated segment sits slightly off the true orbit near its top ($x \approx 3-4$). The $5.3\times$ gap in extrapolation MSE between them is therefore a small difference in absolute terms, not a qualitative one. The paper-specification tanh MLP converges to a different, stable but wrong closed orbit: its prey range is roughly right, but its predator range collapses to $[1.0, 1.9]$ against the true $[0.26, 4.58]$, and its training and extrapolated segments coincide, so it has not diverged. Its Lipschitz estimate of 38.2, against the KAN-ODE’s 6.9, reflects a much sharper learned field. Figure 10 shows the KAN-ODE’s learned vector field itself, whose streamlines carry the trajectory around the orbit. Away from the orbit the field is not constrained by any data, and its streamlines there need not match the true system’s.

Table 9. Controlled breakdown of why the paper’s literal MLP baseline architecture, $[2, 50, 2]$ with hyperbolic tangent (tanh) activation, fails to train under this project’s protocol. All four runs use 2000 epochs. These four runs have no archived result files in the repository.

Run	Architecture	Activation	Learning rate	Training MSE	Extrapolation MSE	R^2
A	$[2, 50, 2]$	tanh	2×10^{-3}	1.21×10^0	1.98×10^0	0.319
B	$[2, 50, 2]$	tanh	5×10^{-4}	1.85×10^0	9.23×10^0	-2.169
C	$[2, 14, 8, 8, 2]$	SiLU	2×10^{-3}	4.28×10^{-4}	6.97×10^{-4}	0.99976
D	$[2, 50, 2]$	SiLU	2×10^{-3}	3.62×10^{-3}	2.68×10^2	-91.06

Runs A and D share the $[2, 50, 2]$ architecture and differ only in activation. With SiLU (run D) the network fits the training window $334\times$ better than with tanh (3.62×10^{-3} against 1.21), but it extrapolates $135\times$ worse (268 against 1.98): SiLU lets this architecture fit the data, but not generalize from it. Only run C, which changes both

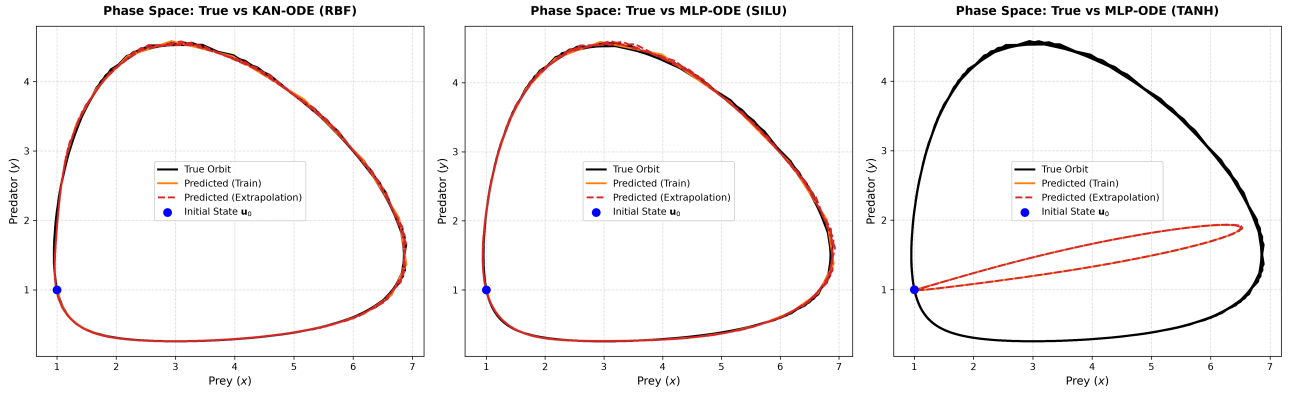


Figure 9. Predicted Lotka-Volterra orbits at 10,000 epochs. Left: KAN-ODE (RBF, 240 parameters). Middle: parameter-matched SiLU MLP. Right: the paper’s $[2, 50, 2]$ tanh MLP. Solid orange is the training window, dashed red the extrapolation, black the true orbit.

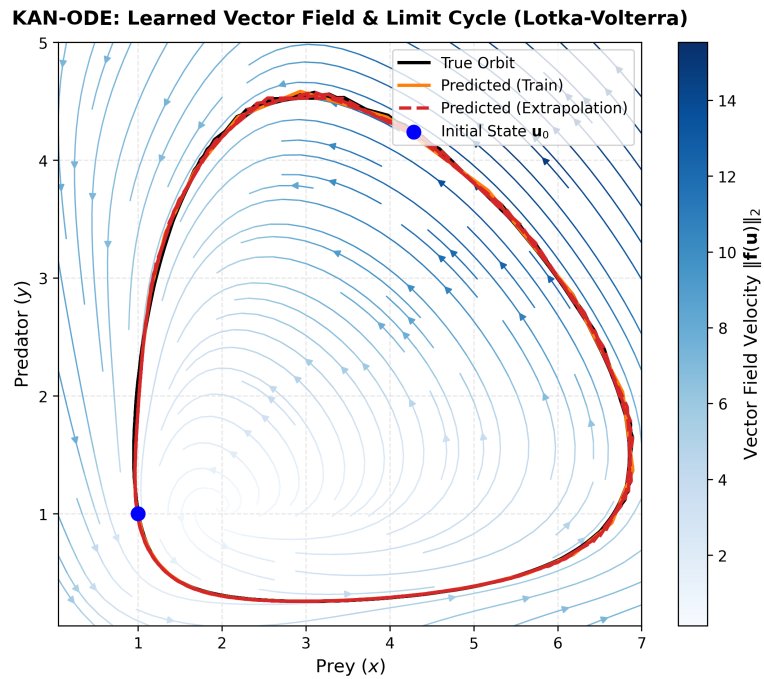


Figure 10. The KAN-ODE’s learned vector field on Lotka-Volterra (streamlines, colored by $\|f_\theta\|$), with the true and predicted orbits.

the depth and the widths while keeping SiLU, both fits and extrapolates. Because C differs from D in depth and width together, the table does not isolate either one. Lowering the learning rate (run B) does not rescue tanh.

The conclusion this supports is narrow. Under this recipe, the paper’s [2, 50, 2] tanh network fails to train, and Section 12 confirms that more epochs do not rescue it. It does not show that the paper’s own MLP result is wrong. The paper trained this network successfully, and its code differs from this recipe in two ways that plausibly matter for a tanh network: a *higher* learning rate of 10^{-2} rather than 2×10^{-3} , and initial weights divided by 10^5 , so the network starts near zero (Table 30). Whether those settings rescue tanh was not tested.

6.3 Robustness to observation noise

The reference KAN-ODE recipe was retrained with Gaussian noise of standard deviation $\sigma \in \{0.01, 0.05, 0.1\}$ added to the 36 training observations only, and scored against the clean trajectory (Section 2). Table 10 gives the results. The same four models are the KAN-ODE side of the sparse-regression comparison of Section 11.

Table 10. KAN-ODE under observation noise (Lotka-Volterra, RBF, Tsit5, 10,000 epochs). Noise is added to the training points only; every metric is computed against the clean trajectory.

Noise σ	Training MSE	Extrapolation MSE	Extrapolation R^2	Relative L_2	Extrapolation MSE vs. clean
0.00	8.83×10^{-5}	8.92×10^{-5}	0.99997	0.33%	–
0.01	1.23×10^{-4}	1.22×10^{-3}	0.99958	1.22%	13.6×
0.05	7.41×10^{-4}	2.54×10^{-2}	0.99129	5.55%	284×
0.10	2.28×10^{-3}	1.37×10^{-1}	0.95282	12.92%	1540×

The degradation is steady rather than abrupt. Between $\sigma = 0.01$ and 0.1 a $10\times$ increase in noise raises the extrapolation error $113\times$, close to the σ^2 scaling expected when noise in the fitted field simply propagates through the integration, and with no sign of a stability breakdown. Even at $\sigma = 0.1$ the model still explains 95% of the extrapolation variance and keeps the orbit closed. The noisier runs are also the only ones in the Lotka-Volterra ablation that clearly stopped improving: their best training loss changed by only 8–9% over the last 2,000 epochs, against a factor of 1.5–2 for the clean and $\sigma = 0.01$ runs, consistent with having reached the floor set by the noise.

6.4 Diagnosing and fixing two cross-domain failures

Applying the Lotka-Volterra recipe unchanged to the damped pendulum and to SIR failed on both systems, each for its own reason. Neither was fixed by a hyperparameter search. The pendulum was settled by a small factorial control that crossed training-window length with activation; SIR was diagnosed by measuring the untrained network’s field against the true one before any training step.

Pendulum: extending the training window. The pendulum recipe originally trained on $t \in [0, 3]$ and diverged in extrapolation ($R^2 = -1.318$). Extending the training window to $t_{\text{train}} = 5.0$ – a single flag – resolves it:

Question	Why does the Lotka-Volterra recipe, applied unchanged, diverge on the pendulum?
Expected (hypothesis)	SiLU’s residual branch cannot represent the sign-changing derivative $\dot{\theta} = \omega$ – an architectural limit.
Observed	Refuted. The longer window removes the divergence under both activations, and SiLU extrapolates better of the two; SiLU fits the sign change fine once the window is long enough.

An earlier hypothesis held that SiLU’s residual branch $W \cdot \text{silu}(x)$ cannot represent a sign-changing derivative such as $\dot{\theta} = \omega$. A factorial control comparing activation against training-window length refutes it directly: with the longer window, SiLU fits the angle to a root-mean-square error of 0.0114 rad inside the training window and reproduces the true turning point ($\theta_{\min} = -1.380$ against a true value of -1.385). On the common test window the extended window moves R^2 from -4.64 to $+0.647$ (Table 11); the -1.318 originally quoted for the failing model was scored on its own, longer window $(3, 10]$ and understates the failure on the window the fixed model is judged on. The “identity” variant replaces the residual path $W \text{silu}(x)$ of Equation 9 by the linear Wx , leaving the basis expansion unchanged. With the longer window it also stops diverging, but it extrapolates worse than SiLU ($R^2 = +0.473$ against $+0.647$). Its advantage is fitting speed only: at the short window it

Table 11. Pendulum fix: the original failure, the adopted configuration, and the identity-residual variant. Because R^2 is normalized by the variance of the window it is computed on, each model’s own test window is not a fair comparison across training windows; the “common window” column scores every model on the same interval $(5, 10]$. Full-horizon MSE on $[0, 10]$ is also window-independent. All runs 10,000 epochs.

Configuration	Full-horizon MSE $[0, 10]$	R^2 , own test window	R^2 , common window $(5, 10]$	Verdict
Original ($t_{\text{train}} = 3.0$, SiLU)	8.99×10^{-1}	-1.318 on $(3, 10]$	-4.64	diverges
Extended window ($t_{\text{train}} = 5.0$, SiLU)	4.48×10^{-2}	$+0.647$ on $(5, 10]$	$+0.647$	adopted
Extended window ($t_{\text{train}} = 5.0$, identity residual)	6.68×10^{-2}	$+0.473$ on $(5, 10]$	$+0.473$	not adopted

reaches a roughly $1000\times$ lower training loss in a fifth of the epochs, and at the long window it fits the training window $3.6\times$ better, yet in both cases it does not match the SiLU model’s extrapolation. Figure 11 shows the adopted model.

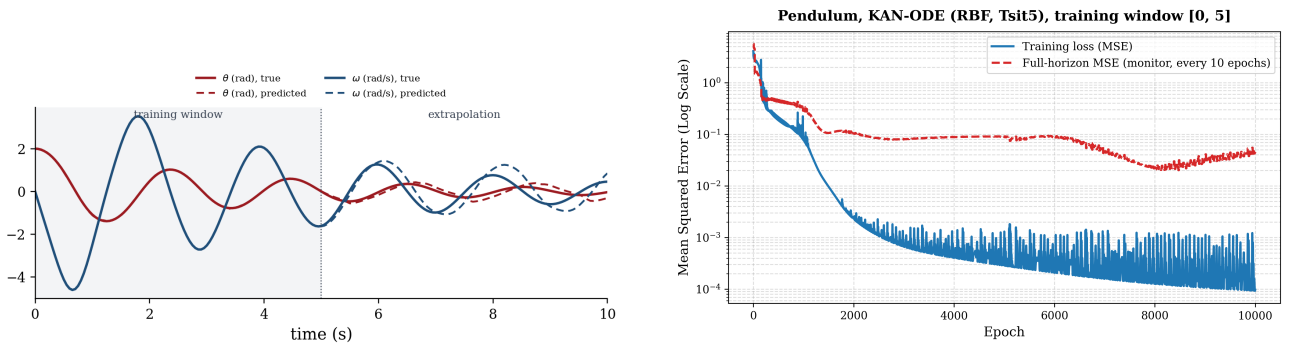


Figure 11. The adopted pendulum recipe (SiLU, training window $[0, 5]$, extrapolation $R^2 = +0.647$). Left: true (solid) and predicted (dashed) angle and angular velocity; the fit is close inside the training window, and in extrapolation the prediction keeps the right frequency but gradually lags and decays too slowly (the under-damping analysed at the end of this section). Right: training and full-horizon loss curves.

SIR: a unit-scale mismatch. The original 10,000-epoch SIR run (gradient clipping at 1.0, no other fix) reached its best checkpoint at epoch 6925, saw its gradient norm climb to 5.82×10^{18} , produced its first non-finite gradient at epoch 8686 and a NaN loss at epoch 8702, and ended with invalid final weights.

Question What causes SIR’s divergence, and is it a scale problem, an optimizer defect, or a modeling gap?

Expected Measuring the untrained network’s own field magnitude against the true dynamics, before any training, should reveal whether the two are mismatched in scale.

Observed A roughly $18\text{--}19\times$ speed mismatch at initialization. Rescaling time removes the divergence entirely; no optimizer or architecture change was needed.

Measuring the learned field magnitude *before any training step* explained the divergence directly. A Glorot-initialised KAN starts with $|f_\theta| \approx 0.203$ against SIR’s true field magnitude of 0.011 – a roughly $18\text{--}19\times$ speed mismatch. To see why this is catastrophic rather than merely inaccurate, consider that backpropagation through an explicit solver differentiates roughly $T/\Delta t$ nested nonlinear steps, and the resulting gradient sensitivity grows exponentially as e^{LT} , where L is the Lipschitz constant of $\mathbf{f}_\theta$ and T is the integration horizon. Lotka-Volterra trains at $T = 3.5$ with $|f| \sim 10^0$ and converges; the pendulum trains at $T = 5.0$ with $|f| \sim 10^0$ and converges after the window fix. SIR’s true field is small ($|f| \sim 10^{-2}$), but what sets the sensitivity is the horizon: SIR is integrated to $T = 50$, and the network’s Lipschitz constant is $L = O(1)$ at initialization, so $LT \approx 50$ and e^{LT} is astronomically large. Trouble starts almost immediately: a 2,000-epoch diagnostic rerun of the same recipe shows the (pre-clipping) gradient norm spiking to 2×10^6 at epoch 11. Clipping keeps each update finite, so that run does not break down, but the spike shows the sensitivity is present from the first epochs, not only near epoch 8686.

The optimizer was behaving rationally: with an epoch-0 loss of 3.16×10^2 , collapsing the field to $\mathbf{f}_\theta \rightarrow \mathbf{0}$ reduces the loss to $\approx 10^{-1}$ – a $3,000\times$ improvement reachable immediately. SIR’s natural timescale is the

recovery period $1/\gamma = 10$ days; substituting $\tau = t/10$ gives $dy/d\tau = 10f(y)$ over $\tau \in [0, 5]$, landing SIR in the same regime the other two systems already train in. Projecting the learned field onto the zero-sum subspace $f_{\text{proj}}(y) = f(y) - \frac{1}{n} \sum_i f_i(y)$ (Equation 16) fixes a separate mass-drift symptom by enforcing the SIR invariant (Equation 5) as a property of the flow rather than a penalty on the fit.

These two fixes remove the divergence and the mass drift, but they leave the end of the outbreak poorly predicted: extrapolation R^2 rises from -20.97 only to -0.145 (Table 12, middle column). The remaining error comes from where the test window sits in state space. The window $t \in (50, 80]$ has $S \approx 0.03$ – 0.04 and small I , a corner the training data visits at only one of its 101 points. A third fix, the vanishing-field gate on I (Equation 17), supplies the missing behaviour by construction: every SIR transition is proportional to I , so the gated field goes to zero with I exactly as the true one does. With the gate added, extrapolation R^2 reaches $+0.9981$ (Figure 12). Unlike the first two fixes, the gate is a physical assumption about the system, so the two results are reported separately.

Table 12. SIR at the full 10,000-epoch budget: before any fix, with the two assumption-free fixes (time rescaling $s = 10$ and zero-sum projection), and with the vanishing-field gate on I added. The first two fixes assume nothing beyond what SIR’s equations state; the gate adds a physical prior.

Metric	Before fix (clipping only)	Rescaling + projection	+ vanishing gate
Training MSE	4.39×10^{-4}	1.08×10^{-5}	2.28×10^{-8}
Extrapolation RMSE	0.0614	0.0140	5.70×10^{-4}
Infected RMSE (extrapolation)	0.0776	0.0194	0.0003
Mass error $\max S + I + R - 1 $, test window	0.0045	0.0000	0.0000
Extrapolation R^2	-20.97	-0.145	+0.9981
Maximum gradient norm	5.82×10^{18}	92.3	1.19
Final model	invalid (NaN)	finite	finite

Both fixed runs select their best checkpoint well before the end of training (epochs 8807 and 7857), and in both the final-epoch model is worse than the selected one. The reported checkpoints are sound, but a mild late-training instability remains and is not claimed to be fixed.

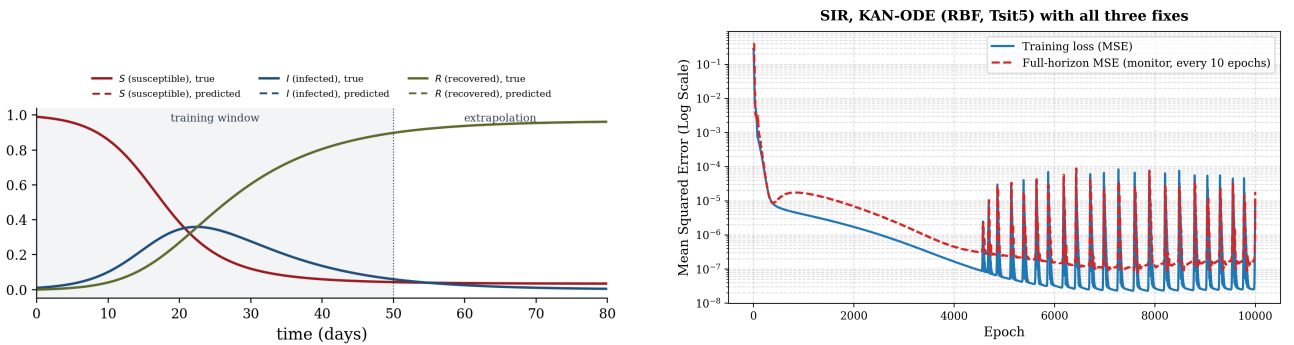


Figure 12. SIR with all three fixes (right-hand column of Table 12). Left: the predicted S, I, R (dashed) lie on top of the true curves (solid) over the whole horizon, including the end of the outbreak after day 50. Right: training and full-horizon loss curves, with no gradient overflow.

Other fixes considered. Several additional interventions were tested and are recorded for completeness rather than adopted; Table 13 lists every intervention with its verdict.

Table 13. Every stability intervention tested, including the rejected ones.

Intervention	Verdict	Evidence
Longer training window (pendulum)	Adopted	R^2 on the common window $(5, 10]$ improves from -4.64 to $+0.647$; removes the divergence under SiLU and the identity residual (a tanh variant at 2,000 epochs reaches only $R^2 = -0.03$)
Non-finite gradient guard with automatic abort	Adopted	In an intermediate SIR run (penalty-based conservation, no time rescaling), it caught the first non-finite gradient at epoch 3055, aborted after 100 consecutive bad steps, and kept the valid epoch-2877 checkpoint instead of overwriting it
Time rescaling (SIR)	Adopted	Removes both the divergence and the frozen field, in which the model learns $\mathbf{f}_\theta \approx \mathbf{0}$ and its extrapolation barely moves (I drifts by 0.0003 over the test window against a true 0.0525)
Conservation via subspace projection (SIR)	Adopted	Extrapolation mass error $ S + I + R - 1 $ from 0.0045 (no fix) and 0.0086 (penalty run) to 0.0000 at four decimals (Table 12)
Vanishing-field gate on I (SIR)	Adopted, as a physical prior	Extrapolation R^2 from -0.145 to $+0.9981$ on top of the two fixes above
Identity activation (pendulum)	Speed only	Roughly $1000\times$ lower training loss in a fifth of the epochs at the short window, and a $3.6\times$ better training fit at the long window, but worse extrapolation there
Standard-deviation loss weighting	System-dependent	Worst-performing pendulum option; superseded for SIR by time rescaling
Conservation via penalty term	Rejected	Wrong mechanism – the training window’s mass was already close to conserved, so the penalty had nothing to correct
Widened basis-function grid limits	Rejected	Performed worse than the untouched baseline configuration

Methodological lesson carried into later studies

The SIR diagnosis came from measuring the untrained network’s field against the true field before any training step. That measurement showed that a rescaling of time was needed before a single run was spent on it. It did not set the factor: $s = 10$ comes from placing the rescaled training horizon in the regime where the other systems train ($T/s = 5$), which for SIR coincides with its recovery time $1/\gamma = 10$ days. The same init-time measurement is reused in the non-mechanistic epidemic test of Section 11. The pendulum was settled differently, by a factorial control, and is the reminder that a control arm is needed even when a diagnosis looks mechanistic.

6.5 Extended extrapolation and its budget dependence

With the fixes in place, the Lotka-Volterra horizon was doubled from $t_{\text{end}} = 14$ (about 4.2 cycles in total, 3.2 of them unseen) to $t_{\text{end}} = 28$ (about 8.4 cycles, 7.4 unseen), using the same trained models with no retraining.

Table 14. Extrapolation MSE as the horizon doubles, same trained models.

Model	Training $[0, 3.5]$	Near extrapolation $(3.5, 14]$	Far extrapolation $(14, 28]$	Degradation
KAN-ODE (RBF, 240 parameters)	8.83×10^{-5}	8.92×10^{-5}	9.72×10^{-5}	1.1$\times$
MLP-ODE (SiLU, 252 parameters)	9.50×10^{-5}	4.77×10^{-4}	3.95×10^{-3}	8.3 $\times$
MLP-ODE (tanh, paper specification)	1.08×10^0	1.69×10^0	4.03×10^0	2.4 $\times$

The KAN-ODE’s error is essentially flat across the doubled horizon (Figure 13; $8.83 \rightarrow 8.92 \rightarrow 9.72 \times 10^{-5}$, a 10% rise from the training window to the far window), while the parameter-matched MLP degrades $8.3\times$ over the same extension. At this budget the KAN-ODE advantage therefore widens with horizon: from $5.3\times$ on the near window to **40.6 $\times$** on the far window, because the KAN-ODE’s error does not accumulate from one cycle to the next and the MLP’s does.

This conclusion depends on the training setting. The same three windows were scored for the 50,000-epoch checkpoints of Section 12, which were also trained with gradient clipping, on a different platform (Table 15;

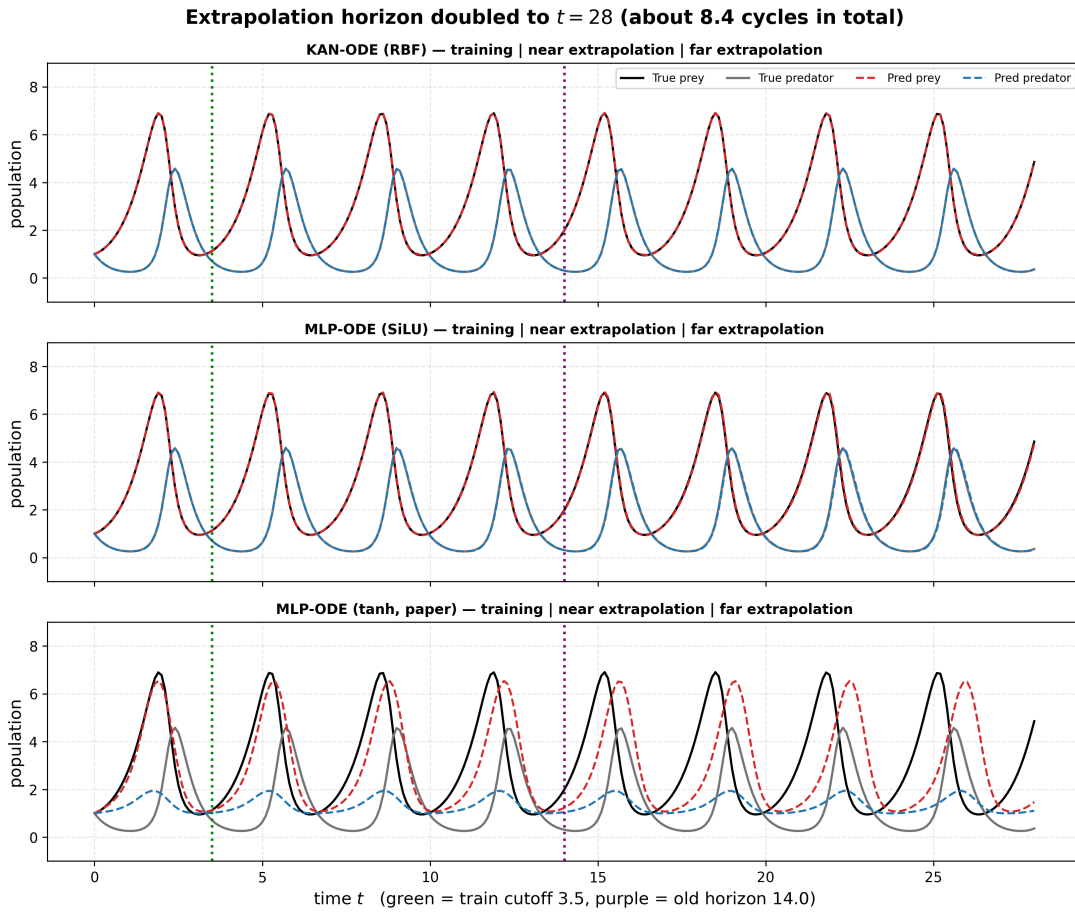


Figure 13. The three 10,000-epoch models integrated to $t = 28$ (green line: end of training at $t = 3.5$; purple line: original horizon $t = 14$). The KAN-ODE and the SiLU MLP stay on the true cycle; the MLP's predator peaks begin to lag slightly after $t \approx 20$. The paper's tanh MLP keeps a stable but wrong cycle: its predator amplitude is far too small and its prey peaks lag further behind every cycle.

the same scoring code reproduces every 10,000-epoch entry of Table 14 to three significant figures, except the SiLU MLP’s near-window MSE, 4.78 against 4.77×10^{-4}). With five times the training, both models fit the training window far better, but the KAN-ODE’s error is no longer flat: it grows $6.2\times$ from the near to the far window, against the MLP’s $3.9\times$. The MLP is then more accurate on both windows, by $6.8\times$ near and $10.8\times$ far.

Table 15. The horizon comparison repeated at 50,000 epochs, with gradient clipping at 1.0 (Section 12); same windows as Table 14. “Degradation” is far-window MSE divided by near-window MSE, as in Table 14.

Model (50,000 epochs)	Training $[0, 3.5]$	Near extrapolation (3.5, 14]	Far extrapolation (14, 28]	Degradation
KAN-ODE (RBF, 240 parameters)	6.59×10^{-6}	4.02×10^{-5}	2.50×10^{-4}	$6.2\times$
MLP-ODE (SiLU, 252 parameters)	1.82×10^{-6}	5.90×10^{-6}	2.31×10^{-5}	$3.9\times$
MLP-ODE (tanh, paper specification)	1.01×10^0	4.32×10^0	7.49×10^0	$1.7\times$

Budget caveat

The $40.6\times$ KAN-ODE advantage and its flat error across horizons describe the two models at 10,000 epochs without clipping. At 50,000 epochs with clipping, on a different platform (Section 12), the parameter-matched MLP is more accurate at every horizon tested. On this problem the KAN-ODE’s lead is a lead in how fast it reaches a good extrapolating field, not in how good a field it can eventually reach (single seed; the effects of budget, clipping and platform were not separated).

6.6 Pendulum energy dissipation: an independent physical check

The fixed pendulum recipe captures 94.2% of the true energy dissipation and tracks $E(t)$ inside the training window to a root-mean-square error of 0.6% of E_0 , but the strict monotonic-decay invariant ($dE/dt \leq 0$, Equation 4) is violated on 40 of 200 predicted steps. The model is under-damped: over $t > 9$ the true angle stays within ± 0.17 rad while the prediction still swings to 0.44 rad. This energy diagnostic is an independent confirmation of the under-damping, expressed as a violated conservation law rather than a trajectory RMSE (Figure 14): after the training window ends at $t = 5$, the predicted energy stops tracking the true curve and stays well above it, between about 0.3 and 1.5, with small upward jumps, while the true energy keeps decaying toward zero.

Damped Pendulum ($\mu = 0.5$): Phase Portrait & Energy Dissipation

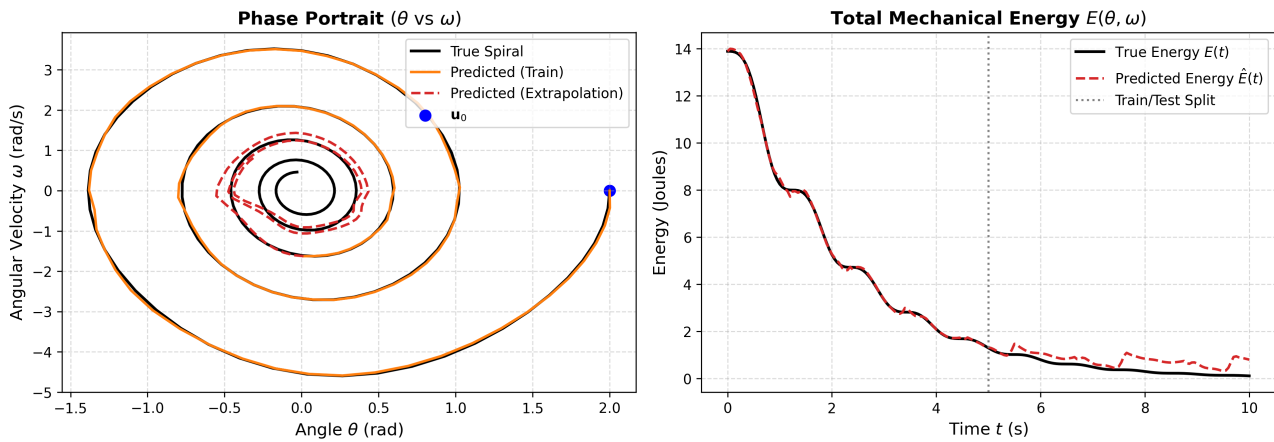


Figure 14. The adopted pendulum model ($\mu = 0.5$, training window $[0, 5]$). Left: predicted and true phase-space spirals; in extrapolation the predicted spiral stops contracting and circles at too large an amplitude. Right: predicted and true mechanical energy.

7 Adjoint Sensitivity vs. Direct Autograd: Memory and Wall-Clock Cost

Two methods exist for computing gradients through a neural ODE. **Direct autograd** treats the numerical integrator as an ordinary computational graph: each of the solver’s $(N_t - 1)m$ forward steps (m substeps per output interval) is recorded, and the reverse pass walks back through every recorded step in turn, so peak memory grows linearly with the number of steps taken. **Adjoint sensitivity** [3] instead treats the forward solve as a black box and derives a second, augmented ODE that produces the same gradient without storing every intermediate state. Writing $\mathbf{a}(t) = \partial L / \partial \mathbf{u}(t)$ for the adjoint state, it satisfies its own backward-time differential equation,

$$\frac{d\mathbf{a}(t)}{dt} = -\mathbf{a}(t)^\top \frac{\partial \mathbf{f}_\theta}{\partial \mathbf{u}}(\mathbf{u}(t)), \quad \frac{dL}{d\theta} = - \int_{t_1}^{t_0} \mathbf{a}(t)^\top \frac{\partial \mathbf{f}_\theta}{\partial \theta}(\mathbf{u}(t)) dt, \quad (29)$$

written here for a loss evaluated at t_1 ; for a loss over many observation times, as in Equation 8, the adjoint picks up a jump $\partial L / \partial \mathbf{u}(t_n)$ at each observation. It is solved backward from t_1 to t_0 alongside the original state $\mathbf{u}(t)$ (recomputed rather than stored), so memory depends only on the size of the state and parameter vectors, not on how many steps the forward solve took – an $O(1)$ memory cost in trajectory length, against direct autograd’s $O(N_t)$. This project’s own implementation documentation adopts direct autograd on the stated grounds that this $O(1)$ advantage does not matter for this project’s short trajectories: the main training windows have $N_t \in [36, 101]$ points (19–71 in the step-size sweep), and the longest trajectory in any training or evaluation run has 281 (the doubled Lotka-Volterra horizon of Table 14, and the $\Delta t = 0.05$ step-size run). This section measures that claim directly. Unlike every other timing in this report, it runs on a GPU (NVIDIA RTX 3060, 12 GiB, PyTorch 2.5.1 with CUDA 12.1), because peak device memory is the quantity being measured. The adjoint path is one of the two components in this report taken from a library rather than written for the project (Section 3): `torchdiffeq`’s `odeint_adjoint` [2], compared against this project’s own integrator with direct autograd. The integrator is held fixed on both paths (classical RK4 at matched step size: 0.05 for Lotka-Volterra, and for SIR, which is trained with the time rescaling $s = 10$ of Section 6, 0.025 in rescaled time τ , i.e. 0.25 days; a forward-trajectory check confirms the two implementations agree to $\sim 10^{-7}$ relative error), so only the differentiation method varies. The training comparison uses the two training-window lengths, $N_t = 36$ and 101; the memory sweep extends to $N_t \in \{36, 101, 201, \dots, 3201\}$ to look for a crossover.

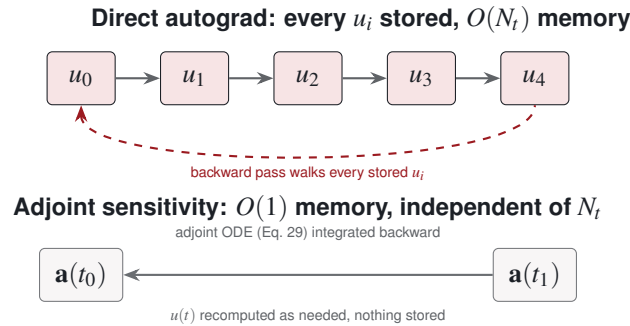


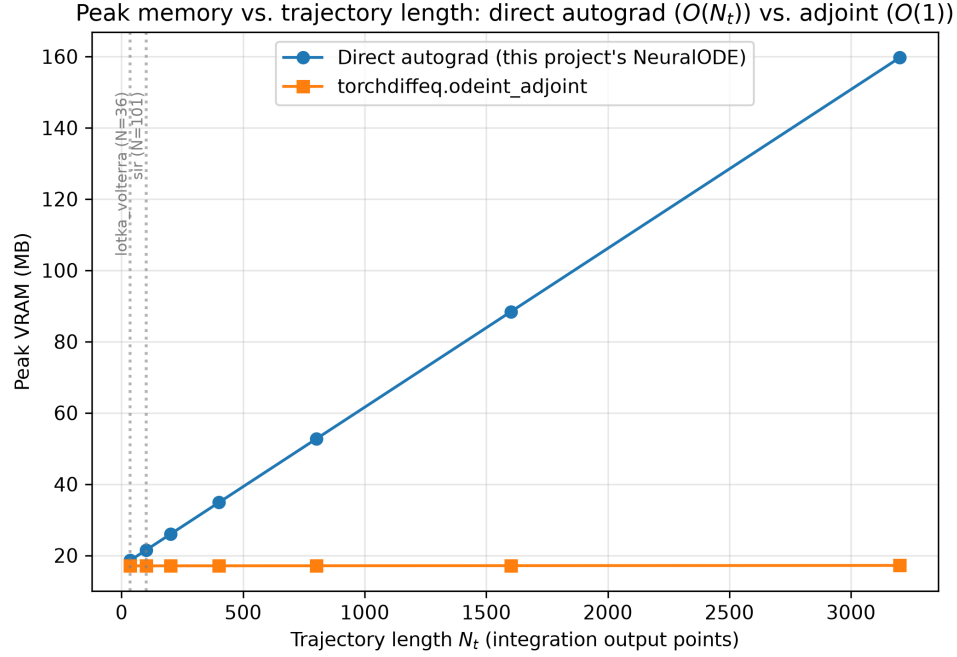
Figure 15. Direct autograd (top) keeps every intermediate state u_i in memory so the backward pass can reuse it, giving $O(N_t)$ memory. Adjoint sensitivity (bottom) instead solves the backward-time equation of Equation 29, recomputing $u(t)$ as needed rather than storing it, giving $O(1)$ memory regardless of N_t .

Question	Is the project documentation right that adjoint sensitivity’s constant memory cost brings no benefit at this project’s trajectory lengths?
Expected	Direct autograd should be cheaper in memory at these short lengths, with adjoint’s advantage only appearing once trajectories grow much longer.
Observed	Adjoint already uses less memory at every length tested. The deciding cost is wall-clock, not memory: adjoint is $1.9\text{--}2.1\times$ slower.

Finding. The measured memory data contradicts the stated premise behind the original choice, while still supporting its conclusion. The original premise implied a crossover length below which direct autograd’s smaller constant factor should still win on raw memory, since it avoids the adjoint method’s own bookkeeping overhead; the data shows no such crossover inside the tested range. Adjoint uses *less* memory than direct autograd at every tested length, including the shortest ($N_t = 36$: 17.08 vs. 18.61 MB) – Figure 16 shows direct

Table 16. Peak VRAM and wall-clock, direct autograd vs. adjoint sensitivity.

System	Trajectory length N_t	Method	Peak VRAM (MB)	Seconds per 1000 epochs
Lotka-Volterra	36	Direct autograd	18.61	317.5
Lotka-Volterra	36	Adjoint	17.08	672.9
SIR	101	Direct autograd	21.52	984.8
SIR	101	Adjoint	17.12	1871.6

**Figure 16.** Peak memory vs. trajectory length $N_t \in \{36, \dots, 3201\}$: direct autograd grows linearly (≈ 0.045 MB per output point); adjoint stays flat (0.7% increase over an $89\times$ length increase).

autograd’s per-step storage cost is already large enough at this project’s scale to outweigh whatever constant overhead the adjoint method’s own bookkeeping carries. Extrapolating the measured linear slope of direct autograd’s memory growth (≈ 0.045 MB per point, memory measured in decimal megabytes), adding 10 GB of activation memory, most of the card’s free capacity, would take $N_t \approx 224,000$ points; using the card’s full 12 GiB ($\approx 12,885$ MB) would take $\approx 289,000$. Either way, it is three orders of magnitude beyond anything this project integrates. The deciding factor for this project is wall-clock: solving the continuous adjoint ODE backward in time costs a consistent $1.9\text{--}2.1\times$ overhead at both tested lengths. Extrapolated to the standard 10,000 epochs on this GPU, a Lotka-Volterra run would take ~ 112 minutes with the adjoint against ~ 53 with direct autograd, for a memory saving that never becomes load-bearing (peak usage is 0.17% of card capacity). Gradient relative error is 2.85×10^{-7} and 4.88×10^{-7} respectively, well inside a 5×10^{-3} tolerance – both methods agree numerically, so the choice reduces to a resource trade-off. Direct autograd was faster at both timed lengths, and nothing in the memory data suggests the adjoint becomes worthwhile below this project’s longest trajectory ($N_t = 281$) (Table 16, Figure 16). The two gradient paths themselves are sketched in Figure 15.

8 Gradient-Norm Dynamics and Solver Order

The benchmark ablation found Forward Euler uniquely unable to reach 10^{-4} training loss within budget, with $28\times$ worse extrapolation MSE than Tsit5 (Table 5). The candidate mechanism tested here: a first-order solver commits an $O(h^2)$ local error every step, accumulating to an $O(h)$ global error, which could inject measurably noisier gradients and destabilize Adam’s moment estimates.

Question	Does a lower-order solver’s larger truncation error show up as measurably noisier gradients during training?
Expected	Gradient-norm noise should fall as solver order rises, correlating with the accuracy ranking already seen in the benchmark ablation.
Observed	No reliable correlation ($\rho = -0.088$, $p = 0.87$); the ranking is non-monotone and its sign flips depending on an arbitrary warm-up cut. Refuted.

The noise measure is the standard deviation of epoch-to-epoch *relative* gradient change, scale-invariant by construction,

$$\eta = \text{std}(\Delta \log_{10} g), \quad \Delta \log_{10} g_i = \log_{10} g_{i+1} - \log_{10} g_i, \quad (30)$$

compared against solver order p by Spearman rank correlation rather than the more familiar Pearson correlation. Pearson correlation measures how linearly two quantities move together, which presumes solver order is a quantity whose actual numeric spacing is meaningful – that the jump from order 1 to order 2 is comparable in size to the jump from order 4 to order 5. There is no reason to expect η to depend on solver order that linearly; the hypothesis under test is only that noise falls *monotonically* as order rises, which is exactly what Spearman correlation measures, by comparing ranks rather than raw values. It also handles the ties at $p = 2$ (Heun, Midpoint) and $p = 5$ (DOPRI5, Tsit5) natively, where Pearson would need an arbitrary tie-breaking rule.

Table 17. Gradient-noise measure η vs. solver order, six solvers, $n = 6$.

Solver	Order p	Noise η	Best training MSE	Noise rank
Forward Euler	1	0.2805	1.377×10^{-4}	1 (noisiest)
Classical RK4	4	0.2726	9.413×10^{-5}	2
Tsit5	5	0.2683	8.833×10^{-5}	3
DOPRI5	5	0.2660	9.283×10^{-5}	4
Midpoint RK2	2	0.2634	8.857×10^{-5}	5
Heun RK2	2	0.2621	8.315×10^{-5}	6

Spearman $\rho = -0.088$, $p = 0.87$ ($n = 6$). Three findings jointly refute the hypothesis: the effect size is a 7% spread across all six solvers; the ranking is non-monotone (RK4, order 4, is the second-noisiest solver); and the sign of ρ flips across warm-up cuts ($-0.088 \rightarrow -0.441 \rightarrow +0.706$ as the first 0, 2000, 5000 epochs are dropped); no cut reaches significance ($p \geq 0.12$), and the direction of any relationship depends on an arbitrary analyst choice.

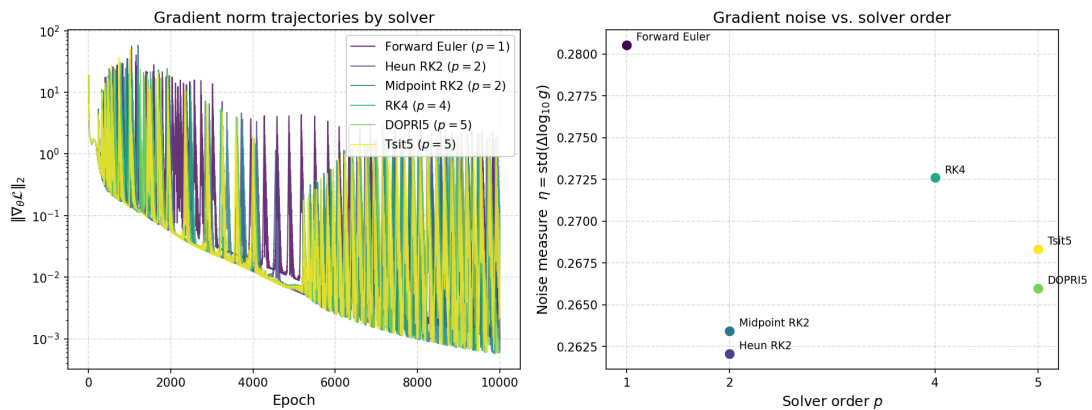


Figure 17. Left: gradient-norm traces, all six solvers: large sustained spike bursts (one to two orders of magnitude above the local median) dominate every trace, including Tsit5, independent of solver order. Right: the noise measure η (Equation 30) against solver order, showing the non-monotone ranking of Table 17.

Finding. Gradient-norm roughness does not explain Euler’s convergence floor (Table 17). All six traces are instead dominated (Figure 17) by shared, order-independent spike bursts (a spike epoch being one whose

gradient norm exceeds $8\times$ its median over epochs 6000–10,000; 1176 spike epochs for Euler and 866 for Tsit5 in the last 4000 epochs, with near-identical burst structure). Four checks rule out a numerical defect: zero non-finite gradients or losses across 60,000 logged epochs; every solver’s loss falls from 4.83 to below 1.4×10^{-4} , reaching its minimum in the last 12 epochs; the loss is lower five epochs after a spike in 62–65% of cases (64.0% pooled over all six solvers); and the median loss is elevated on spike epochs in every solver ($1.5\text{--}2.2\times$; for Tsit5, 3.90×10^{-4} against 1.80×10^{-4} on calm epochs), ruling out a spurious-gradient/still-loss signature. The most plausible reading is edge-of-stability optimizer behaviour. Gradient descent with a fixed step size has been observed to settle into a regime where it repeatedly overshoots the walls of a narrowing loss valley as training approaches a sharp minimum [4], and adaptive methods show a similar regime [5]. Adam at a constant learning rate (2×10^{-3} , no schedule), as used here, plausibly behaves the same way, producing this pattern of a large transient spike followed by fast recovery rather than either smooth convergence or a genuine divergence. Because this behavior appears with near-identical structure in Euler’s and Tsit5’s traces alike, it is a property of the optimizer and the loss surface’s local geometry, not of the integrator’s truncation error. Euler’s convergence floor is better explained by the earlier systematic-bias hypothesis (Section 6): a first-order integrator’s $O(h)$ global error biases *where* the optimizer settles, without making the path it takes to get there measurably rougher.

9 Learnable Basis-Function Blending

The benchmark ablation found B-spline (compact local support, $3.9\times$ RBF’s cost) and RBF (smooth global support, cheap) statistically tied on accuracy. Rather than choosing one basis function B_i in Equation 2 in advance, this study makes the choice itself a trained quantity: a learnable convex blend of the two candidate bases inside every edge,

$$\phi(x) = \alpha \cdot \text{Spline}(x) + \beta \cdot \text{RBF}(x), \quad (\alpha, \beta) = \text{softmax}(\text{logits}), \quad (31)$$

shown in Figure 18 and trained end-to-end from a neutral start ($\alpha = \beta = 0.5$) with no hand-set schedule, on Lotka-Volterra and the stability-fixed pendulum (Section 6). The blend weights are produced by a softmax over two learnable logits rather than, say, a single sigmoid-gated weight, because a softmax guarantees $\alpha + \beta = 1$ by construction – the two bases always form a genuine convex combination, so the gate can only trade one basis off against the other rather than scaling the overall edge output up or down as a side effect. A single pair of logits is shared across every edge in the network, so the whole model learns one global blend rather than letting each edge pick its own; this was a deliberate simplification to keep the number of new parameters at 2, not a claim that per-edge blending would not also work. If a single global blend systematically beats both pure bases, it would suggest the basis-function choice in Equation 2 is itself worth learning rather than fixing by hand, for every future KAN-ODE application.

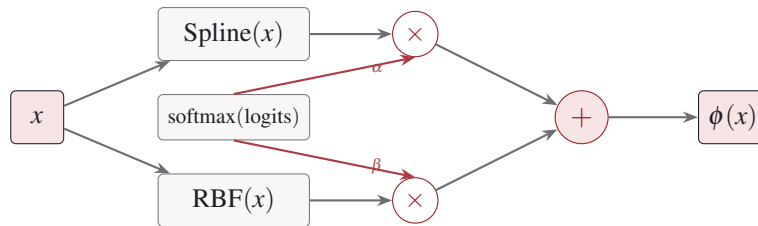
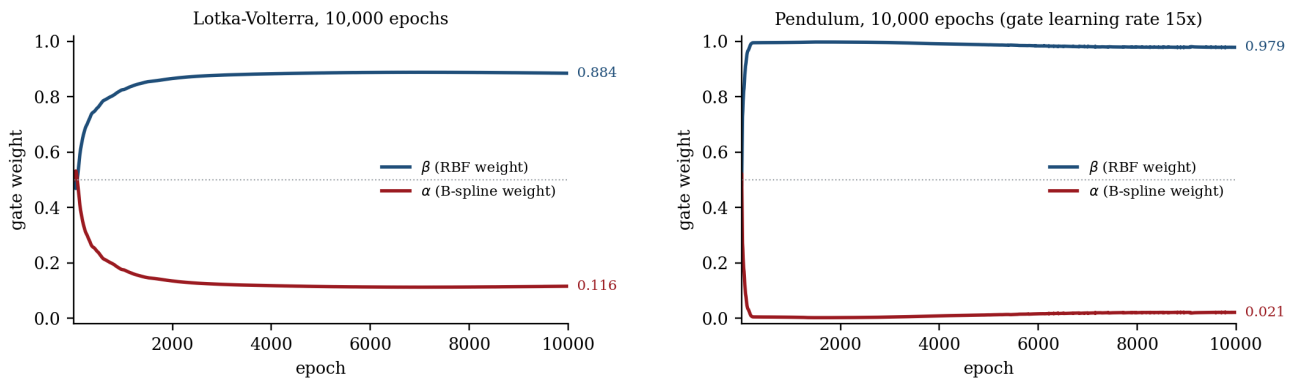


Figure 18. The hybrid-basis edge of Equation 31: every edge evaluates both bases in parallel, and a shared softmax gate produces the scalar weights α, β that multiply each basis’s output before they are summed into $\phi(x)$ – the choice of basis is a trained quantity, not a decision fixed at construction time.

Question	Can a network do better by learning its own blend of B-spline and RBF, rather than committing to one basis in advance?
Expected	A learned blend should match or beat the better pure basis, by adaptively favoring local support where useful and global support elsewhere.
Observed	Hybrid never beats the better pure basis on Lotka-Volterra, or pure RBF on the pendulum (no pure B-spline pendulum run exists). On the pendulum, it actively overfits past 3000 epochs in a way pure RBF does not.

Table 18. Hybrid basis vs. pure bases, full 10,000-epoch budget.

System / basis	Training MSE	Extrapolation MSE	R^2	Final gate
Lotka-Volterra – RBF (pure)	8.83×10^{-5}	8.92×10^{-5}	0.99997	–
Lotka-Volterra – B-spline (pure)	7.20×10^{-5}	8.38×10^{-5}	0.99997	–
Lotka-Volterra – Hybrid	1.47×10^{-4}	1.78×10^{-4}	0.99994	$\alpha=0.12, \beta=0.88$
Pendulum – RBF (pure)	9.28×10^{-5}	0.090	+0.647	–
Pendulum – Hybrid (10,000 epochs)	9.44×10^{-5}	0.809	−2.17	$\alpha=0.02, \beta=0.98$

**Figure 19.** Learned gate weights over training. Left: Lotka-Volterra settles at $\alpha = 0.116$, $\beta = 0.884$. Right: the pendulum gate reaches $\beta = 0.995$ by epoch 250 and peaks at 0.998 near epoch 1600, then drifts slowly back to 0.979, mostly after epoch 3000, when the model overfits (discussed below).

What the gate does and does not measure. Because the coefficients C are shared by both bases and free to grow, the gate is not identifiable on its own: a small α could in principle be offset by large coefficients, so the gate value alone does not say how much each basis contributes. Table 19 therefore measures the contributions directly. It splits each layer’s basis output $C(\alpha \mathcal{B}^{\text{spline}} + \beta \mathcal{B}^{\text{RBF}})$ into its two parts and takes their root-mean-square along the model’s own predicted trajectory. The measured shares track the gate closely, and are smaller still: the coefficients did not compensate for a small gate weight.

Table 19. Actual contribution of each basis in the trained hybrid models: root-mean-square of $\alpha C \mathcal{B}^{\text{spline}}$ and $\beta C \mathcal{B}^{\text{RBF}}$ over the predicted trajectory, per layer, with the residual path $W \text{silu}(x)$ for scale. Each model is evaluated at its selected checkpoint, so the gate can differ in the third decimal from the final-epoch value of Figure 19.

Model	Gate (α, β)	Layer	B-spline part	RBF part	Residual	B-spline share
Lotka-Volterra, 10k epochs	(0.115, 0.885)	1	0.076	1.232	1.910	5.8%
		2	0.165	2.125	2.686	7.2%
Pendulum, 3k epochs	(0.004, 0.996)	1	0.002	1.157	0.546	0.2%
		2	0.005	2.459	1.390	0.2%
Pendulum, 10k epochs	(0.021, 0.979)	1	0.011	1.018	0.792	1.1%
		2	0.025	2.339	1.364	1.1%

Lotka-Volterra: no benefit, real cost. The gate settles at a partial blend in which the B-spline supplies 6–7% of the basis output (Table 19). The hybrid is $1.7\text{--}2.1\times$ worse than the pure bases on both training MSE (1.47×10^{-4} against 8.83×10^{-5} and 7.20×10^{-5}) and extrapolation MSE (1.78×10^{-4} against 8.92×10^{-5} and 8.38×10^{-5} ; Table 18), while paying roughly the *sum* of their per-epoch compute. This run used the same learning rate for the gate as for every other parameter.

Pendulum: overfitting emerges only past a specific budget. A short probe run at 2000 epochs first surfaced a practical wrinkle: the gate moved in the correct direction (toward RBF) but too slowly, so the network still carried meaningful B-spline contamination at an epoch count where pure RBF had already reached a training MSE of 2.3×10^{-3} , against the probe’s 0.27. Giving the gate its own, faster learning rate ($15\times$ the base rate, direction untouched) lowered the probe’s training MSE to 1.1×10^{-2} at the same budget and was carried into the full pendulum runs (Figure 20, right). The faster gate rate was applied to the pendulum only; the Lotka-Volterra hybrid above used the base rate.

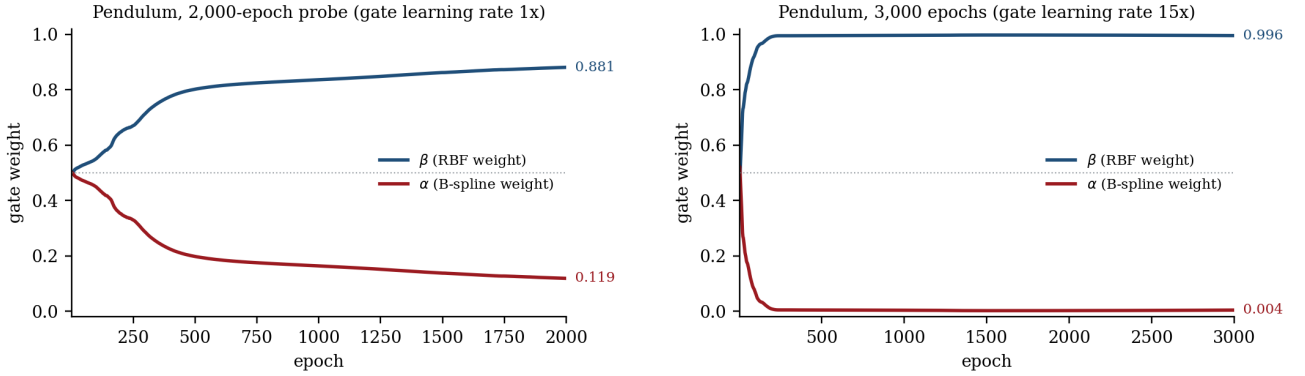


Figure 20. Left: the 2,000-epoch pendulum probe at the base gate learning rate, whose gate drifts toward RBF only slowly ($\beta = 0.881$ at the end), which motivated the faster gate rate. Right: the 3,000-epoch pendulum run at the faster gate rate discussed below, settling at $\beta = 0.996$.

At the full 10,000-epoch budget, hybrid matches pure RBF’s training fit almost exactly, but its extrapolation collapses ($R^2 = -2.17$): the monitor loss bottoms out near epoch 2950 (≈ 0.054) then rises to a ≈ 0.4 plateau for the remaining 7000 epochs while training loss keeps improving – textbook overfitting, visible directly in the saved training history. The gate settles well before this onset: β reaches 0.995 by epoch 250 (Figure 19), and pure RBF trained on the identical recipe shows no such pattern at all. After overfitting sets in, the gate drifts from 0.996 to 0.979, but the B-spline part still supplies only 1.1% of the basis output at the end (Table 19). So the overfitting cannot be blamed on a large hidden B-spline contribution. Its cause is not identified here: the two models differ in their early training path as well as in that small residual B-spline term.

One further number needs a caveat. Stopping at epoch 3000 gives the hybrid 0.055 full-horizon MSE against pure RBF’s 0.081 at the same epoch, with extrapolation $R^2 = +0.580$. That stopping point was chosen from the minimum of the monitored full-horizon loss, which includes the test window, so it is an *oracle* early stop that uses test data, which this report’s protocol otherwise forbids (Section 5). It shows that a better hybrid model exists along the training path, although even this oracle stop does not match pure RBF’s full-budget extrapolation ($R^2 = +0.647$, full-horizon MSE 0.045). It does not show that training could find it without looking at the test window, and it is not counted in the verdict below.

Verdict

Under this report’s protocol, where the checkpoint is selected on training loss, the learnable hybrid basis never outperforms the better pure basis on Lotka-Volterra, or pure RBF on the pendulum, where no pure B-spline run was made. On Lotka-Volterra it is $1.7\text{--}2.1\times$ worse than the pure bases at roughly the sum of their cost. On the pendulum it overfits badly after about 3000 epochs even though the B-spline supplies only about 1% of its basis output. The gate moves in a sensible direction on each system, but a learned blend buys no accuracy here.

10 Damping–Solver Stability and Loss-Landscape Analysis

Numerical stiffness – a system whose fastest and slowest timescales differ by orders of magnitude – is where explicit integrators are classically expected to struggle. This study sweeps the pendulum’s damping coefficient $\mu \in \{0.1, 0.5, 1.0, 2.0, 5.0, 8.0\}$ (Equation 4; the dimensionless damping ratio is $\zeta = \mu/(2\sqrt{g/L})$, from 0.016 to 1.28) against four explicit Runge-Kutta solvers (Euler, Midpoint, RK4, Tsit5), to map which (solver, μ) combinations train and which do not, and to test whether any failure is a stiffness effect. Every cell uses the

adopted pendulum recipe (Table 2: $[2, 10, 2]$, $G = 8$, 360 parameters, training window $[0, 5]$ of $[0, 10]$), data spacing $\Delta t = 0.05$ with two substeps, so the integration step is $h = 0.025$, and 2,000 epochs.

As the analysis below shows, this sweep never reaches a stiff regime: the largest ratio between the system’s two timescales is 4.3, at $\mu = 8$. It is therefore a study of how *damping* interacts with the solver and with training, not of stiffness in the numerical-analysis sense. A genuinely stiff test would need a system with a timescale ratio of 10^3 or more.

Question	Does stronger damping push some (solver, μ) combinations outside the solver’s stability region, so that lower-order explicit solvers fail?
Expected	If any cell leaves its solver’s stability region, that cell should diverge, and lower-order solvers should fail first.
Observed	Linear stability theory predicts no divergence (at most a mild anti-damping for Euler at $\mu = 0.1$), and none occurs. The failures at $\mu \in \{0.5, 1.0, 2.0\}$ depend on the random seed rather than the solver, apart from one Euler-specific failure at $\mu = 1.0$; $\mu = 0.1$ fails for every seed because its trajectory never settles within the training window.

10.1 The stability question resolves quickly

Linearizing the true system at its equilibrium gives eigenvalues $\lambda = (-\mu \pm \sqrt{\mu^2 - 4g/L})/2$. For every under-damped $\mu < 2\sqrt{g/L} \approx 6.26$ the eigenvalues are a complex pair, $\lambda = -\mu/2 \pm i\sqrt{g/L - \mu^2/4}$, with constant modulus $|\lambda| = \sqrt{g/L} \approx 3.13$, so $|h\lambda| \approx 0.078$ at $h = 0.025$. Because they are complex, the relevant test is the full stability region of Equation 14, not its negative-real-axis interval [8]. Midpoint, RK4 and Tsit5 are stable at every μ tested. Euler, whose stability region is the disk $|1 + z| \leq 1$, satisfies $|1 + h\lambda|^2 = 1 - h\mu + h^2g/L$ and is therefore stable only if $\mu \geq hg/L \approx 0.25$. At $\mu = 0.1$ its amplification factor is $|1 + h\lambda| \approx 1.0018$ per step: a mild numerical anti-damping, about a $2\times$ growth over $[0, 10]$, not a divergence. The overdamped case $\mu = 8$ has real eigenvalues -1.51 and -6.49 , a timescale ratio of only 4.3, and $|h\lambda_{\text{fast}}| \approx 0.162$, comfortably stable for every solver.

What the integrator actually steps, however, is the *learned* field, not the true one. Its Jacobian was evaluated along the true trajectory for all 24 seed-42 models, every solver at every μ . The largest eigenvalue magnitude per model is 3.5–11.8, up to about four times the true system’s, which is consistent with the Lipschitz estimates of Section 6. Even so, $|h\lambda| \leq 0.30$ for every model. That is small enough that each solver’s amplification factor $R(h\lambda)$ closely tracks the exact $e^{h\lambda}$: where the learned Jacobian has eigenvalues with positive real part, the resulting local growth belongs to the learned field itself, not to the integrator. Consistent with this, across all 24 cells not one run diverged or produced a non-finite gradient at any point in training. All four solvers are explicit Runge-Kutta methods with a bounded stability region, and no implicit solver is implemented in this project, so this analysis cannot say how an implicit method would behave on a genuinely stiff system.

10.2 What actually drives the failures: initialization

Each cell is classified from its training record alone. A run is *diverged* if it was aborted by the non-finite guard (Algorithm 3) or never produced a valid checkpoint; it is *converged* if it had no non-finite steps, its best training MSE is below 10^{-2} , and its final training MSE is no more than $10\times$ its best; every other run is *not converged*. No run in this study diverged, so every failure below is a run that stayed finite but never fit, or fit and then regressed. The 10^{-2} threshold is coarse: $\mu = 0.5$ cells at $4.6\text{--}9.2 \times 10^{-3}$ count as converged while $\mu = 0.1$ cells at $1.7\text{--}3.0 \times 10^{-2}$ do not, so verdicts near the threshold should be read together with the MSE values in Table 20.

Two anomalies stand out in Table 20 and Figure 21: $\mu = 2.0$ fails identically across all four solvers to two significant figures, and $\mu = 1.0$ fails only for Euler. Retraining Euler at $\mu = 2.0$ with two more seeds isolates the cause (seed 42: not converged, 1.82×10^{-2} ; seed 1: not converged, 1.68×10^{-2} ; seed 7: **converged**, 7.18×10^{-5}) – the trap depends on the random initialization. Extending this to all four solvers $\times$ six $\mu \times$ three seeds (72 cells, checkpoints saved throughout) confirms the pattern generally: only the two extremes ($\mu = 0.1$, uniformly hard; $\mu \in \{5.0, 8.0\}$, uniformly easy) hold their verdict across every seed. Every transitional $\mu \in \{0.5, 1.0, 2.0\}$ has at least one of three seeds land in a bad local minimum, and at $\mu = 1.0$ two *different* seeds fail for two different reasons: seed 1 produces a universally bad initialization shared by all four solvers (MSE $\approx 9.7\text{--}9.8 \times 10^{-2}$, the same solver-independent pattern as the $\mu = 2.0$ shared trap), while seed 42 produces a separate, Euler-specific failure (the other three solvers converge under seed 42). Euler’s lower three-seed

Table 20. Original 24-cell sweep (single seed 42): best training MSE and verdict. “Not conv.” is not converged under the rule above; no cell diverged.

Damping coeffi- cient μ	Euler	Midpoint	RK4	Tsit5
0.1	not conv., 1.7×10^{-2}	not conv., 3.0×10^{-2}	not conv., 2.2×10^{-2}	not conv., 2.4×10^{-2}
0.5	converged, 4.6×10^{-3}	converged, 7.9×10^{-3}	converged, 4.7×10^{-3}	converged, 9.2×10^{-3}
1.0	not conv., 1.0×10^{-1}	converged, 5.4×10^{-4}	converged, 5.3×10^{-4}	converged, 4.0×10^{-4}
2.0	not conv., 1.8×10^{-2}	not conv., 1.8×10^{-2}	not conv., 1.8×10^{-2}	not conv., 1.8×10^{-2}
5.0	converged, 6.1×10^{-5}	converged, 7.0×10^{-5}	converged, 6.9×10^{-5}	converged, 6.8×10^{-5}
8.0	converged, 2.1×10^{-5}	converged, 2.3×10^{-5}	converged, 2.6×10^{-5}	converged, 2.6×10^{-5}

converged fraction at $\mu = 1.0$ (0.33 against 0.67 for the other solvers) is therefore the sum of two separate effects, one shared by all solvers (seed 1) and one solver-specific (seed 42); the single-seed table shows only the second.

Consequence for the original heatmap

The 24-cell sweep used one shared seed for all four solvers. All four solvers failing identically at $\mu = 2.0$ reflects four solvers inheriting the same coin flip from a shared initialization. At this step size, the seed $\times \mu$ interaction determines the outcome far more than the solver choice does.

10.3 The mechanism: a genuine spurious equilibrium

For the trapped $\mu = 2.0$ models, the learned field is far from zero at the true equilibrium: $|f_\theta(0,0)| \in [2.65, 2.88]$ across all four solvers’ own trained models. A coarse grid search first located a nearby point where $|f_\theta| = 0.0086$. To confirm that this is a genuine equilibrium, Newton’s method was run on $f_\theta(\mathbf{u}) = \mathbf{0}$ from that point, for each solver’s model (Table 21). All four converge to a true zero of the learned field, to a residual of $\sim 10^{-6}$, at almost the same location, about 0.05 from the true equilibrium. At that point the Jacobian’s eigenvalues are a complex pair with negative real part, so the point is a *stable spiral*: trajectories near it wind inward and stop there. Four networks trained from the same initial weights but through four different integrators learned essentially the same wrong but genuine, attracting fixed point (Figure 22).

Table 21. Equilibria of the learned field for the trapped $\mu = 2.0$ models (seed 42), found by Newton’s method on $f_\theta(\mathbf{u}) = \mathbf{0}$. The true equilibrium is $(0,0)$, where the true Jacobian has eigenvalues $-1 \pm 2.97i$.

Solver	$ f_\theta(0,0) $	Learned equilibrium (θ, ω)	Residual $ f_\theta $	Jacobian eigenvalues there
Euler	2.77	$(-0.049, +0.016)$	1.5×10^{-6}	$-0.62 \pm 3.35i$
Midpoint	2.88	$(-0.049, +0.014)$	1.2×10^{-6}	$-0.45 \pm 3.94i$
RK4	2.65	$(-0.047, +0.014)$	6.7×10^{-7}	$-0.48 \pm 3.96i$
Tsit5	2.86	$(-0.050, +0.016)$	1.5×10^{-6}	$-0.47 \pm 3.93i$

The full predicted trajectories tell the same story over time rather than in a single snapshot. At $\mu = 2.0$ every solver’s predicted angle and angular velocity settle onto the same wrong trajectory, which ends about 0.05 from the true equilibrium; the offset is too small to see on the trajectory panels and shows in the error panel. At $\mu = 1.0$ only Euler’s prediction (angle and angular velocity) visibly departs from the true trajectory. Figures 34 and 35 in Appendix B show both cases panel by panel.

10.4 Weight-space loss landscape: an independent measurement

The phase-space plots above describe the learned dynamics; a separate question is how forgiving the *training loss* is around each solution – whether a small nudge to the weights leaves the fit essentially unchanged or makes it collapse. This was measured with the random-direction technique of Li et al. [13], adapted to KAN-ODE’s 360-parameter weight space. Two random directions $\mathbf{d}_1, \mathbf{d}_2$ are drawn and each is rescaled parameter tensor by parameter tensor to the norm of the corresponding trained tensor; this is a per-tensor version of Li et

Damping-solver verdict map, seed 42 (dt = 0.05, two substeps)

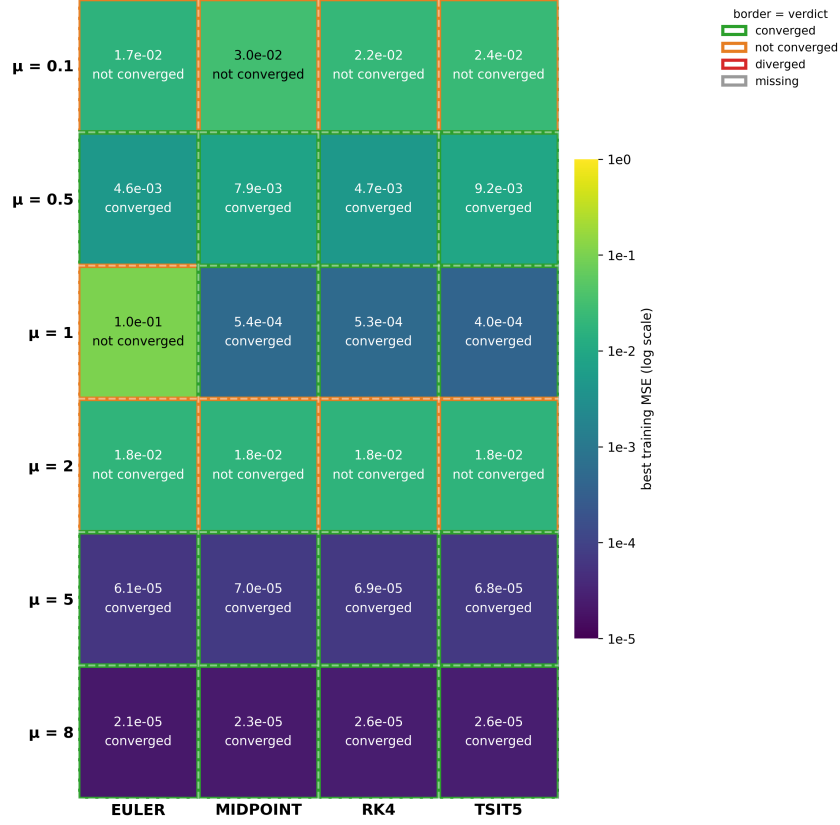


Figure 21. Verdict map of the 24-cell single-seed sweep ($\Delta t = 0.05$, two substeps). Each cell shows its best training MSE and verdict under the rule above; the border color repeats the verdict.

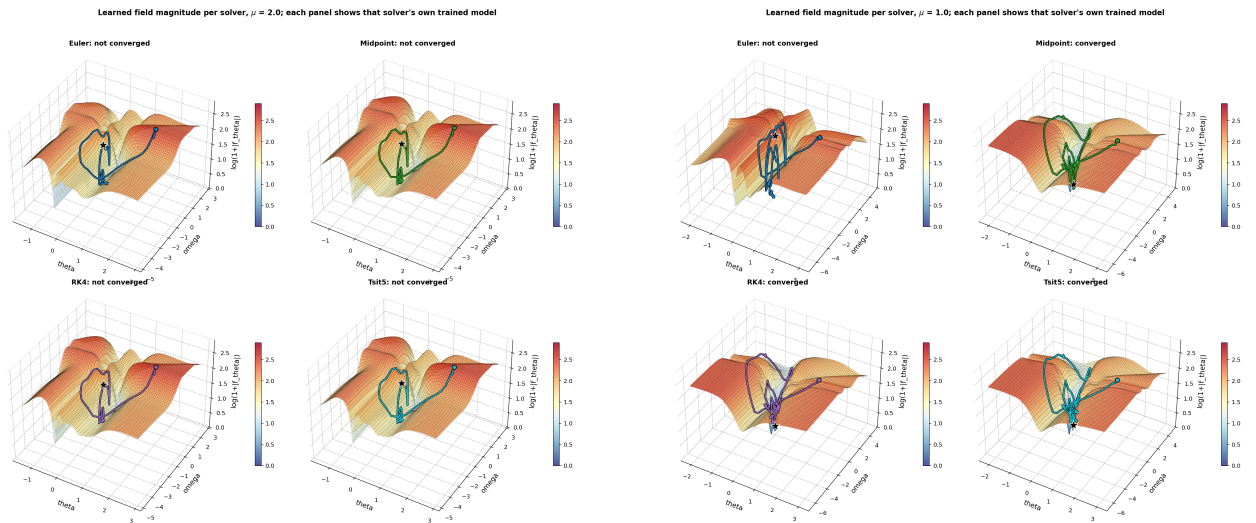


Figure 22. Learned vector-field landscape $\log(1 + |\mathbf{f}_\theta(\theta, \omega)|)$, per solver's own trained model. Left ($\mu = 2.0$): every solver's true equilibrium (star) sits partway up a slope, while a separate, lower point nearby is where each trajectory actually settles. Right ($\mu = 1.0$): Euler's star sits high on a ridge, well apart from where its trajectory spirals down and rests; for the other three solvers, star and resting point coincide at the valley floor.

al.’s filter normalization, since a KAN layer has no filters. The training MSE $\mathcal{L}(\theta^* + a\mathbf{d}_1 + b\mathbf{d}_2)$ is evaluated on a 21×21 grid of $(a, b) \in [-1, 1]^2$, and each cell is summarized by its *rise*,

$$\text{rise} = \log_{10} \max_{(a,b)} \mathcal{L}(\theta^* + a\mathbf{d}_1 + b\mathbf{d}_2) - \log_{10} \mathcal{L}(\theta^*), \quad (32)$$

the number of decades by which the loss climbs from the trained point to the highest point of the grid.

Table 22. Loss-landscape rise (Equation 32) for the seed-42 models, one random direction-pair ($\mu = 0.1$ and 0.5 omitted). Bold cells are trapped.

μ	euler	midpoint	rk4	tsit5
1.0	5.53	9.03	8.99	9.12
2.0	4.23	4.24	4.22	4.20
5.0	8.75	8.75	8.76	8.76
8.0	9.14	9.15	9.12	9.12

Read at face value, Table 22 says converged models (rise ≈ 9) sit in steep, narrow basins, while trapped models (bold, rise ≈ 4 – 5.5) sit on much flatter ground. That reading is confounded. Rise is measured from the trained point, and trapped models start from a training loss roughly 30 to 5000 times higher than converged ones (Table 20), so they have less room to rise even if their surroundings are just as steep. Adding the trained-point loss back gives the absolute height of the grid’s highest point, $\log_{10} \max \mathcal{L} = \text{rise} + \log_{10} \mathcal{L}(\theta^*)$ (Table 23; the trained-point loss is taken as the recorded best training MSE, since the grids themselves were not kept).

Table 23. Absolute height of the loss grid, $\log_{10} \max \mathcal{L}$, recovered from Table 22 and the trained-point losses of Table 20 ($\mu = 0.1$ and 0.5 omitted). Bold cells are trapped.

μ	Euler	Midpoint	RK4	Tsit5
1.0	4.5	5.8	5.7	5.7
2.0	2.5	2.5	2.5	2.5
5.0	4.5	4.6	4.6	4.6
8.0	4.5	4.5	4.5	4.5

Two different pictures emerge. For Euler at $\mu = 1.0$, most of the low rise is the starting-point effect: its rise is 3.5 decades below the converged solvers at the same μ , but its grid maximum is only 1.2 decades lower, and it is as high as those of the converged cells at $\mu = 5$ and 8 . For the $\mu = 2.0$ trap, the walls really are lower: every trapped model’s grid maximum ($10^{2.5}$) is two to three decades below every converged cell in Table 23. Comparisons across different μ remain loose, because each μ is a different dataset with its own loss scale. The measurement is also coarse: it reports only the highest point of a $[-1, 1]^2$ grid, not the curvature at the minimum, which a Hessian-eigenvalue measurement would give; that was not computed.

Figure 23 renders the $\mu = 2.0$ case as a surface: the terrain around Euler’s trapped solution is broad and gently rolling, in visual contrast to Figure 25’s converged $\mu = 1.0$ solution, which sits in a narrow, sharply walled valley. Figure 24 shows Euler’s trapped $\mu = 1.0$ solution, whose basin looks similarly shallow near the trained point even though, as Table 23 shows, its far walls are as high as a converged cell’s.

Robustness check across three direction-pairs. Repeating the measurement with two more independent random direction pairs per cell (72 measurements total) found one direction-pair giving the highest rise in all 24 cells. Because every model shares the same architecture, one particular raw random draw runs toward a steeper wall in every cell at once, inflating that direction’s rise by roughly 1.6 – $3\times$ relative to the other two (about 2.5 – $3\times$ for the trapped cells, under $2\times$ at $\mu \in \{5, 8\}$). The size of the measured effect depends on how many direction choices are averaged: the $\approx 2\times$ gap between trapped and converged cells seen with a single direction-pair narrows to roughly 25 – 35% once all three direction-pairs are averaged, since the outlier direction inflates the trapped cells’ mean more than the converged cells’ mean. The three-direction means keep the same ordering, but their ranges across direction-pairs overlap heavily (Figure 26), so with three directions per cell the gap is suggestive rather than established.

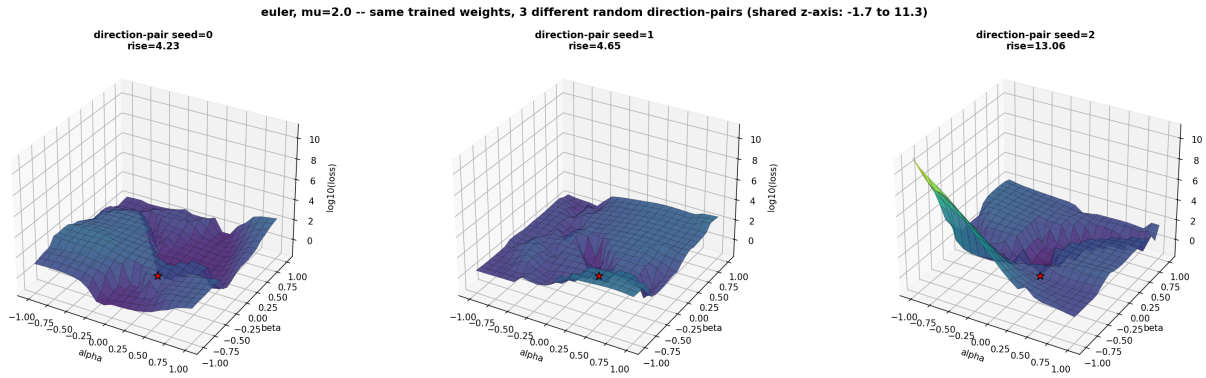


Figure 23. Euler, $\mu = 2.0$ (trapped): the training-loss surface around the trained weights, viewed under three independent random direction-pairs. The basin near the trained point (the marked star) is broad and shallow in all three views; the third panel's steep spike on the far side is the outlier direction discussed below, not the basin itself.

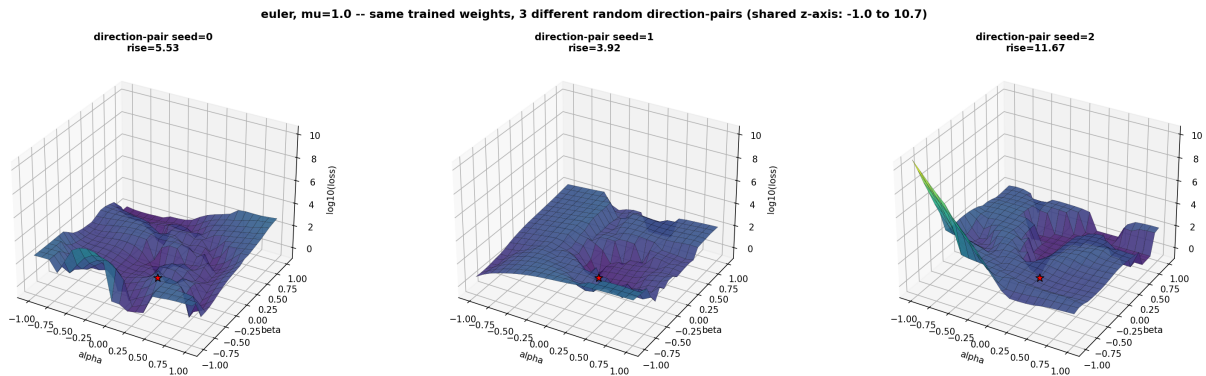


Figure 24. Euler, $\mu = 1.0$ (also trapped): the same broad, shallow basin shape recurs at a different damping value under the same solver.

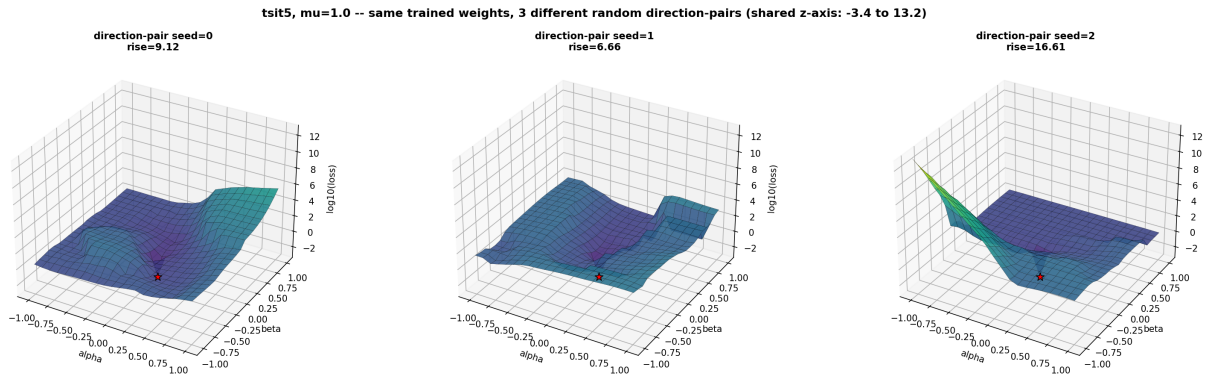


Figure 25. Tsit5, $\mu = 1.0$ (converged): the basin around the trained point is visibly narrower, in every one of the three direction-pairs, than either trapped case above. Its floor sits lower mainly because its trained-point loss is lower; note that the three figures use different vertical ranges.

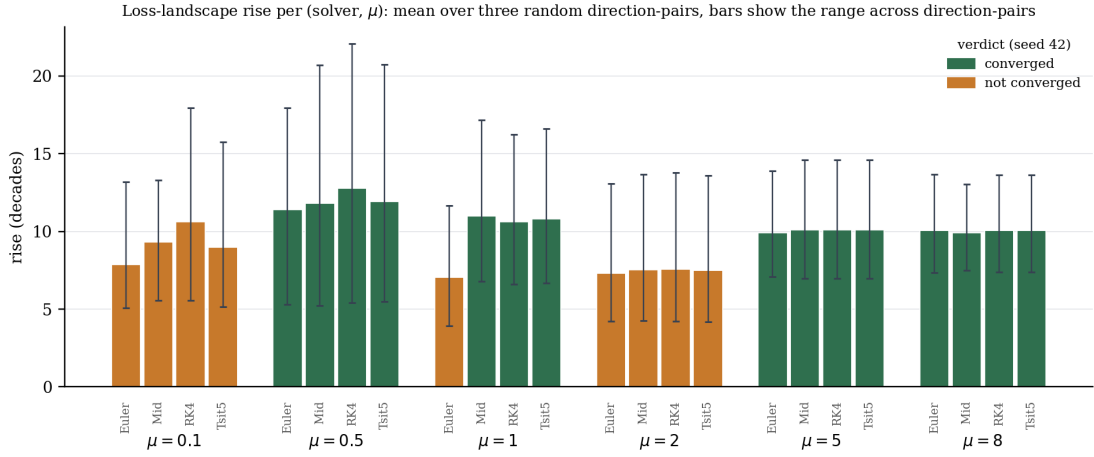


Figure 26. Mean rise per (solver, μ) over three random direction-pairs, with the range across direction-pairs as error bars, colored by the seed-42 verdict.

The trapped cells – $\mu = 2.0$ across all solvers (a confirmed spurious equilibrium) and Euler’s $\mu = 1.0$ (whose trap was not root-found) – sit roughly 25–35% lower on this chart than converged cells (mean rise 7.0–7.6 against 9.9–12.8; Euler at $\mu = 1.0$ is 35% below the three converged solvers at the same μ), a modest gap within overlapping ranges rather than an order-of-magnitude separation, and by Table 23 partly a starting-point effect. $\mu = 0.1$ is not converged for an unrelated reason: its trajectory never settles within the training window at all, and its mean rises (7.9–10.6) span from just above the trapped cells to just above the converged $\mu \in \{5.0, 8.0\}$ cells. Whatever flatness there is belongs to the trapped cells, not to every cell the classifier marks as not converged. Figure 27 summarizes all 72 seeded cells.

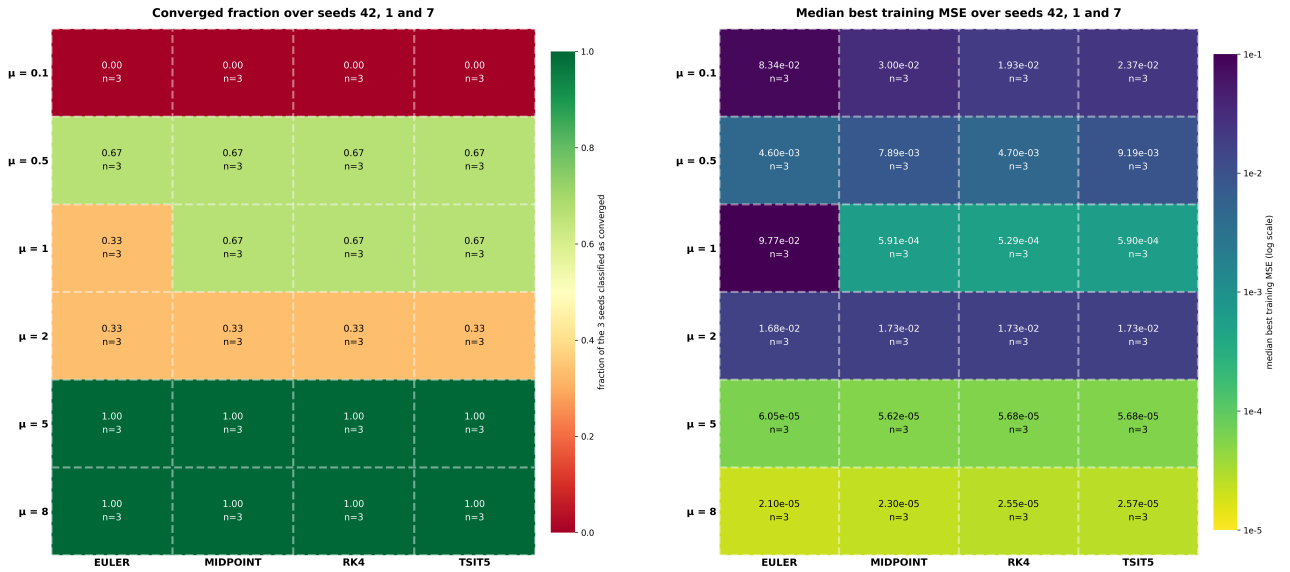


Figure 27. Converged fraction (left) and median training MSE (right) over 3 seeds, all 4 solvers $\times$ 6 μ (72 cells). The two broadly agree: the cells with low converged fraction are the cells with elevated median error.

Verdict

Linear stability theory predicted no divergence (at most a mild anti-damping for Euler at $\mu = 0.1$), and none occurred. The sweep never reached a stiff regime, so it cannot test stiffness itself. At $\mu \in \{0.5, 1.0, 2.0\}$ what decides the outcome is, apart from one Euler-specific failure, a seed-dependent local minimum of training: at $\mu = 2.0$ it produces a genuine, attracting spurious equilibrium of the learned field, confirmed by Newton’s method and by its Jacobian, and shared by all four solvers trained from the same initialization. The loss-landscape measurement is consistent with this at $\mu = 2.0$, but it is coarse and partly confounded by the trained-point loss, so it is supporting evidence rather than an independent proof.

11 Sparse Regression Comparison and Non-Mechanistic Epidemic Validation

This section compares two fundamentally different ways of discovering governing equations from data – sparse symbolic regression and gradient-based neural fitting – and separately tests whether the cross-domain stability fixes of Section 6 generalize beyond the mechanistic system that produced them.

11.1 Noise robustness: KAN-ODE vs. sparse symbolic regression

Sparse Identification of Nonlinear Dynamics (SINDy) [1] fits a governing equation as a sparse linear combination of candidate terms drawn from a fixed library – here, every polynomial in x and y up to degree 2, i.e. $\{1, x, y, x^2, xy, y^2\}$. Coefficients are estimated by sequentially thresholded least squares: a least-squares fit (with a small ridge penalty, $\alpha = 0.05$) is performed against numerically estimated derivatives, every coefficient below a fixed threshold is zeroed out, and the fit is repeated on the surviving terms until the active set stops changing (at most 20 iterations), followed by an unpenalized refit on the final active set. This is what makes the recovered model sparse and, in principle, human-readable – unlike the KAN’s 240 free parameters, SINDy returns a short list of surviving terms with numeric coefficients attached, directly comparable to the true equation’s own coefficients. The implementation is `pysindy`’s [6, 10] sequentially thresholded least squares with threshold 0.05, held fixed across the entire noise sweep so that the comparison measures noise sensitivity rather than per-level tuning. The derivatives it regresses against are estimated by plain finite differences from the observations. It is fit on the same 36 training observations the KAN-ODE noise-sweep models of Table 10 were trained on ($t \in [0, 3.5]$, $\Delta t = 0.1$, noise $\sigma \in \{0, 0.01, 0.05, 0.1\}$ added to the training points; Section 2). The recovered equation is then integrated over $[0, 14]$ and scored against the clean ground truth, exactly as the KAN-ODE is.

The two methods are not fitted the same way. KAN-ODE is trained through the integrator on trajectory error, while SINDy fits pointwise derivative estimates, so SINDy’s accuracy is limited by how well 36 points at spacing 0.1 determine a derivative. That limit turns out to dominate the comparison, as the check at the end of this subsection shows.

Question	How does a sparse, symbolic method compare to a dense, 240-parameter neural method as observation noise increases?
Expected	Sparse regression’s tiny library should make it more robust to noise; KAN-ODE’s larger parameter count should make it more accurate.
Observed	With finite-difference derivatives, both hold: KAN-ODE is more accurate at every noise level, but its advantage collapses from $1110\times$ to $6.5\times$. Given exact derivatives, SINDy recovers the true equation outright, so the accuracy ranking reflects the derivative estimator, not the two model classes.

Table 24. Extrapolation MSE on $(3.5, 14]$ vs. observation noise σ (sparse regression with finite-difference derivatives, threshold 0.05).

Noise level σ	Sparse regression	KAN-ODE	KAN-ODE advantage
0.00	9.90×10^{-2}	8.92×10^{-5}	$1110\times$
0.01	1.58×10^{-1}	1.22×10^{-3}	$130\times$
0.05	1.48×10^{-1}	2.54×10^{-2}	$5.8\times$
0.10	8.89×10^{-1}	1.37×10^{-1}	$6.5\times$

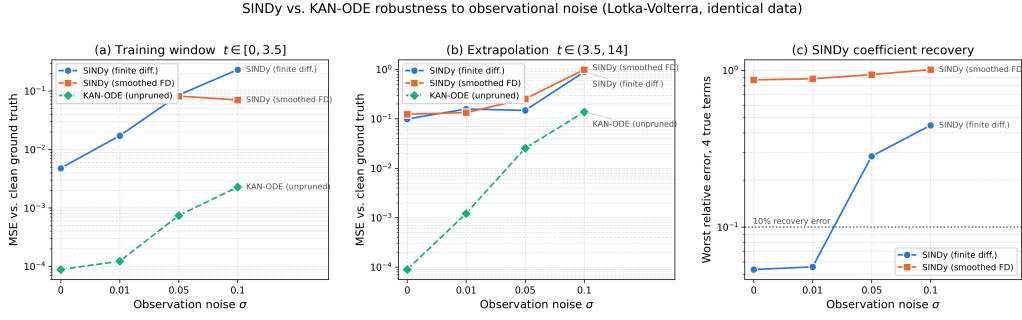


Figure 28. Sparse regression (plain and smoothed finite-difference derivatives) and KAN-ODE against noise level: (a) training-window MSE, (b) extrapolation MSE, (c) worst relative error among the four true coefficients recovered by sparse regression.

In Table 24 and Figure 28, KAN-ODE wins on absolute accuracy at every level, but the margin collapses $170\times$ across the sweep (sparse regression degrades $9.0\times$ from $\sigma = 0$ to 0.1 ; KAN-ODE degrades $1540\times$). The two ratios start from very different floors, though: sparse regression’s noise-free error is already set by its finite-difference derivatives (as the check below shows), while KAN-ODE starts three orders of magnitude lower, so the smaller degradation factor is not by itself evidence that the method resists noise better.

How much of SINDy’s error is the derivative estimate? SINDy’s noise-free error of 9.9×10^{-2} is surprisingly large for a library that contains the exact Lotka-Volterra terms. To separate the method from its derivative estimator, the noise-free fit was repeated with the exact derivatives, computed from Equation 3 at the same 36 points, in place of the finite differences (Table 25).

Table 25. Noise-free sparse regression on the 36 training points with estimated and with exact derivatives (threshold 0.05). The finite-difference row reproduces Table 24.

Derivatives	Recovered equation	Extrapolation MSE
Finite differences	$\dot{x} = 1.470x - 0.980xy; \quad \dot{y} = -0.175 + 0.060x - 2.840y + 0.946xy$	9.90×10^{-2}
Exact	$\dot{x} = 1.500x - 1.000xy; \quad \dot{y} = -3.000y + 1.000xy$	8.8×10^{-13}
For comparison: KAN-ODE, noise-free, 10,000 epochs		8.92×10^{-5}

With exact derivatives SINDy recovers the true equation to three decimals and extrapolates eight orders of magnitude better than KAN-ODE. The finite differences carry only a 3.5% root-mean-square error in the derivatives, but that is enough for the fit to admit two spurious terms, a constant and a linear x term, which then accumulate over the extrapolation window. On noise-free data, therefore, the ranking in Table 24 is a statement about finite-difference derivatives at this sampling density, not about sparse regression against KAN-ODE. A fairer comparison would fit SINDy without pointwise derivatives, for example with a weak (integral) formulation or by fitting its coefficients to trajectory error as the KAN-ODE is. Neither was run, and whether SINDy would keep its advantage under noise is untested. Table 26 lists the coefficient each fit recovers for the four true terms. The bilinear cross-term (xy) is recovered to within 5.7% at every noise level. What degrades with noise is the linear coefficient α of x , from 2.0% to 44.6% error, together with the number of spurious terms admitted by the fixed threshold, which rises from 2 to 7–8. SINDy’s breakdown at $\sigma \geq 0.05$ is therefore a failure to pin the prey growth rate α , together with the spurious terms the threshold admits, not a failure to find the interaction structure.

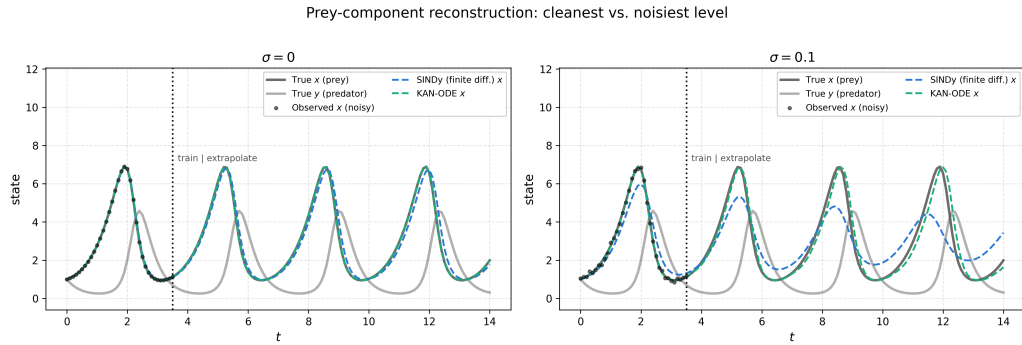
Smoothed derivatives do not help here. `pysindy`’s smoothed finite differences, a common defence against noisy derivatives, were also tried. They recover the coefficients worse than plain finite differences at every noise level: the worst term’s error is 87% at $\sigma = 0$ and 101% at $\sigma = 0.1$, against 5.4% and 44.6%, and the fit keeps 8 spurious terms at every level. In extrapolation they are worse at three of the four noise levels (only at $\sigma = 0.01$ are they slightly better, 0.133 against 0.158). The likely cause is the filter window rather than the method (a shorter window was not part of the reported runs): the default Savitzky-Golay window of 11 points spans 1.0

Table 26. Relative error of each true coefficient recovered by sparse regression with finite-difference derivatives, and the number of spurious library terms kept (threshold 0.05, 36 training points).

Noise σ	x in $\dot{x}$ (α)	xy in $\dot{x}$ (β)	y in $\dot{y}$ (γ)	xy in $\dot{y}$ (δ)	Spurious terms
0.00	2.0%	2.0%	5.3%	5.4%	2
0.01	5.6%	5.3%	4.9%	4.9%	4
0.05	28.3%	5.2%	11.9%	3.7%	8
0.10	44.6%	5.7%	2.9%	3.1%	7

time unit at $\Delta t = 0.1$, nearly a third of the 3.32-unit cycle and 11 of the 36 training points, so the smoothing flattens the peaks the derivative information lives in.

The cross-term result matters for the comparison because xy is exactly the term a single KAN layer cannot represent directly, since each edge is a function of only one input variable and a layer sums its edges additively. This asymmetry – sparse regression writes xy as a literal library column, while a KAN can only approximate it through composition across a hidden layer – means a term-by-term symbolic comparison would be comparing a coefficient against a distributed representation, so the comparison used here is trajectory reconstruction instead (Figure 29).

**Figure 29.** Reconstructed prey trajectories $x(t)$ at $\sigma = 0$ and $\sigma = 0.1$ (sparse regression with finite-difference derivatives): sparse regression loses amplitude under noise; KAN-ODE keeps amplitude but drifts in phase.

11.2 Edge pruning: testing whether a term-by-term comparison is even possible

Part of the original motivation for a smooth, globally-supported basis such as RBF is that a sparse, pruned KAN edge structure could in principle be read back as a closed-form symbolic law, the same interpretability goal sparse regression achieves by construction. The base paper reaches this by adding an L_1 penalty on all trainable parameters to the training loss,

$$\mathcal{L} = \text{MSE}(\mathbf{u}_{\text{true}}, \mathbf{u}_{\text{pred}}) + \lambda \sum_i |c_i|, \quad (33)$$

where c_i ranges over every trainable parameter (the spline coefficients of Equation 2 and the residual weights) and λ (the paper’s γ_{sp}) trades off trajectory accuracy against how many edges the penalty drives toward zero. Every checkpoint trained for this project used $\lambda = 0$: the plain mean-squared-error term alone, with no sparsity pressure ever added to Equation 33. A KAN-ODE analogue of a sparse coefficient list would instead come from pruning weak edges *post hoc* and reading off what survives. Edges are ranked within each layer by the mean absolute value of their spline coefficients, and the spline branch of the weakest is zeroed while the residual $W \text{silu}(x)$ term is kept. Applied to the existing (never retrained) noise-sweep checkpoints, zeroing the spline branches of the two weakest of forty edges (one per layer) raises the noise-free full-horizon MSE from 8.90×10^{-5} to 3.43 – a factor of 3.9×10^4 , worse than predicting the constant dataset mean (2.91; Figure 30, left). Removing the same two edges entirely, residual included, gives 2.79 (right). Every checkpoint was trained with no sparsity-inducing regularization applied at all, so nothing during training ever asked the network to concentrate its fit onto a few edges; the flat plateau at ≈ 3.5 – 3.8 once half or more of the spline branches are pruned is the signature of a model where the fit is spread across many edges: no small subset of edges

carries it, so there is no sparse structure for a magnitude-based pruning rule to recover. This says nothing about whether the pruning procedure would find such structure in a network trained to have it; that would need the L_1 -regularized retraining of Equation 33, which was not run.

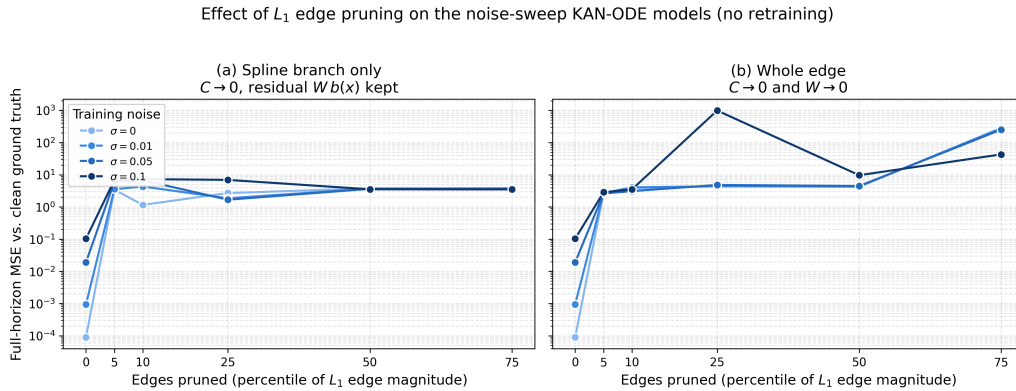


Figure 30. Full-horizon MSE vs. edge-pruning fraction for the four noise-sweep KAN-ODE models, pruned without retraining. Edges are ranked within each layer by the L_1 magnitude of their spline coefficients; no L_1 penalty was used in training. Left: only the spline branch is zeroed, the residual term is kept. Right: the whole edge is removed.

11.3 Testing whether SIR’s fixes transfer to a non-mechanistic epidemic curve

A synthetic, asymmetric outbreak curve generated by no compartmental ODE (Equation 6, Table 1) tests whether the SIR fixes of Section 6 generalize by mechanism rather than by coincidence of the SIR system’s own structure. The same init-time diagnostic used to root-cause SIR reproduces an almost identical speed-mismatch signature ($18.77\times$ against SIR’s roughly $18\text{--}19\times$), which says that time rescaling is needed. It does not by itself set the factor. The factor comes from a horizon rule: choose s so that the rescaled horizon is $T/s = 5$, the regime in which Lotka-Volterra ($T = 3.5$) and the fixed pendulum ($T = 5$) train. For SIR this rule gives the $s = 10$ already used ($50/10 = 5$). For the epidemic curve it was applied to the full 120-day record, giving $s = 120/5 = 24$; applied to the 45-day training window, as for SIR, it would give $s = 9$. The time-scale sweep (Figure 31) was run in the same batch as the $s = 24$ arms, so the project’s records cannot show that 24 was fixed before the sweep’s results were seen. The sweep’s training loss does not single out 24 (it is highest there, 2.74×10^{-3} against $1.28\text{--}2.00 \times 10^{-3}$ for $s \in \{10, 20, 30, 40\}$), but its extrapolation MSE is lowest there (6.99 against $10.3\text{--}14.3$), so the records cannot show that the choice was independent of the test results. Every arm below is compared at the same s .

Question	SIR’s three stability fixes were each derived from a property of the SIR generator itself – do they still work on data no compartmental model produced?
Expected	Only the unit-rescaling fix, which assumes nothing about the data’s structure, should transfer; the two structural fixes should not.
Observed	One in three predictions was wrong: the vanishing-field gate transfers and is the best single option, despite being predicted inapplicable.

Table 27. Which of SIR’s three fixes transfer to the non-mechanistic curve. All factors compare best training MSE at $s = 24$ against the time-rescaling-only arm trained for the same number of epochs: 2,000 epochs for the projection (6.80×10^{-2} against 2.74×10^{-3}) and 5,000 epochs for the gate (4.67×10^{-5} against 2.20×10^{-3} ; in extrapolation MSE the gate’s advantage is $26\times$, 0.155 against 4.06).

Fix	Precondition	Holds?	Outcome
Time rescaling	None – a unit change	Yes	Removes the $263,049\times$ spike ratio (maximum over median gradient norm)
Conservation via subspace projection	An exact invariant, $\sum y = c$	No ($\sum y$ spans $0.002\text{--}1.54$)	Inapplicable; $25\times$ worse training fit than rescaling alone
Vanishing-field gate on I	$I = 0$ is a manifold of equilibria	Yes (unexpectedly)	Best single option; $47\times$ better training fit, $26\times$ better extrapolation than rescaling alone

Table 27 summarizes the outcome. The vanishing-field gate was *predicted* inapplicable: the infected fraction at $t = 0$ is 0.0027, seemingly too small to avoid freezing the field entirely. A control arm overturns that prediction: gating the learned field by the current infected fraction, $\mathbf{f} \leftarrow I \cdot \mathbf{f}$, correctly encodes that the rate of change is proportional to current prevalence, and is the only configuration that reproduces the existence of a peak (day 48 predicted vs. true day 45) from a training window that never observed one.

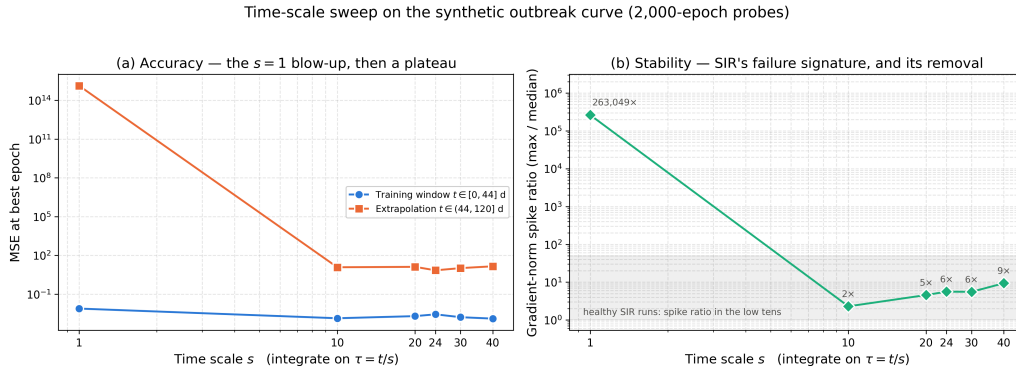


Figure 31. Time-rescaling sweep: accuracy and gradient-spike ratio vs. the rescaling factor.

The extrapolation runaway traces to the train/test split. The default split lands exactly on the infected peak (0% of the decay phase is in the training window), so a model has to infer a decay it has never observed. The rescaling-only model ($s = 24$, no gate, 2,000 epochs) fails to: its predicted infected curve never turns over and reaches $6.18\times$ the true peak (extrapolation MSE 6.99). The gate model above is the exception that the structural prior makes possible. The split controls retrain the same rescaling-only configuration with the cut moved later. Moving it just 15 days, to day 60, collapses the predicted maximum from 6.18 to 0.964 (true maximum = 1.0), turning over at day 47 against a true day 45 (Figure 32, right). So the runaway *peak* is a property of where the window ends. The tail is not fixed: past day 70 that same model's prediction drops steeply below zero, and its extrapolation MSE is still 0.397. Seeing the peak is enough to place it, but not enough to learn how the decay ends.

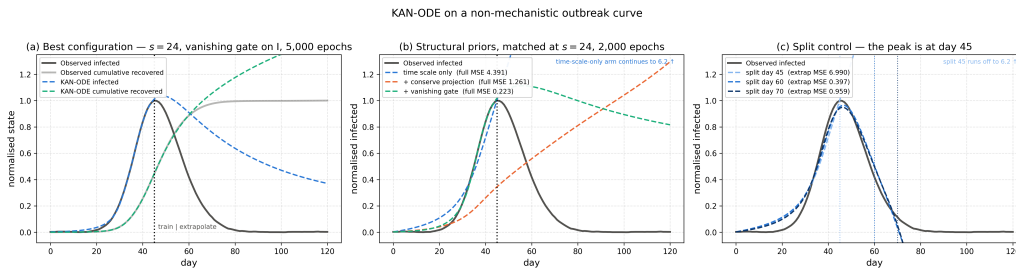


Figure 32. KAN-ODE on the synthetic, non-mechanistic outbreak curve. Left: the best configuration ($s = 24$, vanishing gate on I , 5,000 epochs). Middle: the structural priors compared at $s = 24$ and 2,000 epochs. Right: moving the train/test split from day 45 to day 60 or 70.

Verdict

Time rescaling transfers unconditionally because it is a unit change with no structural precondition. The other two fixes are structural priors that transfer exactly insofar as their structure is actually present in the new data: the conservation projection's precondition is confirmable in advance by directly measuring the compartment sum, while the vanishing-field gate's precondition turned out to require running the control arm to settle — argument alone predicted the wrong outcome. A single 2,000-epoch control arm overturned the prediction.

12 Validation: Epoch Budget, Implementation Audit and Paper Claims

Every comparison so far was made at a fixed 10,000-epoch budget and under this project's own recipe. This section tests both choices. It retrains the configurations whose ranking matters most at up to five times the

budget, compares this project’s recipe field by field with the base paper’s text and code, and sets each of the paper’s claims next to what this project measured.

12.1 Epoch-budget sensitivity

Question	Are the rankings in Tables 5–8 a property of the configurations, or of stopping at 10,000 epochs?
Expected	Every configuration was still improving at 10,000 epochs, so absolute errors should fall; the ranking should hold if 10,000 epochs was long enough to separate them.
Observed	The ranking among KAN variants mostly holds. The KAN-versus-MLP comparison reverses: at 50,000 epochs (with gradient clipping, on a different platform) the parameter-matched MLP extrapolates $6.8\times$ better than KAN-ODE.

Five configurations were chosen to cover the comparisons a reader would question first: Euler (the one solver that does not reach 10^{-4} training loss within 10,000 epochs), Tsit5 with RBF (the reference point every other entry is compared against), the cubic B-spline (statistically tied with RBF in Table 7), and both MLP baselines. Each was retrained from scratch on cloud CPUs with the same architecture, data, solver, learning rate and seed. Three things changed, not one. The obvious one is the number of epochs. The second is gradient-norm clipping at 1.0, which the training code had adopted as its default after the first cross-domain runs exploded (Sections 3 and 6), whereas the original 10,000-epoch Lotka-Volterra runs predate it and were unclipped. The third is the platform: the original runs used Windows with PyTorch 2.5.1 (the SiLU MLP, PyTorch 2.13.0 on CPU), the reruns Linux with PyTorch 2.10.0 on CPU, and the same seed on a different platform already changes a run’s result. Clipping engaged on 4.3% of steps for Tsit5/RBF, 4.7% for Euler, 2.2% for the B-spline, 11% for the SiLU MLP and 99.3% for the paper-specification MLP. Its effect was not measured, so the comparison below is not a pure epoch-budget comparison, and the paper-specification MLP’s extended run is in effect a different optimizer. Four runs went to 50,000 epochs ($5\times$ the original budget). The B-spline ran to 25,000 ($2.5\times$), because an early timing measurement projected a 50,000-epoch run beyond the length of one compute session. That estimate later proved $1.6\times$ too pessimistic, but the shorter budget was kept and every B-spline comparison below is 10,000 against 25,000 epochs.

Table 28. The same five configurations at the original and the extended budget (Lotka-Volterra; best-training-loss checkpoint, as throughout).

Configuration	Epochs	Training MSE	Extrapolation MSE	Extrap. R^2	L
Euler, RBF	10k	1.38×10^{-4}	2.51×10^{-3}	0.99914	8.55
	50k	8.85×10^{-6}	1.93×10^{-4}	0.99993	7.72
Tsit5, RBF	10k	8.83×10^{-5}	8.92×10^{-5}	0.99997	6.91
	50k	6.59×10^{-6}	4.02×10^{-5}	0.999986	7.75
Tsit5, B-spline	10k	7.20×10^{-5}	8.38×10^{-5}	0.99997	8.44
	25k	9.65×10^{-6}	4.26×10^{-5}	0.999985	7.05
MLP-ODE, SiLU	10k	9.50×10^{-5}	4.77×10^{-4}	0.99984	5.07
	50k	1.82×10^{-6}	5.90×10^{-6}	0.999998	5.65
MLP-ODE, tanh (paper)	10k	1.08	1.69	0.42	38.2
	50k	1.01	4.32	−0.48	104.8

Table 29. Ranking by extrapolation MSE before and after the extended budget (the paper-specification MLP is last at both budgets and is omitted).

Rank	At 10,000 epochs	At 25,000–50,000 epochs
1	B-spline (8.38×10^{-5})	MLP-ODE, SiLU (5.90×10^{-6})
2	Tsit5, RBF (8.92×10^{-5})	Tsit5, RBF (4.02×10^{-5})
3	MLP-ODE, SiLU (4.77×10^{-4})	B-spline (4.26×10^{-5})
4	Euler (2.51×10^{-3})	Euler (1.93×10^{-4})

Tables 28 and 29 give three separate answers.

The comparisons among KAN variants mostly hold. At 10,000 epochs RBF and B-spline are 6% apart in extrapolation error. The extended runs are 6% apart as well, but that compares B-spline at 25,000 epochs with RBF at 50,000, so it is not a matched comparison. At a matched 25,000 epochs, only the training loss can be compared exactly, because the RBF run saved a checkpoint only at its best epoch overall. There, from the RBF run’s own training history, the B-spline is ahead: 9.65×10^{-6} against 1.28×10^{-5} , 24% lower. So the two bases remain close, within a factor of two on every exactly comparable metric, but “statistically tied at every budget” is not established; with one seed, neither is a B-spline lead. Euler stays last. Its gap to Tsit5 narrows from $28\times$ to $4.8\times$, so part of Table 5’s Euler penalty was slower convergence rather than a permanent floor, but a first-order integrator remains measurably worse after five times the training. These checks cover the two ends of the solver table; the middle-order solvers were not retrained.

The KAN-versus-MLP comparison reverses. The parameter-matched MLP improves $81\times$ in extrapolation error, against KAN-ODE’s $2.2\times$, and ends $6.8\times$ ahead on the near window and $10.8\times$ ahead on the far window (Table 15). The KAN-ODE advantage of Tables 8 and 14 is therefore an advantage in how quickly it reaches a well-extrapolating field, not in the best field it can reach on this problem. Two confounds remain: clipping acted on 11% of the MLP’s steps but only 4.3% of the KAN’s, and neither its effect nor the platform’s was separated from that of the longer budget. A 50,000-epoch unclipped rerun of both on the same platform would settle it.

The paper-specification MLP is not an epoch-budget problem. Given five times the budget, the $[2, 50, 2]$ tanh network does not converge: its training loss barely moves ($1.08 \rightarrow 1.01$), its extrapolation gets worse (R^2 from 0.42 to -0.48), its Lipschitz estimate nearly triples to 104.8, and gradient clipping engages on 99.3% of steps (its median gradient norm before clipping is 19.8). More epochs, with clipping, do not rescue it: its failure under this protocol comes from the recipe, not the budget. The audit below identifies which parts of the recipe differ from the paper’s.

Training histories. Figure 33 shows the best training loss of the four 50,000-epoch runs. The SiLU MLP leads for the first $\sim 2,500$ epochs; the KAN-ODE then leads briefly while the MLP stalls near 2×10^{-3} , and from about epoch 4,800 the MLP stays ahead. Its lead grows to about $3.5\times$ by epoch 20,000 and then stays roughly constant; both are still descending at 50,000 epochs. The histories also make the clipping and platform confound visible. By epoch 10,000 the clipped reruns had already reached a lower training loss than the original unclipped 10,000-epoch runs: 3.85×10^{-5} against 8.83×10^{-5} for the KAN-ODE, and 2.24×10^{-5} against 9.50×10^{-5} for the MLP. So the reruns differ from the originals already within the original budget, not only beyond it.

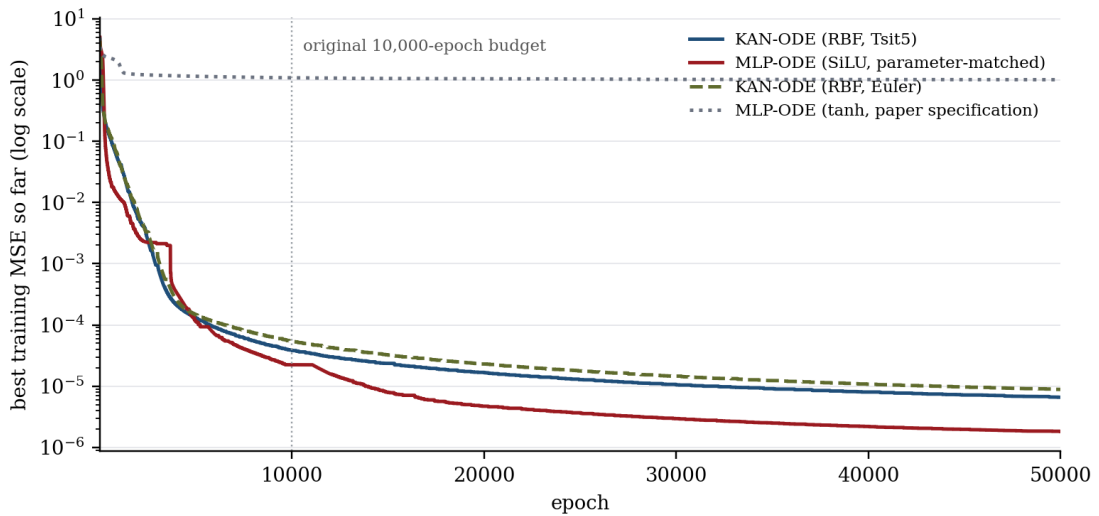


Figure 33. Best training MSE so far over 50,000 epochs for the four 50,000-epoch runs (all with gradient clipping at 1.0); the B-spline (25,000 epochs) is not shown. The vertical dotted line marks the original 10,000-epoch budget.

Cost. The five runs took 23.2 hours of training time in total (the sum of their wall-clock times), with 10.7 hours for the longest run. On the cloud CPUs, four of the five ran $2.2\text{--}2.5\times$ slower per epoch than on the development machine used to plan them; the SiLU MLP ran at about the same speed. A five-epoch timing sample overestimated the steady-state time per epoch by $1.6\times$ (2.52 against 1.54 s) because it was dominated by start-up cost. Budget estimates for runs like these should be calibrated on a few hundred epochs.

12.2 Implementation audit against the base paper

The paper’s Lotka-Volterra recipe was read from its text [11] and from the driver script and layer code in its official repository [12]. The repository’s own README notes that its scripts were re-run shortly before release and “are not the exact runs reported in the manuscript”, so where text and code disagree, both are listed. Table 30 compares every setting that could change a result.

Table 30. Base-paper recipe for Lotka-Volterra against this project’s. Verdicts: *match*; *explained* (a deliberate difference whose effect is measured in this report); *open* (a difference whose effect has not been measured).

Setting	Base paper (text / code)	This project	Verdict
System, parameters, $\mathbf{u}_0$	same system; $\mathbf{u}_0 = (1, 1)$	same	match (the paper swaps the names of γ and δ)
Time window, spacing	$[0, 14]$, train on $[0, 3.5]$, spacing 0.1	same	match; the paper’s code keeps 35 training points ($t \leq 3.4$), this project 36 ($t \leq 3.5$)
Ground truth	Tsit5, tolerance 10^{-12}	eighth-order Dormand-Prince, 10^{-12}	match in effect (both far below model error)
KAN architecture	$[2, 10, 2]$, $G = 5$, 240 parameters	same	match
Grid, normalizer, residual	G centres on $[-1, 1]$; tanh on basis path; SiLU (Swish) on the raw input	same	match
RBF kernel	text: $e^{-r^2/2\delta^2}$; code: e^{-r^2/δ^2}	$e^{-r^2/2\delta^2}$ (the text)	open: this project’s kernels are $\sqrt{2}$ wider than the paper’s code
Initialization	Glorot uniform, then all parameters divided by 10^5	Glorot uniform, not rescaled	open: the paper’s field starts near zero, this project’s does not (Section 6, SIR)
Optimizer	Adam, constant rate	Adam, constant rate	match
Learning rate	text: not stated in version 3 (version 1: baseline 10^{-3}); code: 5×10^{-4} (KAN), 10^{-2} (MLP)	2×10^{-3} for both	open, and the most likely reason the paper’s MLP trains there but not here
Epochs	10^5	10^4 ; five configurations at $2.5\text{--}5 \times 10^4$	explained (Table 28)
Training solver	adaptive Tsit5, default tolerances	fixed-step Tsit5, $h = 0.05$	explained (Table 5; truncation error is below the optimization noise)
Gradient method	adjoint sensitivity (interpolating adjoint, library default)	direct autograd	explained (Section 7: gradients agree to $\sim 10^{-7}$)
Regularization	none for the main run; L_1 on all trainable parameters, $\gamma_{\text{sp}} = 5 \times 10^{-4}$ (the paper’s Eq. 12; Eq. 33 here) for the sparse run	none	explained (Section 11, pruning)
MLP baseline	$[2, 50, 2]$, tanh, 252 parameters	same, plus a parameter-matched SiLU MLP	match; the SiLU MLP is an addition
Seed	0	42	match in kind (single seed in both)

Most settings match or differ for a reason whose effect this report measures. Three do not: the RBF width, the initial-weight rescaling and the learning rate. None was tested at the paper’s values (the learning rate was varied only downward, to 5×10^{-4} , in Table 9), and the last two plausibly explain why the paper’s $[2, 50, 2]$ tanh MLP trains in the paper and fails here, since a tanh network started at full Glorot scale and trained at a fifth of the paper’s MLP learning rate is a different optimization problem. Retraining the paper-specification MLP with the paper’s learning rate and initialization is the direct test, and it is the most important open item this audit leaves.

12.3 The paper’s claims against this project’s measurements

Table 31 sets each quantitative Lotka-Volterra claim of the base paper next to the corresponding measurement in this report. One claim is reproduced outright: the 240-parameter KAN-ODE does learn the dynamics from a single training cycle and extrapolates with R^2 above 0.9999. One is closely matched: KAN-ODE is a few times

slower per epoch than the paper’s MLP, by $2.3\times$ here against the paper’s $2.7\times$ (19 against 7 minutes per 10^4 epochs), just below the paper’s stated $2.5\text{--}3\times$ range.

The paper’s central efficiency claim, that KAN-ODE needs about $10\times$ fewer epochs than a similarly sized MLP, is not reproduced: under this protocol the parameter-matched MLP reaches 10^{-3} training loss $4.1\times$ *sooner*. In the clipped reruns the KAN-ODE reaches 10^{-3} first (epoch 2892 against 3676) but 10^{-4} later (5695 against 5225); neither ratio is near $10\times$. The paper’s accuracy claim holds only at the shorter budget. KAN-ODE extrapolates $5.3\times$ better at 10,000 epochs, and $6.8\times$ worse at 50,000 (with the clipping and platform caveat above). The paper’s own MLP result could not be reproduced at all, because its $[2, 50, 2]$ tanh network does not train under this recipe. The audit (Table 30) points to the learning rate and the initial-weight scale as the likely reasons, so this row is a statement about the recipe rather than evidence against the paper. The paper itself quotes this MLP’s final training loss inconsistently: as 3.0×10^{-5} in the text of its Section III.1.1 and as 4.1×10^{-5} in its Table 1 (width-50, 252-parameter row; arXiv version 3 [11]). Finally, the paper’s interpretability claim rests on L_1 -regularized training and pruning, which this project did not run, so it is neither confirmed nor refuted here.

Table 31. The main quantitative Lotka-Volterra claims of the base paper, and what this project found under its own protocol (single seed).

Paper’s claim	This project’s measurement	Verdict
A 240-parameter KAN-ODE learns the dynamics from $[0, 3.5]$ and extrapolates to $t = 14$	Extrapolation $R^2 = 0.99997$ at 10k epochs, 0.999986 at 50k (Tables 5, 28)	reproduced
KAN-ODE reaches 2.6×10^{-5} training loss within 10^4 epochs	8.8×10^{-5} at 10^4 epochs; 6.6×10^{-6} at 5×10^4	same order; slower here
KAN-ODE needs about $10\times$ fewer epochs than a similar MLP	The parameter-matched MLP reaches 10^{-3} $4.1\times$ <i>sooner</i> and 10^{-4} <i>sooner</i> as well (Table 8); in the clipped 50k reruns KAN-ODE reaches 10^{-3} first (2892 vs. 3676) but 10^{-4} later (5695 vs. 5225)	not reproduced
KAN-ODE iterates $2.5\text{--}3\times$ slower per epoch than the MLP (19 vs. 7 min per 10^4 epochs)	$2.3\times$ slower than the same MLP (0.1705 vs. 0.0736 s per epoch)	close ($2.3\times$ vs. the paper’s $2.7\times$)
KAN-ODE generalizes better than a similarly sized MLP	At 10k epochs: $5.3\times$ lower extrapolation error. At 50k epochs with gradient clipping, on a different platform: $6.8\times$ <i>higher</i> (Table 28)	depends on budget (and clipping, platform)
The paper’s $[2, 50, 2]$ tanh MLP reaches 3×10^{-5} training loss in 10^5 epochs	Fails to train at 10^4 and 5×10^4 epochs under this recipe (training loss ≈ 1)	not reproduced; recipe differs (Table 30)
Sparse, prunable edges recover a readable symbolic law	Not tested with sparsity; without it, pruning the spline branches of 2 of 40 edges destroys the fit (Section 11)	not tested as specified

Status

The epoch-budget reruns are complete. The implementation audit is complete as a comparison of settings, but none of the differences it found has been tested by retraining. Together they show three things. The ordering among KAN variants (Euler last, RBF and B-spline close) survives a longer budget. The KAN-ODE’s extrapolation advantage over a parameter-matched MLP holds at 10,000 epochs but not at 50,000, where gradient clipping and the platform also changed. And three recipe differences from the paper (learning rate, initial-weight scale and RBF width) remain unmeasured. Almost every result is still single-seed.

13 Discussion

Seven patterns recur across studies that were designed independently and address unrelated questions. Their recurrence is itself informative about how this class of model should be studied going forward.

A reasoned hypothesis still requires a control arm. Three separate studies advanced a specific, mechanistically plausible hypothesis and then tested it directly: gradient-norm roughness as the cause of Euler’s convergence floor (Section 8), SiLU’s representational limit as the cause of the pendulum’s early failure (Section 6), and the predicted inapplicability of the vanishing-field gate to non-mechanistic data (Section 11). Each hypothesis had a clear argument behind it before any run was launched, and each was overturned by the run

itself. The result reported in every one of these three cases is the outcome of the control arm, and the original argument survives only as the motivation for running it. This is not a coincidence of three unlucky guesses; it reflects the difficulty of reasoning about a system where the integrator, the optimizer, and the learned function interact inside the same backpropagation graph. The standard practice of ablating one variable while holding others fixed rests on an assumption of near-independence that does not hold here.

A single seed and a single train/test split account for results that first looked like properties of the solver or the method. The damping-sweep failures at $\mu \in \{0.5, 1.0, 2.0\}$ (Section 10) and the epidemic-fit extrapolation runaway (Section 11) both appeared, on first inspection, to indict a solver or a modeling approach. Retraining with additional seeds, in the damping-sweep case, and shifting the train/test split by fifteen days, in the epidemic case, resolved the first into seed-dependent local minima of training, apart from one Euler-specific failure at $\mu = 1.0$ ($\mu = 0.1$ fails for every seed and solver, because its trajectory never settles within the training window), and the runaway peak (though not the decay tail) into an artifact of the window boundary. Neither observation survives as a claim about solver order or about KAN-ODE’s capacity once the seed or the split is varied. Almost every quantitative result in this report is $N = 1$; the damping study’s small extension to $N = 3$ was enough to dissolve a headline finding, which underscores how much of the remaining project rests on the same single-seed foundation.

A fixed training budget is itself an experimental variable. Comparing models at one epoch count answers “which is better after this much training”, not “which is better”. The two questions largely agreed for the comparisons among KAN variants: with $2.5\text{--}5\times$ the budget, RBF and B-spline stayed within a factor of two of each other and Euler stayed last (Section 12). They disagreed for the comparison that matters most to the base paper: the KAN-ODE’s $5.3\times$ extrapolation lead over a parameter-matched MLP at 10,000 epochs became a $6.8\times$ deficit at 50,000. The longer runs also used gradient clipping, which the 10,000-epoch runs did not, and ran on a different platform (operating system and PyTorch version), so that reversal is a combined effect of budget, clipping and platform that this project did not separate. Rankings between models of different families, which converge at different rates, need to be checked at more than one budget, under otherwise identical settings, before they are read as properties of the models.

Measuring a quantity before training is more informative than tuning after a failed run. The SIR divergence (Section 6), the non-mechanistic epidemic fit (Section 11), and the damping sweep’s spurious equilibrium (Section 10) were each root-caused by evaluating the model’s own initial field, or its trained field at a candidate equilibrium, and comparing that number against the system’s true scale. In every case this produced a specific, falsifiable prediction – that time must be rescaled, that a given fix should or should not transfer, where a spurious equilibrium lies – that a subsequent short run or a direct computation confirmed or refuted, in contrast to the open-ended search a hyperparameter sweep would have required. The measurement did not always supply the numbers as well: the rescaling factors came from a separate horizon rule, not from the measured mismatch (Section 11).

Training accuracy and generalization quality are decoupled. On the pendulum, the hybrid basis matches pure RBF’s training loss to 1.7% at 10,000 epochs yet collapses in extrapolation ($R^2 = -2.17$ vs. $+0.647$, Section 9). The damping sweep’s trapped models (Section 10) reach training losses of 1.8×10^{-2} – qualitatively acceptable on a casual inspection – while their learned field is far from zero at the true equilibrium ($|\mathbf{f}_\theta(0,0)| \approx 2.7\text{--}2.9$) and has its fixed point elsewhere. And the rescaling-only epidemic model (Section 11) produces a plausible training fit while predicting an infected peak six times the physical maximum. In all three cases, a reader who judged the model solely by training-window MSE would reach the wrong conclusion. The evaluation protocol established in Section 5 – scoring strictly on extrapolation, never on training loss (which is used only to select the checkpoint) – was adopted precisely because this decoupling was anticipated, but these studies go further: they show that configurations with numerically indistinguishable training fits can differ by nearly an order of magnitude in extrapolation error (the pendulum hybrid and pure RBF, 0.809 against 0.090, from training losses 1.7% apart). The number to watch is always the one computed outside the training window, and no training-window metric tested in this project reliably predicts it.

On Lotka-Volterra, extra cost did not buy accuracy. Across the solver ablation (Section 6), the basis ablation, and the hybrid-basis study (Section 9), the same pattern appears on Lotka-Volterra at one seed. The cheapest methods that work (Midpoint at $1/3$ the function evaluations of Tsit5; RBF at $1/3.9$ the wall-clock of B-spline) match their more expensive alternatives about as closely as two runs at one seed can be told apart; the $3.5\times$ gap between Midpoint and Heun, two methods of equal cost, shows how large that single-seed spread can be. Adding cost, whether more function evaluations per step, more basis evaluations per edge, or a second basis evaluated in parallel, did not buy a measurable accuracy improvement in these runs. The hybrid basis is the clearest case, on both systems it was tried on: the blend pays roughly the sum of both bases' per-epoch compute and never recovers that cost in accuracy. Whether the pattern holds at more seeds, or on the other systems for the solver and basis ablations, was not tested.

These KAN edges do not produce interpretable symbolic structure without explicit encouragement. The pruning experiment (Section 11) found that zeroing the spline branches of the two weakest of forty edges degrades a noise-free model to worse than a constant predictor – a factor of 3.9×10^4 . The fit is spread across many edges, so no small set of edges can be read off as the terms of the governing equation. This is consistent with training without sparsity-inducing regularization, though the regularized retraining that would confirm it was not run, and it means the interpretability advantage that motivated the original choice of a smooth, globally-supported basis function remains a hypothesis rather than a demonstrated property of the trained models in this project. A future retraining with explicit edge-level regularization would be needed to test whether sparse, readable edge functions emerge when the training loss actively asks for them.

14 Conclusion

This project reimplemented a published KAN-ODE recipe from its components (a KAN layer with seven interchangeable bases, six explicit Runge-Kutta integrators and an instrumented training loop), verified them against known solutions, and reproduced the recipe on the Lotka-Volterra predator-prey system. It extended the recipe to two additional dynamical systems (a damped pendulum and an SIR epidemic model), diagnosed and fixed two genuine cross-domain training failures before running five further studies that each isolated a specific question about solver behavior, gradient dynamics, basis-function design, damping (originally intended as a stiffness probe), and comparison against a fundamentally different modeling paradigm.

The principal quantitative findings are:

- At 10,000 epochs, KAN-ODE extrapolation error on Lotka-Volterra is essentially flat out to twice the original horizon ($1.1\times$ degradation), while a parameter-matched MLP degrades $8.3\times$ – a $40.6\times$ KAN-ODE advantage (Table 14). At 50,000 epochs, with gradient clipping and on a different platform (Section 12), this reverses, and the MLP is $6.8\text{--}10.8\times$ more accurate at every horizon (Table 15). The KAN-ODE reaches a well-extrapolating field sooner; it does not reach a better one on this problem.
- Of the base paper's Lotka-Volterra claims, the 240-parameter model's accuracy is reproduced and its per-epoch cost relative to an MLP closely matched ($2.3\times$ against the paper's $2.7\times$); the tenfold epoch advantage is not (the MLP reaches 10^{-3} loss $4.1\times$ sooner; in the clipped 50,000-epoch reruns the KAN-ODE reaches 10^{-3} first but 10^{-4} later, neither by a factor near ten), and the generalization advantage holds only at the shorter budget (Table 31).
- SIR's training failure traced to a roughly $18\text{--}19\times$ unit-scale mismatch between the network's initial field and the true dynamics. A change of time units and a conservation projection, which assume nothing beyond SIR's own equations, remove the divergence and the mass drift but leave extrapolation R^2 at -0.145 (from -20.97); adding a vanishing-field gate on the infected fraction, a physical prior, raises it to $+0.9981$.
- The implementation is verified independently of the benchmarks: every integrator reaches its theoretical order of convergence (Table 3), and 249 of 250 automated tests pass; the remaining one, which needs a GPU, is skipped.
- Direct autograd is the correct default for this project's scale: adjoint sensitivity already uses less memory at every tested trajectory length but costs a consistent $1.9\text{--}2.1\times$ wall-clock overhead.
- Gradient-norm roughness does not explain Forward Euler's convergence floor (Spearman $\rho = -0.088$, $p = 0.87$); the more likely mechanism is a systematic bias in the learned field, consistent with the solver ablation's own Lipschitz-constant measurements.

- A learnable blend of B-spline and RBF basis functions never outperforms the better pure basis on Lotka-Volterra, or pure RBF on the pendulum (no pure B-spline pendulum run was made), and measuring each basis’s actual contribution shows the learned gate is a fair guide to how much each is used.
- The damping sweep never reached a stiff regime, and no cell diverged, as expected from the small step size ($|h\lambda| \leq 0.30$), which keeps every solver’s discrete map close to the learned field’s own flow (the only predicted instability is a mild anti-damping for Euler at $\mu = 0.1$). Its failures at $\mu \in \{0.5, 1.0, 2.0\}$ trace to random initialization rather than solver order, apart from one Euler-specific failure at $\mu = 1.0$, confirmed at $\mu = 2.0$ by a genuine, attracting spurious equilibrium that Newton’s method locates in all four solvers’ learned fields.
- With finite-difference derivatives, sparse regression is less accurate than KAN-ODE at every noise level but degrades far less ($9\times$ vs. $1540\times$ from clean to $\sigma = 0.1$, starting from a much higher noise-free error). With exact derivatives it recovers the true equation, so on clean data the ranking reflects the derivative estimator, not the method. Two of the SIR stability fixes transfer to a synthetic, non-mechanistic epidemic curve; the one that transfers best was predicted inapplicable.

Every quantitative claim in this report traces to a saved metrics file, a regenerable figure, or a stated post-hoc analysis of a saved checkpoint, and every study that tested a specific mechanistic hypothesis reports the outcome of a direct control arm rather than the hypothesis alone.

14.1 Limitations

The conclusions above are bounded by the following, each stated where it applies and collected here.

- **Single seed.** Almost every result rests on one random seed. The damping study, where extending from one seed to three dissolved a headline finding, and the $3.5\times$ gap between the two second-order solvers (Table 5), show how large single-seed variance can be.
- **Unmeasured recipe differences.** The audit (Table 30) found three differences from the paper’s code whose effect has not been measured: the learning rate, the initial-weight scale and the RBF width. None was tested at the paper’s values (the learning rate was varied only downward, to 5×10^{-4}). Retraining the paper-specification MLP with the paper’s learning rate and initialization is the single most informative next run, because it would show whether that MLP’s failure here comes from the recipe or from the architecture.
- **Budget, clipping and platform.** The KAN-versus-MLP reversal at 50,000 epochs coincides with the introduction of gradient clipping and a change of platform (Windows with PyTorch 2.5.1 for the original runs, 2.13 for the SiLU MLP; Linux with PyTorch 2.10 for the reruns), and neither was separated from the budget. Only five configurations were retrained at the longer budget.
- **Scope of the ablations.** The solver, basis and step-size ablations, the budget check and the sparse-regression comparison were run on Lotka-Volterra only.
- **No stiff system, no implicit solver.** The damping sweep’s largest timescale ratio is 4.3, and every solver is explicit, so this report says nothing about stiffness proper.
- **Synthetic data only.** Every dataset, including the non-mechanistic epidemic curve, is generated rather than recorded; no real observational data was used.
- **Derivative estimation in the baseline.** Sparse regression was fitted to finite-difference derivatives, which dominate its noise-free error (Table 25); a weak or trajectory-fitted formulation was not tried.
- **Hardware.** All timings are CPU timings on two different machines, except the GPU memory study, and are comparable only within a machine.

References

- [1] Steven L. Brunton, Joshua L. Proctor, and J. Nathan Kutz. Discovering governing equations from data by sparse identification of nonlinear dynamical systems. *Proceedings of the National Academy of Sciences*, 113(15):3932–3937, 2016.
- [2] Ricky T. Q. Chen. torchdiffeq, 2018. <https://github.com/rtqichen/torchdiffeq>.
- [3] Ricky T. Q. Chen, Yulia Rubanova, Jesse Bettencourt, and David Duvenaud. Neural ordinary differential equations. *Advances in Neural Information Processing Systems*, 31, 2018.
- [4] Jeremy M. Cohen, Simran Kaur, Yuanzhi Li, J. Zico Kolter, and Ameet Talwalkar. Gradient descent on neural networks typically occurs at the edge of stability. In *International Conference on Learning Representations*, 2021.
- [5] Jeremy M. Cohen, Behrooz Ghorbani, Shankar Krishnan, Naman Agarwal, et al. Adaptive gradient methods at the edge of stability, 2022. arXiv:2207.14484.
- [6] Brian M. de Silva, Kathleen Champion, Markus Quade, Jean-Christophe Loiseau, J. Nathan Kutz, and Steven L. Brunton. PySINDy: A Python package for the sparse identification of nonlinear dynamical systems from data. *Journal of Open Source Software*, 5(49):2104, 2020.
- [7] John R. Dormand and Peter J. Prince. A family of embedded Runge-Kutta formulae. *Journal of Computational and Applied Mathematics*, 6(1):19–26, 1980.
- [8] Ernst Hairer and Gerhard Wanner. *Solving Ordinary Differential Equations II: Stiff and Differential-Algebraic Problems*. Springer, 2 edition, 1996.
- [9] Ernst Hairer, Syvert P. Nørsett, and Gerhard Wanner. *Solving Ordinary Differential Equations I: Nonstiff Problems*. Springer, 2 edition, 1993.
- [10] Alan A. Kaptanoglu, Brian M. de Silva, Urban Fasel, Kadierdan Kaheman, et al. PySINDy: A comprehensive Python package for robust sparse system identification. *Journal of Open Source Software*, 7(69): 3994, 2022.
- [11] Benjamin C. Koenig, Suyong Kim, and Sili Deng. KAN-ODEs: Kolmogorov-arnold network ordinary differential equations for learning dynamical systems and hidden physics. *Computer Methods in Applied Mechanics and Engineering*, 432:117397, 2024. arXiv:2407.04192.
- [12] Benjamin C. Koenig, Suyong Kim, and Sili Deng. KAN-ODEs: official code repository, 2024. <https://github.com/DENG-MIT/KAN-ODEs>.
- [13] Hao Li, Zheng Xu, Gavin Taylor, Christoph Studer, and Tom Goldstein. Visualizing the loss landscape of neural nets. In *Advances in Neural Information Processing Systems*, volume 31, 2018.
- [14] Ziming Liu, Yixuan Wang, Sachin Vaidya, Fabian Ruehle, James Halverson, Marin Soljačić, Thomas Y. Hou, and Max Tegmark. KAN: Kolmogorov-arnold networks. *arXiv preprint arXiv:2404.19756*, 2024.
- [15] Vedant Puri. KolmogorovArnold.jl: Julia implementation of Kolmogorov-Arnold networks, 2024. <https://github.com/vpuri3/KolmogorovArnold.jl>.
- [16] Ch. Tsitouras. Runge-kutta pairs of order 5(4) satisfying only the first column simplifying assumption. *Computers & Mathematics with Applications*, 62(2):770–775, 2011.

A Butcher Tableaus of the Fifth-Order Solvers

A Butcher tableau lists the coefficients of Equation 11: the nodes c_i (left column, the fraction of the step at which stage i is evaluated), the stage weights a_{ij} (lower triangle) and the solution weights b_i (bottom row). Tables 32 and 33 list the two fifth-order methods used in this report. Both have seven stages with $b_7 = 0$; the seventh stage exists only for the embedded error estimate and is skipped by this project’s fixed-step integrator, so each costs six field evaluations per step. Coefficients are the ones used in this project’s code (DOPRI5 exact, Tsit5 to the precision given in [16], shown here to six decimals).

Table 32. DOPRI5 [7], fifth-order solution weights.

0							
1	1						
$\frac{1}{5}$	$\frac{1}{5}$						
$\frac{3}{5}$	$\frac{3}{5}$	9					
$\frac{10}{13}$	$\frac{40}{13}$	$\frac{40}{13}$					
$\frac{4}{5}$	$\frac{44}{5}$	$\frac{56}{5}$	32				
$\frac{5}{8}$	$\frac{45}{8}$	$\frac{15}{8}$	$\frac{9}{8}$				
$\frac{8}{9}$	19372	25360	64448	212			
$\frac{9}{10}$	6561	2187	6561	729			
1	9017	355	46732	49	5103		
1	3168	33	5247	176	18656		
1	35	0	500	125	2187	11	
	384		1113	192	6784	84	
b	35	0	500	125	2187	11	0
	384		1113	192	6784	84	

Table 33. Tsit5 [16], fifth-order solution weights.

0							
0.161	0.161						
0.327	−0.008481	0.335481					
0.9	2.897153	−6.359448	4.362295				
0.980026	5.325865	−11.748884	7.495539	−0.092495			
1	5.861455	−12.920969	8.159368	−0.071585	−0.028269		
1	0.096461	0.01	0.479890	1.379009	−3.290070	2.324711	
b	0.096461	0.01	0.479890	1.379009	−3.290070	2.324711	0

In both tableaus the last row of a_{ij} equals b : the final stage is evaluated at the new solution itself (“first same as last”), which is what makes it free to reuse as the next step’s first stage in an adaptive solver. The solution weights sum to one in both cases, the first of the order conditions, and this is one of the automated checks (Section 4).

B Damping-Study Trajectories

The two figures below expand the trajectory comparison of Section 10 at full page width. In each, the top row shows the predicted angle $\theta(t)$ and angular velocity $\omega(t)$ of every solver's model next to the true trajectory, the bottom left shows the magnitude of each solver's own learned field, as $\log(1 + |\mathbf{f}_\theta|)$, over (θ, ω) with that model's trajectory drawn on it, and the bottom right shows the logarithm of the pointwise error. Legends give each model's verdict under the rule of Section 10. The trajectories were recomputed from the saved seed-42 checkpoints, and each reproduces its recorded best training MSE.

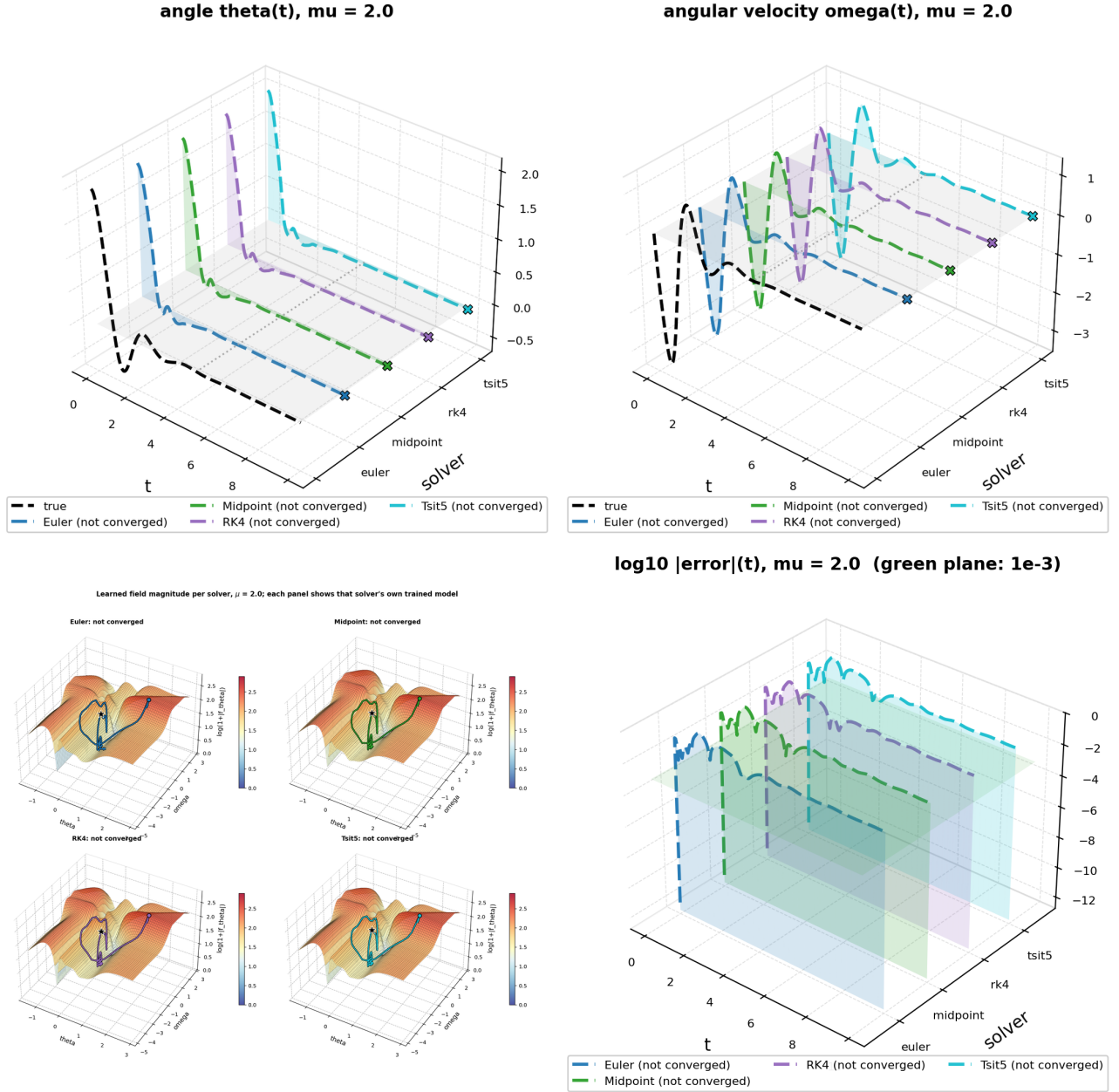


Figure 34. $\mu = 2.0$, all four solvers (seed 42). Every model settles onto the same wrong trajectory, which ends at the learned spurious equilibrium of Table 21, about 0.05 from the true one; the offset is visible in the error panel rather than in the trajectory panels.

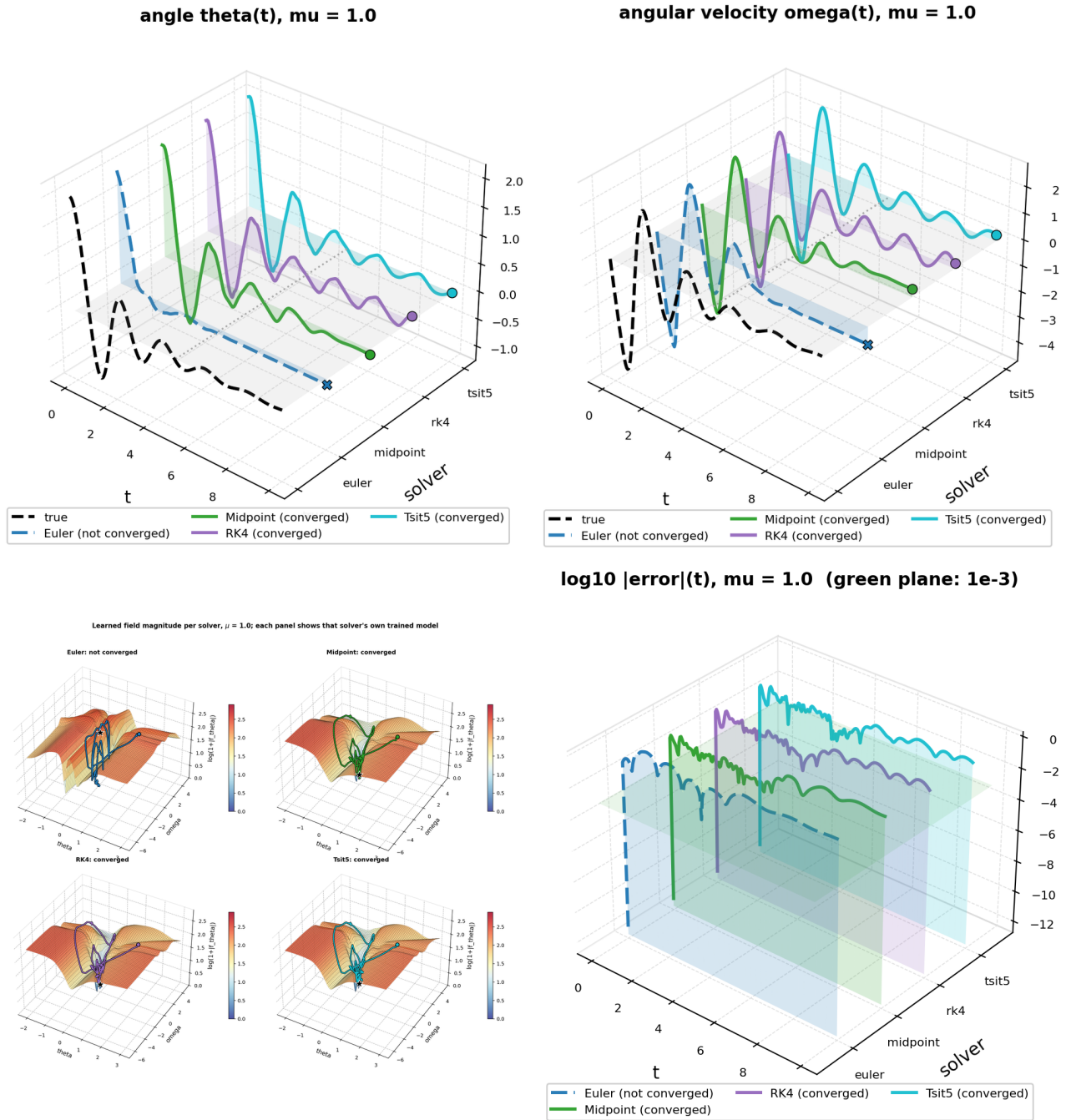


Figure 35. $\mu = 1.0$, all four solvers (seed 42), in the same layout. Only Euler's prediction (angle and angular velocity) departs visibly from the true trajectory; the other three track it closely.